

СПбГУ · МКН · СП · 1 КУРС

Математические ОСНОВЫ алгоритмов

Билеты

Лектор Косолобов Д. А.

Санкт-Петербург, 2026

Составители

координация проекта, техническая
редакция, вёрстка

Ярослав Власов
[@yarosvla](#)

проверка и научное редактирование

Никита Бекоев
[@vzialvsosh](#)

Александр Поданёв
[@alex-pods](#)

Кирилл Пирогов
[@NothingToThink](#)

Варвара Сарычева
[@Margaritca2008](#)

Валентина Млтыхан
[@robertvlasov](#)

Содержание

- 1 Дискретное преобразование Фурье (DFT), обратное DFT. Свёртки и умножение многочленов при помощи алгоритма FFT. Обобщения на другие кольца, NTT. Вычислительная схема FFT (преобразования бабочки). 3
- 2 Вычисления схемами как модель параллельных вычислений. Компараторы, сортирующие сети. Принцип нулей-единиц. Битонная сортировка и сортировка чётно-нечётным слиянием. 8
- 3 Префиксные суммы (scan), схема Ладнера–Фишера. Модель CRCW/EREW PRAM. Сортировка подсчётом в модели PRAM. 17
- 4 Трудоёмкость локальных вычислений, коммуникации и синхронизации при вычислениях в модели барьерно-синхронного параллелизма (BSP). Префиксные суммы в BSP, оптимальность. 22
- 5 Параллельное вычисление обобщённых линейных рекуррентных соотношений и параллельное сложение двоичных чисел при помощи задачи префиксного накопления. 26
- 6 Алгоритмы во внешней памяти. Сортировка слиянием. Сортировка связанного списка 28
- 7 Задача линейного программирования. Допустимые и оптимальные решения. Стандартная и каноническая формы. Существование оптимального решения (без доказательства). Лемма Фаркаша. 34
- 8 Теорема о двойственности в линейном программировании (без доказа-

тельности леммы Фаркаша). Системы линейных неравенств. Полиномиальные алгоритмы решения лин. прог. Общая идея алгоритма внутренней точки. Преобразование задачи для нахождения какой-то внутренней точки.	38
9 Задача о максимальном потоке в сети. Формулировка в лин. прог. Дополняющие пути. Алгоритм Форда–Фалкерсона. Анализ трудоёмкости для целочисленной задачи о максимальном потоке. Теорема о максимальном потоке и минимальном разрезе	42
10 Приближённые алгоритмы. Задача о вершинном покрытии минимальной мощности, приближённый полиномиальный алгоритм для её решения. Задача о вершинном покрытии минимальной стоимости, её приближённое решение при помощи лин. прог.	49
11 Метрическая задача коммивояжёра. Алгоритм двойного дерева и алгоритм Кристофидеса–Сердюкова для её приближённого решения	52
12 Вероятностные алгоритмы. Задача проверки умножения матриц. Алгоритм Фрейвальдса. Задача проверки равенства многочленов. Лемма Шварца–Зиппеля	57
13 Хеш-таблицы, ключи. Амортизированное и ожидаемое время работы. Время работы хеш-таблицы с открытой адресацией при идеализированном случайном равновероятном хешировании	61
14 Универсальные семейства хеш-функций. 2-универсальность семейства multiply-shift. Хеш-таблицы с разрешением коллизий цепочками	64
15 Совершенное хеширование. Двухуровневое совершенное хеширование	68
16 Задача проверки простоты числа. Тест Ферма. Числа Кармайкла. Алгоритм Миллера–Рабина	70
17 Онлайн-алгоритмы. Задача кэширования. Помечающий алгоритм	73
18 Задача нахождения минимального остовного дерева. Алгоритмы Ярника–Прима–Дейкстры и Краскала. Системы непересекающихся множеств (Union-Find)	78
19 Задача булевой выполнимости (SAT). Задачи k SAT и MAX- k SAT. Вероятностный и детерминированный приближённые алгоритмы для MAX-3SAT	83

1. Дискретное преобразование Фурье (DFT), обратное DFT. Свёртки и умножение многочленов при помощи алгоритма FFT. Обобщения на другие кольца, NTT. Вычислительная схема FFT (преобразования бабочки).

Свёртка и умножение многочленов

Определение

Пусть заданы последовательности чисел $a_0, \dots, a_{n-1}$ и $b_0, \dots, b_{n-1}$. Определим последовательность $c_0, \dots, c_{2n-2}$ по формуле

$$c_k = \sum_{t=0}^{n-1} a_t b_{k-t}, \quad k = 0, \dots, 2n-2,$$

(считаем $a_i = b_i = 0$ при $i \notin [0, n-1]$). Эта операция называется **конволюцией** (**сверткой**) и обозначается $c = a * b$.

Умножение многочленов

Рассмотрим многочлены

$$A(x) = a_0 + a_1x + \dots + a_{n-1}x^{n-1}, \quad B(x) = b_0 + b_1x + \dots + b_{n-1}x^{n-1}.$$

Тогда их произведение

$$C(x) = A(x)B(x) = c_0 + c_1x + \dots + c_{2n-2}x^{2n-2},$$

где коэффициенты c_k задаются конволюцией $c = a * b$. Таким образом, умножение многочленов и вычисление свёртки их коэффициентов — одна и та же задача.

Сложность

- Наивное умножение: $O(n^2)$.
- Карацуба: $O(n^{\log_2 3})$.
- FFT (быстрое преобразование Фурье): $O(n \log n)$.

Идея FFT-умножения

Хотим быстро вычислить свёртку (а значит, и произведение многочленов). Для этого:

1. Выбираем число точек N так, чтобы $N \geq 2n - 1$ и обычно $N = 2^t$. Коэффициенты A, B дополняем нулями до длины N .
2. Быстро считаем значения в N точках $A(\omega_k), B(\omega_k), k = 0, \dots, N - 1$, где ω — примитивный N -й корень из единицы.
3. Перемножаем покомпонентно: $y_k = A(\omega_k) B(\omega_k)$.
4. По значениям y_k восстанавливаем коэффициенты C обратным FFT.

Ключевой факт: можно выбрать точки ω_k так, что и «вычисление значений», и «восстановление коэффициентов» делаются за $O(N \log N)$.

Реализация FFT

Далее считаем, что $N = 2^t$. Если исходная длина не степень двойки, дополняем нулями.

Прямое FFT: из коэффициентов в значения

Выбираем

$$\omega = e^{2\pi i/N} = \cos \frac{2\pi}{N} + i \sin \frac{2\pi}{N}, \quad \omega_k = \omega^k.$$

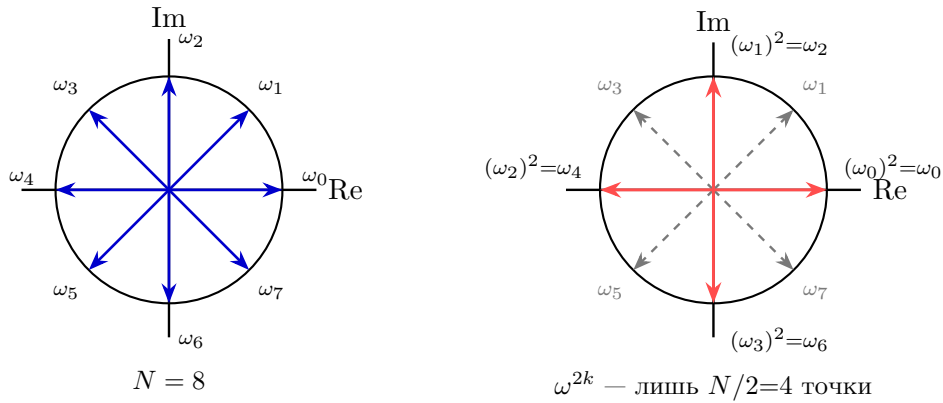


Рис. 1: Слева — корни из единицы $\omega_k = e^{2\pi i k/8}$ при $N = 8$. При возведении в квадрат восемь корней попарно сливаются: квадраты ω^{2k} (справа, красные стрелки) пробегают лишь $N/2 = 4$ различные точки. Именно это позволяет свести задачу размера N к двум задачам размера $N/2$. Серые пунктирные стрелки — нечётные узлы $\omega_1, \omega_3, \omega_5, \omega_7$.

Нужно вычислить

$$y_k = A(\omega^k) = \sum_{j=0}^{N-1} a_j (\omega^k)^j, \quad k = 0, \dots, N-1.$$

Разобьём A на чётные и нечётные коэффициенты:

$$A_0(x) = a_0 + a_2x + \dots + a_{N-2}x^{\frac{N}{2}-1}, \quad A_1(x) = a_1 + a_3x + \dots + a_{N-1}x^{\frac{N}{2}-1}.$$

Тогда $A(x) = A_0(x^2) + xA_1(x^2)$, и при подстановке $x = \omega^k$ получаем

$$A(\omega^k) = A_0(\omega^{2k}) + \omega^k A_1(\omega^{2k}).$$

Обозначим $y_k^{(0)} = A_0(\omega^{2k})$ и $y_k^{(1)} = A_1(\omega^{2k})$. Так как значения ω^{2k} пробегают только $N/2$ различных точек, достаточно посчитать $y_k^{(0)}, y_k^{(1)}$ для $k = 0, \dots, \frac{N}{2} - 1$ (рекурсивно), а затем «склеить» ответы:

$$y_k = y_k^{(0)} + \omega^k y_k^{(1)}, \quad y_{k+\frac{N}{2}} = y_k^{(0)} - \omega^k y_k^{(1)}.$$

(Здесь использовано $\omega^{k+N/2} = -\omega^k$.)

Оценка времени прямого FFT

На каждом уровне рекурсии решаем 2 подзадачи размера $N/2$ и делаем склейку за $O(N)$, поэтому $T(N) = 2T(N/2) + O(N) = O(N \log N)$.

Обратное FFT: из значений в коэффициенты

Прямое преобразование можно записать матрично:

$$\begin{pmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega^1 & (\omega^1)^2 & \dots & (\omega^1)^{N-1} \\ 1 & \omega^2 & (\omega^2)^2 & \dots & (\omega^2)^{N-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{N-1} & (\omega^{N-1})^2 & \dots & (\omega^{N-1})^{N-1} \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_{N-1} \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_{N-1} \end{pmatrix}.$$

Это матрица Вандермонда F_ω .

Утверждение 1.1

Выполнено $F_\omega F_{\omega^{-1}} = N \cdot \mathbf{I}$.

Доказательство. Элемент произведения в позиции (i, j) равен

$$\sum_{k=0}^{N-1} (\omega^i)^k (\omega^{-j})^k = \sum_{k=0}^{N-1} (\omega^{i-j})^k.$$

Если $i \neq j$, то $q := \omega^{i-j} \neq 1$, и по формуле суммы геометрической прогрессии $\sum_{k=0}^{N-1} q^k = \frac{1-q^N}{1-q}$ получаем

$$\sum_{k=0}^{N-1} (\omega^{i-j})^k = \frac{1 - (\omega^{i-j})^N}{1 - \omega^{i-j}} = \frac{1 - (\omega^N)^{i-j}}{1 - \omega^{i-j}} = \frac{1 - 1}{1 - \omega^{i-j}} = 0.$$

Если же $i = j$, то $\sum_{k=0}^{N-1} (\omega^{i-j})^k = \sum_{k=0}^{N-1} 1 = N$. ■

Следовательно, обратное преобразование — это прямое FFT с корнем ω^{-1} , после чего нужно поделить все коэффициенты на N : $a = \frac{1}{N} F_{\omega^{-1}} y$.

Сложность умножения многочленов через FFT

Нужно:

- одно прямое FFT для A ,
- одно прямое FFT для B ,
- покомпонентное умножение ($O(N)$),
- одно обратное FFT.

Итого $O(N \log N)$, где N — ближайшая степень двойки, не меньшая $2n + 1$. При этом $2n + 1 \leq N < 2(2n + 1) < 4n$, то есть $N = \Theta(n)$, поэтому итоговая сложность равна $O(n \log n)$.

Обобщение: NTT (Number-Theoretic Transform)

Вместо $\mathbb{C}$ можно работать в кольце/поле R , если:

- в R существует элемент ω порядка N (примитивный N -й корень из единицы);
- число N обратимо в R (нужно для деления на N в обратном преобразовании).

Пример (поле $\mathbb{Z}_p$)

Пусть $R = \mathbb{Z}_p$, где p — простое, и $N = 2^k$, $p = c \cdot 2^k + 1$. Тогда $p - 1$ кратно N , и существует первообразный корень g по модулю p (порождает мультипликативную группу порядка $p - 1$). Можно взять $\omega \equiv g^c \pmod{p}$; тогда ω имеет порядок N , и NTT работает полностью по модулю p .

Приложение: смысл коэффициентов DFT

Пусть дан сигнал (последовательность) $a_0, a_1, \dots, a_{N-1}$. Определим дискретное преобразование Фурье:

$$y_k = \sum_{t=0}^{N-1} a_t \omega^{kt}, \quad k = 0, \dots, N-1, \quad \text{где } \omega = e^{2\pi i/N}.$$

Так как $\omega^{kt} = \cos\left(\frac{2\pi k}{N}t\right) + i \sin\left(\frac{2\pi k}{N}t\right)$, то

$$\Re(y_k) = \sum_{t=0}^{N-1} a_t \cos\left(\frac{2\pi k}{N}t\right), \quad \Im(y_k) = \sum_{t=0}^{N-1} a_t \sin\left(\frac{2\pi k}{N}t\right).$$

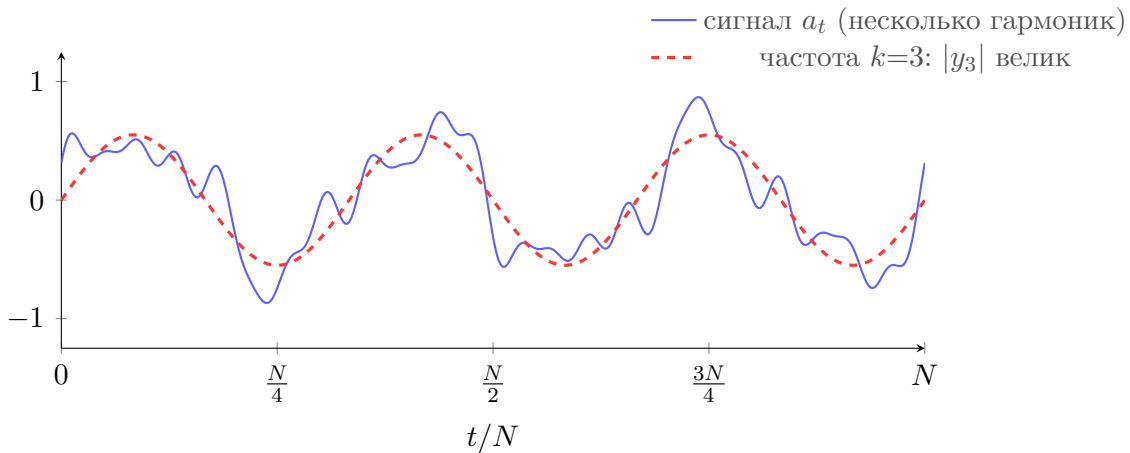


Рис. 2: Если в сигнале (синий) доминирует частота $k=3$, то коэффициент $y_3 = \sum_t a_t e^{2\pi i \cdot 3t/N}$ велик по модулю: вклады $a_t \cos \frac{2\pi \cdot 3t}{N}$ складываются (красный пунктир), а для других k — взаимно компенсируются.

Эти суммы — скалярные произведения сигнала с косинусом и синусом частоты k . Если в сигнале сильно выражена гармоника частоты k , то вклады «складываются», и модуль $|y_k|$ получается большим; если такой частоты нет — вклады в основном взаимно компенсируются и $|y_k|$ близок к нулю.

Вычислительная схема FFT: преобразование бабочки

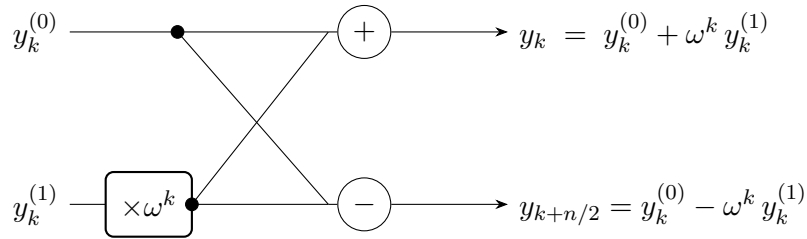


Рис. 3: Одна «бабочка» — элементарный шаг схемы FFT. Нижний входной провод умножается на ω^k ; затем два провода попарно складываются и вычитаются. Знак минус объясняется тем, что $\omega^{k+n/2} = -\omega^k$.

Рекурсивная схема FFT состоит из двух блоков $\text{FFT}_{n/2}$, к выходам которых добавляется один слой из $n/2$ бабочек: k -я бабочка умножает правый вход на ω^k , а затем складывает и вычитает.

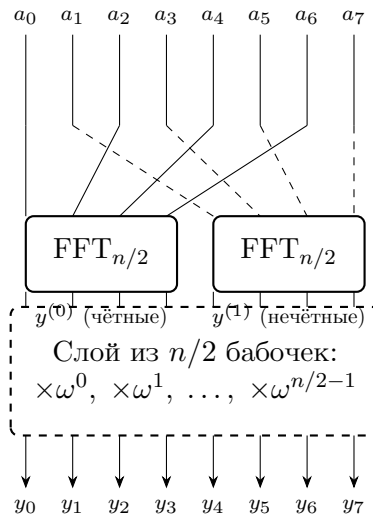


Рис. 4: Рекурсивная схема FFT (на примере $n = 8$): чётные коэффициенты a_0, a_2, a_4, a_6 поступают в левый блок $\text{FFT}_{n/2}$ (он даёт $y^{(0)}$), нечётные a_1, a_3, a_5, a_7 — в правый (он даёт $y^{(1)}$); затем слой из $n/2$ бабочек объединяет результаты в $y_0, \dots, y_{n-1}$.

Сложность

Для схемы FFT глубина есть $O(\log n)$, а размер — $O(n \log n)$. Глубина удовлетворяет рекуррентности $T(n) = T(n/2) + O(1)$, откуда $T(n) = O(\log n)$; размер удовлетворяет $S(n) = 2S(n/2) + O(n)$, откуда по мастер-теореме $S(n) = O(n \log n)$.

Предложение 1.1

Пусть $y = F_\omega a$, где

$$F_\omega = (\omega^{jk})_{j,k=0}^{n-1},$$

а ω — примитивный корень степени n из единицы. Тогда $F_\omega^2 = nP$, где P — перестановка $P(a_0, a_1, \dots, a_{n-1}) = (a_0, a_{n-1}, a_{n-2}, \dots, a_1)$.

Доказательство. Для любого $a = (a_0, \dots, a_{n-1})^T$ и любого $i = 0, \dots, n-1$ имеем

$$(F_\omega^2 a)_i = \sum_{j=0}^{n-1} \omega^{ij} \sum_{k=0}^{n-1} \omega^{jk} a_k = \sum_{k=0}^{n-1} a_k \sum_{j=0}^{n-1} \omega^{j(i+k)}.$$

Но

$$\sum_{j=0}^{n-1} \omega^{jt} = \begin{cases} n, & t \equiv 0 \pmod{n}, \\ 0, & t \not\equiv 0 \pmod{n}. \end{cases}$$

Следовательно, $(F_\omega^2 a)_i = n a_{-i \bmod n}$, то есть $F_\omega^2 a = n(a_0, a_{n-1}, a_{n-2}, \dots, a_1)^T = nPa$, откуда $F_\omega^2 = nP$.

Отсюда, если $y = F_\omega a$ и $\tilde{a} = F_\omega y$, то

$$\tilde{a} = F_\omega^2 a = nPa, \quad a = \frac{1}{n} P \tilde{a}.$$

■

Итак, чтобы восстановить a по y , достаточно:

1. вычислить $\tilde{a} = F_\omega y$;
2. обратить порядок координат $\tilde{a}_1, \dots, \tilde{a}_{n-1}$, оставив $\tilde{a}_0$ на месте;
3. разделить все координаты на n .

2. Вычисления схемами как модель параллельных вычислений. Компараторы, сортирующие сети. Принцип нулей-единиц. Битонная сортировка и сортировка чётно-нечётным слиянием.

2.1. Вычислительные схемы

Определение

Вычислительная схема — это ациклический ориентированный граф, состоящий из:

1. n входных вершин, для которых $\text{indeg}(v) = 0$;
2. m выходных вершин, для которых $\text{outdeg}(v) = 0$;
3. внутренних вершин, каждая из которых помечена элементарной операцией.

В качестве элементарных операций могут выступать арифметические, булевы, а также любые другие операции, считающиеся простыми в данной модели.

Размер схемы — число её внутренних вершин.

Глубина схемы — длина максимального ориентированного пути от входа к выходу.

Замечание

Глубина схемы интерпретируется как параллельное время вычисления: все вершины одного *слоя* (то есть вершины, у которых все входящие данные уже готовы) могут

вычисляться одновременно.

Алгоритм в модели вычислительных схем — это семейство схем $\{C_n\}_{n \geq 1}$, где схема C_n решает задачу на входах размера n .

Пример

Простейший пример: схема вычисления $(x_1 + x_2) \cdot (x_3 - x_4)$ имеет четыре входа, два слоя и размер 3.

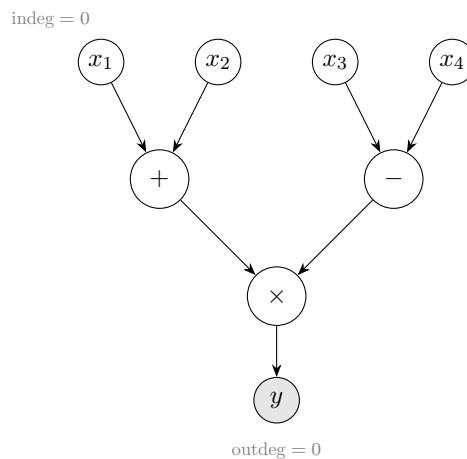


Рис. 5: Не заштрихованные вершины без входящих рёбер — *входы*; заштрихованная вершина без исходящих рёбер — *выход*. **Размер** схемы — 3. **Глубина** — 2.

2.2. Сортирующие сети

Определение

Сортирующая сеть — это схема, которая по входу из n чисел $x_1, \dots, x_n$ выдаёт числа $y_1, \dots, y_n$, упорядоченные по неубыванию, и использует в качестве внутренних элементов только *компараторы*.

Определение

Компаратор с входами u, v и выходами u', v' вычисляет

$$u' = \min\{u, v\}, \quad v' = \max\{u, v\}.$$

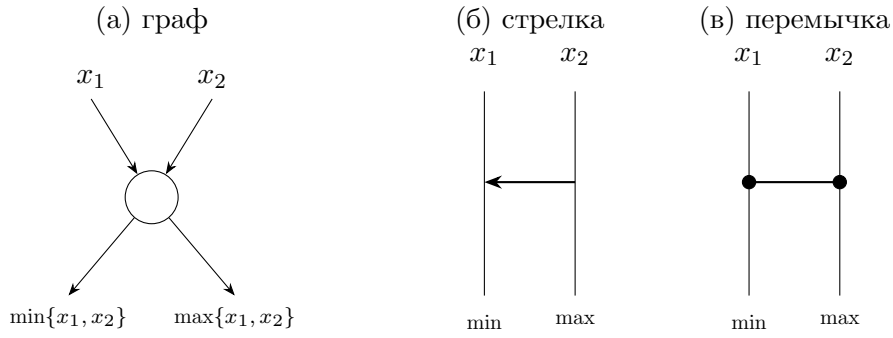


Рис. 6: Три способа изображения компаратора. В обозначениях (б) и (в) сигнал течёт сверху вниз по вертикальным проводам; горизонтальная перемычка сравнивает два соседних элемента и кладёт меньший на левый (верхний) провод.

Нижние оценки

- **Размер** любой сортирующей сети не менее $\Omega(n \log n)$. Поскольку каждый компаратор порождает не более 2 различных исходов, а число различных упорядочений n элементов равно $n!$, то для различения всех входов нужно не менее $\log_2(n!) = \Omega(n \log n)$ компараторов.
- **Глубина** любой сортирующей сети не менее $\Omega(\log n)$. В схеме глубины d значение одного выходного провода может зависеть не более чем от 2^d входных проводов. Так как максимальный элемент на выходе должен зависеть от всех n входов, получаем $2^d \geq n$, откуда $d \geq \log_2 n$.

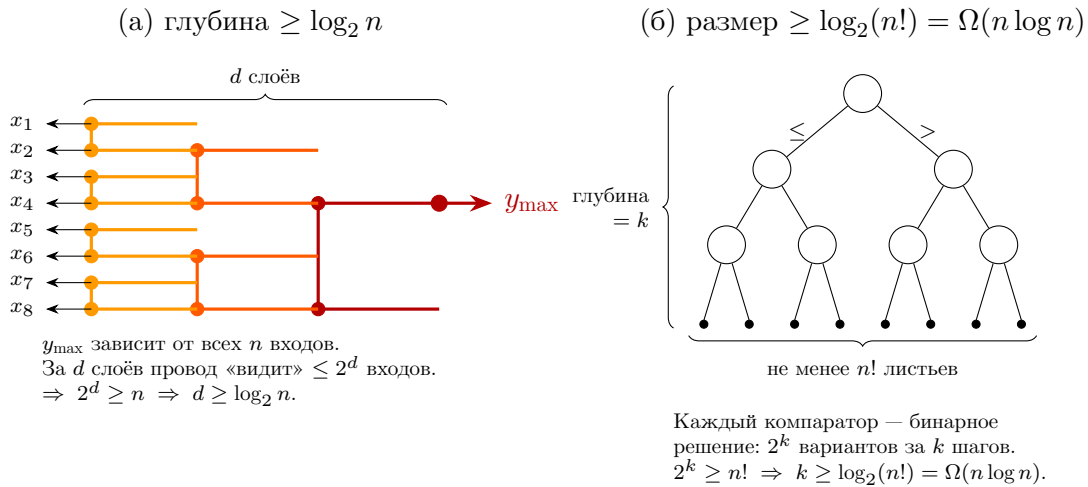


Рис. 7: (а) Нижняя оценка глубины $\Omega(\log n)$: за d слоёв провод максимума может «видеть» не более 2^d входов, а он должен видеть все n . (б) Нижняя оценка размера $\Omega(n \log n)$: нужно различить $n!$ порядков.

Замечание

Существуют сортирующие сети глубины $O(\log n)$ и размера $O(n \log n)$ (сеть AKS). Конструкция весьма сложна, поэтому далее рассматривается более простая *битонная сортировка*.

2.3. Принцип нулей и единиц

Теорема 2.1

Сортирующая сеть корректна на всех вещественных входах тогда и только тогда, когда она корректна на всех входах из нулей и единиц.

Доказательство. Необходимость очевидна: $\{0, 1\}$ -входы являются частным случаем вещественных.

Докажем достаточность. Предположим, что сеть корректно сортирует все $\{0, 1\}$ -входы, но не сортирует некоторый вещественный вход $x = (x_1, \dots, x_n)$. Тогда на выходе получается последовательность $y = (y_1, \dots, y_n)$, для которой существуют индексы $i < j$ такие, что $y_i > y_j$.

Выберем число t так, что $y_j < t \leq y_i$, и определим булевый вход

$$\tilde{x}_k = \begin{cases} 0, & x_k < t, \\ 1, & x_k \geq t. \end{cases}$$

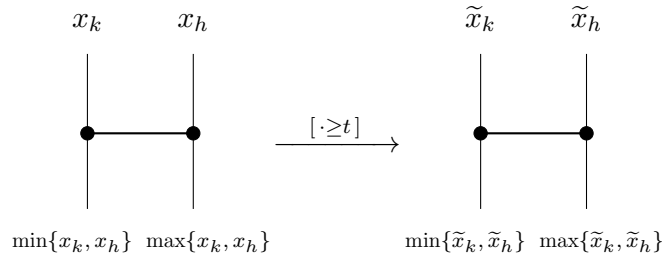


Рис. 8: Применение пороговой функции $[\cdot \geq t]$ до компаратора даёт тот же результат, что применение после: $[\min\{x_k, x_h\} \geq t] = \min\{[x_k \geq t], [x_h \geq t]\}$ и аналогично для $\max$.

Ключевое наблюдение. Пороговая функция $[\cdot \geq t]$ коммутирует с каждым компаратором. Действительно, рассмотрим компаратор на входах u, v , вычисляющий $(\min\{u, v\}, \max\{u, v\})$:

- Если $u, v < t$, то $\min$ и $\max$ тоже $< t$; после порогования: $(0, 0)$. Пороговые значения $(\tilde{u}, \tilde{v}) = (0, 0)$; компаратор на них даёт $(0, 0)$. Совпадает.
- Если $u, v \geq t$, то $\min$ и $\max$ тоже $\geq t$; после порогования: $(1, 1)$. $(\tilde{u}, \tilde{v}) = (1, 1)$; компаратор на них даёт $(1, 1)$. Совпадает.
- Если $u < t \leq v$ (или наоборот), то $\min\{u, v\} < t$ и $\max\{u, v\} \geq t$; после порогования: $(0, 1)$. $(\tilde{u}, \tilde{v}) = (0, 1)$ (или $(1, 0)$); компаратор на них даёт $(0, 1)$. Совпадает.

Таким образом, если сеть переводит x в y , то она переводит $\tilde{x}$ в $\tilde{y}$, где

$$\tilde{y}_k = \begin{cases} 0, & y_k < t, \\ 1, & y_k \geq t. \end{cases}$$

Из выбора t следует, что $\tilde{y}_i = 1$ и $\tilde{y}_j = 0$, то есть выход $\tilde{y}$ не отсортирован. Получено противоречие с корректностью сети на $\{0, 1\}$ -входах. ■

2.4. Битонные последовательности

Определение

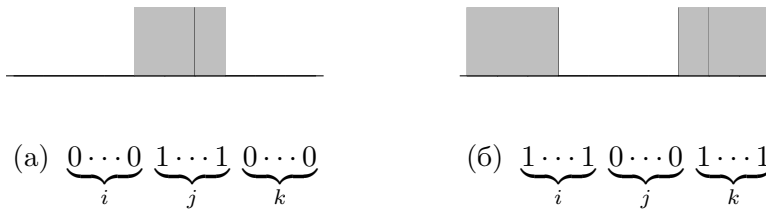
Последовательность $z_1, \dots, z_m$ называется *битонной*, если она является циклическим сдвигом последовательности, которая сначала неубывает, а затем невозрастает.

Пример

В случае $\{0, 1\}$ -последовательностей битонная последовательность имеет вид

$$\underbrace{0 \dots 0}_i \underbrace{1 \dots 1}_j \underbrace{0 \dots 0}_k \quad \text{или} \quad \underbrace{1 \dots 1}_i \underbrace{0 \dots 0}_j \underbrace{1 \dots 1}_k,$$

где $i, j, k \geq 0$, то есть состоит не более чем из трёх чередующихся блоков.



2.5. Битонное слияние

Пусть $m = 2^t$. Для битонной последовательности $z_1, \dots, z_m$ определим первый слой компараторов:

$$z_k^{\min} = \min\{z_k, z_{k+m/2}\}, \quad z_k^{\max} = \max\{z_k, z_{k+m/2}\}, \quad k = 1, \dots, \frac{m}{2}.$$

Лемма 2.1

Пусть $z_1, \dots, z_m$ — битонная последовательность. Тогда после одного слоя компараторов:

1. последовательность $(z_1^{\min}, \dots, z_{m/2}^{\min})$ битонна;
2. последовательность $(z_1^{\max}, \dots, z_{m/2}^{\max})$ битонна;
3. каждый элемент первой последовательности не превосходит каждого элемента второй.

Более точно, в случае $\{0, 1\}$ -последовательностей: либо $z^{\min} = (0, \dots, 0)$ и $z^{\max}$ битонна, либо $z^{\min}$ битонна и $z^{\max} = (1, \dots, 1)$.

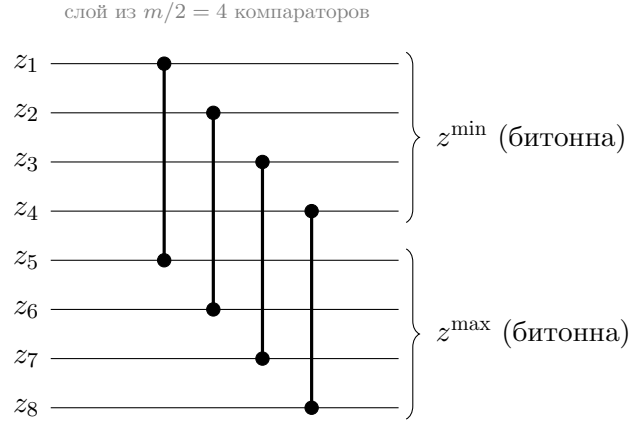


Рис. 9: Первый шаг `bi_merge` при $m = 8$: компаратор k сравнивает z_k и $z_{k+m/2}$. После слоя верхняя половина проводов несёт минимумы, нижняя — максимумы; обе половины являются битонными.

Это позволяет рекурсивно продолжить сортировку: отсортировать обе половины независимо и объединить результаты.

Algorithm 1 `BI_MERGE`($z_1, \dots, z_m$)

Require: Битонная последовательность $z_1, \dots, z_m$, где $m = 2^t$ для некоторого $t \geq 0$

Ensure: Последовательность $z_1, \dots, z_m$, отсортированная по неубыванию

- 1: **if** $m = 1$ **then**
 - 2: **return** (z_1)
 - 3: **for** $k = 1, \dots, \frac{m}{2}$ **do**
 - 4: $z_k^{\min} = \min(z_k, z_{k+m/2})$
 - 5: $z_k^{\max} = \max(z_k, z_{k+m/2})$
 - 6: $L = \text{BI_MERGE}(z_1^{\min}, \dots, z_{m/2}^{\min})$ ▷ один слой компараторов
 - 7: $R = \text{BI_MERGE}(z_1^{\max}, \dots, z_{m/2}^{\max})$ ▷ рекурсивно обрабатываем меньшие элементы
 - 8: **return** $L \parallel R$ ▷ рекурсивно обрабатываем большие элементы
- ▷ конкатенация двух половин
-

Лемма 2.2

Глубина сети `bi_merge` на входе длины m равна $O(\log m)$.

Доказательство. На каждом рекурсивном уровне добавляется один слой компараторов, после чего задача распадается на две независимые подзадачи длины $m/2$. Поэтому для глубины $T(m)$ выполнено $T(m) = T(\frac{m}{2}) + O(1)$, откуда $T(m) = O(\log m)$. ■

Лемма 2.3

Размер сети `bi_merge` на входе длины m равен $O(m \log m)$.

Доказательство. На верхнем уровне используется $m/2$ компараторов, после чего возникают две подзадачи длины $m/2$, поэтому $S(m) = 2S(\frac{m}{2}) + O(m)$, откуда $S(m) = O(m \log m)$. ■

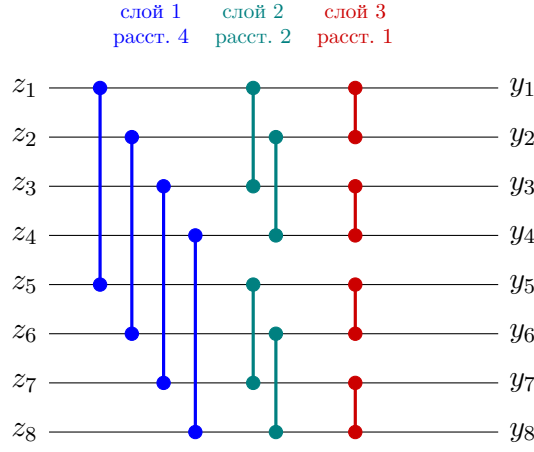


Рис. 10: Полная сеть `bi_merge` для $m = 8$: $\log_2 8 = 3$ слоя. Слой i сравнивает пары на расстоянии $m/2^i$. Глубина $O(\log m)$, размер $O(m \log m)$.

2.6. Битонная сортировка

Предполагается, что $n = 2^t$; если это не так, вход дополняется значением $+\infty$ до ближайшей степени двойки.

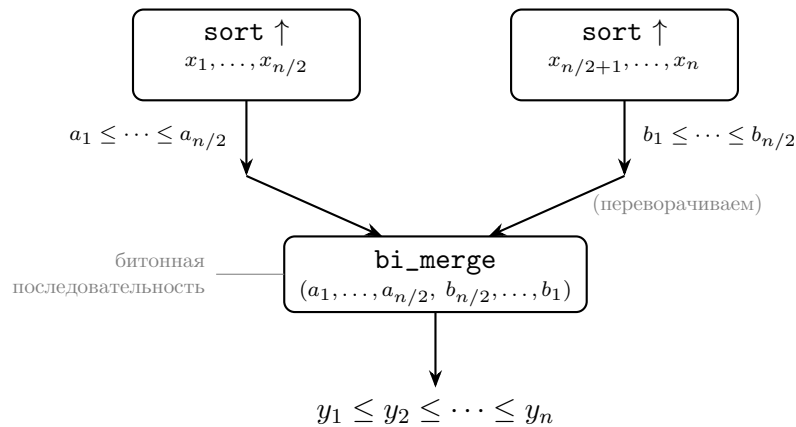
Algorithm 2 BITONIC_SORT($x_1, \dots, x_n$)

Require: Последовательность $x_1, \dots, x_n$, где $n = 2^t$ для некоторого $t \geq 0$

Ensure: Последовательность из тех же элементов, отсортированная по неубыванию

- 1: **if** $n = 1$ **then**
 - 2: **return** (x_1)
 - 3: $(a_1, \dots, a_{n/2}) = \text{BITONIC_SORT}(x_1, \dots, x_{n/2})$ ▷ левая половина сортируется по неубыванию
 - 4: $(b_1, \dots, b_{n/2}) = \text{BITONIC_SORT}(x_{n/2+1}, \dots, x_n)$ ▷ правая половина сортируется по неубыванию
 - 5: **return** $\text{BI_MERGE}(a_1, \dots, a_{n/2}, b_{n/2}, \dots, b_1)$ ▷ получаем битонную последовательность и сливаем её
-

Конкатенация $(a_1, \dots, a_{n/2}, b_{n/2}, \dots, b_1)$ является битонной: первая половина возрастает, вторая убывает.



Замечание

Корректность битонной сортировки на всех вещественных входах вытекает из теоремы о принципе нулей и единиц. Для $\{0, 1\}$ -входа достаточно убедиться, что все n одинаковых элементов и блоки $0 \dots 0 1 \dots 1$ обрабатываются правильно, что проверяется по лемме 2.1.

Теорема 2.2

Битонная сортировка имеет глубину $O(\log^2 n)$ и размер $O(n \log^2 n)$.

Доказательство. Обозначим через $D(n)$ глубину схемы, а через $S(n)$ — её размер.

Глубина. После рекурсивной сортировки половин (глубина $D(n/2)$) выполняется `bi_merge` глубины $O(\log n)$, поэтому $D(n) = D(n/2) + O(\log n)$. Раскрывая рекуррентность:

$$D(n) = \sum_{k=0}^{\log n - 1} O(\log \frac{n}{2^k}) = O\left(\sum_{k=0}^{\log n - 1} (\log n - k)\right) = O(\log^2 n).$$

Размер. Шаг `bi_merge` имеет размер $O(n \log n)$, поэтому $S(n) = 2S(n/2) + O(n \log n)$, откуда $S(n) = O(n \log^2 n)$. ■

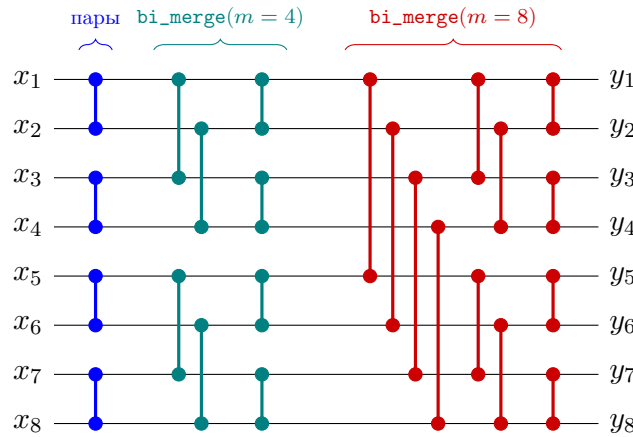


Рис. 11: Полная сортирующая сеть `bitonic_sort` для $n = 8$ ($6 = 1 + 2 + 3 = \frac{\log_2 n (\log_2 n + 1)}{2}$ слоёв). Синие компараторы сортируют пары; зелёные объединяют их в упорядоченные четвёрки; красные выполняют финальное битонное слияние.

Замечание

По сравнению с сетью AKS (глубина $O(\log n)$, размер $O(n \log n)$) битонная сортировка асимптотически хуже, но её конструкция существенно проще и потому она широко используется как базовый пример параллельной сортировки, в частности для реализации на FPGA и в GPU.

2.7. Нечётно-чётная сортировка (Batcher)

Альтернативный способ построения сортирующей сети с той же асимптотикой — нечётно-чётная сортировка Батчера. Ключевая операция — *нечётно-чётное слияние* двух уже отсортированных последовательностей.

Algorithm 3 ODD_EVEN_MERGE($a_1, \dots, a_m; b_1, \dots, b_m$)

Require: Две отсортированные по неубыванию последовательности длины m , где $m = 2^t$

Ensure: Последовательность длины $2m$, полученная слиянием входов и отсортированная по неубыванию

```
1: if  $m = 1$  then
2:    $u = \min(a_1, b_1)$ 
3:    $v = \max(a_1, b_1)$ 
4:   return  $(u, v)$ 
5:  $(c_1, \dots, c_m) = \text{ODD\_EVEN\_MERGE}(a_1, a_3, \dots, a_{m-1}; b_1, b_3, \dots, b_{m-1})$   $\triangleright$  нечётные позиции
6:  $(d_1, \dots, d_m) = \text{ODD\_EVEN\_MERGE}(a_2, a_4, \dots, a_m; b_2, b_4, \dots, b_m)$   $\triangleright$  чётные позиции
7:  $r_1 = c_1$ 
8: for  $i = 1, \dots, m - 1$  do
9:    $r_{2i} = \min(c_{i+1}, d_i)$ 
10:   $r_{2i+1} = \max(c_{i+1}, d_i)$ 
11:  $r_{2m} = d_m$   $\triangleright$  финальный слой из  $m - 1$  компараторов
12: return  $(r_1, \dots, r_{2m})$ 
```

Лемма 2.4

Процедура ODD_EVEN_MERGE корректно сливает две отсортированные последовательности длины m .

Доказательство. По принципу нулей и единиц рассмотрим вход из $\{0, 1\}$. Докажем корректность индукцией по m .

База: $m = 1$ очевидна.

Шаг: Пусть $m > 1$. По предположению индукции c и d отсортированы. Пусть z_a, z_b — количество нулей во входах. Тогда в c и d их количество равно:

$$z_c = \lceil z_a/2 \rceil + \lceil z_b/2 \rceil, \quad z_d = \lfloor z_a/2 \rfloor + \lfloor z_b/2 \rfloor$$

Так как $0 \leq \lceil x/2 \rceil - \lfloor x/2 \rfloor \leq 1$, разность $z_c - z_d$ может принимать значения $\{0, 1, 2\}$.

Рассмотрим последовательность $c_1, d_1, c_2, d_2, \dots, c_m, d_m$. Если $z_c - z_d \in \{0, 1\}$, она уже отсортирована. Если $z_c - z_d = 2$, то c содержит на два нуля больше, чем d . Это создает единственную инверсию ($d_i = 1, c_{i+1} = 0$) на границе раздела нулей и единиц.

Финальный слой компараторов (d_i, c_{i+1}) для всех i устраняет эту возможную инверсию, делая последовательность полностью отсортированной. ■

Algorithm 4 ODD_EVEN_SORT($x_1, \dots, x_n$)

Require: Последовательность $x_1, \dots, x_n$, где $n = 2^t$ для некоторого $t \geq 0$

Ensure: Последовательность из тех же элементов, отсортированная по неубыванию

```
1: if  $n = 1$  then
2:   return  $(x_1)$ 
3:  $(a_1, \dots, a_{n/2}) = \text{ODD\_EVEN\_SORT}(x_1, \dots, x_{n/2})$   $\triangleright$  сортировка левой половины
4:  $(b_1, \dots, b_{n/2}) = \text{ODD\_EVEN\_SORT}(x_{n/2+1}, \dots, x_n)$   $\triangleright$  сортировка правой половины
5: return ODD_EVEN_MERGE( $a_1, \dots, a_{n/2}; b_1, \dots, b_{n/2}$ )
```

Замечание

Корректность ODD_EVEN_SORT доказывается индукцией по n : сначала рекурсивно сортируются левая и правая половины, после чего они корректно сливаются процедурой ODD_EVEN_MERGE.

3. Префиксные суммы (scan), схема Ладнера–Фишера. Модель CRCW/EREW PRAM. Сортировка подсчё- том в модели PRAM.

3.1. Задача о префиксных суммах

Построить вычислительную схему, которая принимает на вход n чисел $x_0, \dots, x_{n-1}$ и (используя только бинарную операцию «+») выдаёт числа $y_0, \dots, y_{n-1}$, где

$$y_i = \sum_{j=0}^i x_j, \quad i = 0, \dots, n-1.$$

3.2. Схема Ладнера–Фишера

Пусть n чётно. Алгоритм состоит из трёх фаз.

1. **Свёртка (upsweep).** Попарно складываем соседние входы:

$$s_i = x_{2i} + x_{2i+1}, \quad i = 0, \dots, \frac{n}{2} - 1.$$

2. **Рекурсия.** Считаем префиксные суммы для массива s :

$$p_i = \sum_{j=0}^i s_j.$$

3. **Развёртка (downsweep).** Восстанавливаем выходы:

$$y_{2i+1} = p_i, \quad y_{2i} = \begin{cases} x_0, & i = 0, \\ p_{i-1} + x_{2i}, & i \geq 1. \end{cases}$$

Утверждение 3.1: Оценки глубины и размера схемы Ладнера–Фишера

Обозначим через $T(n)$ глубину схемы (длину критического пути), а через $S(n)$ — её размер (число операций). Тогда при $n = 2^m$:

$$\begin{aligned} T(n) &= 2 + T\left(\frac{n}{2}\right), \quad T(1) = 0 \implies T(n) = 2 \log n, \\ S(n) &= (n-1) + S\left(\frac{n}{2}\right) \implies S(n) = 2n - \log n - 2. \end{aligned}$$

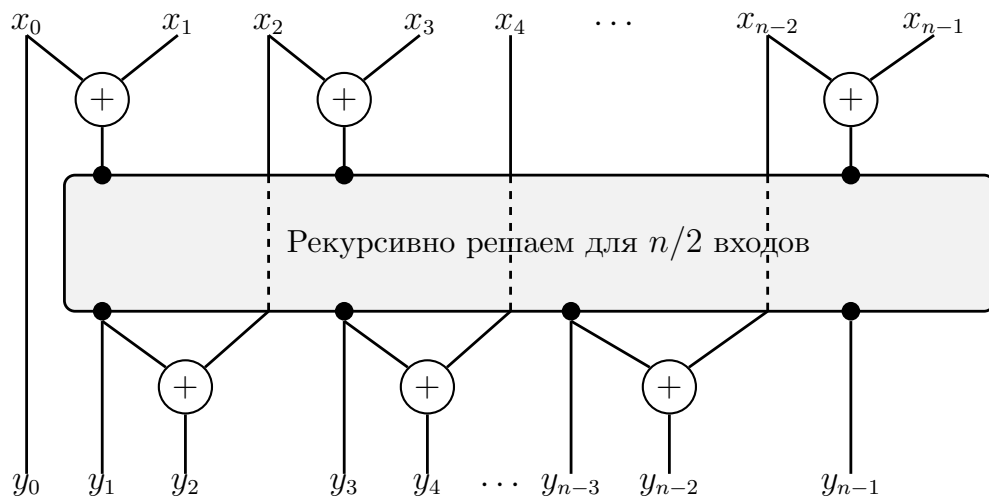


Рис. 14: Схема Ладнера–Фишера для префиксных сумм

Замечание

Схема Ладнера–Фишера хорошо ложится на GPU: на каждом уровне рекурсии выполняется большое количество независимых операций, которые можно запускать параллельно.

3.3. Обобщение: операция scan

Определение

Операция **scan** (префиксное накопление) для ассоциативной бинарной операции $\odot$ на множестве S (то есть для **полугруппы** $(S, \odot)$) определяется как вычисление массива

$$y_i = x_0 \odot x_1 \odot \cdots \odot x_i, \quad i = 0, \dots, n-1.$$

Схема Ладнера–Фишера работает для любой ассоциативной операции. Примеры допустимых операций $\odot$:

$$(\mathbb{R}, +), \quad (\mathbb{R}, \min), \quad (\mathbb{R}, \max), \quad (\mathbb{R}^{k \times k}, \cdot) \text{ (умножение матриц)}.$$

3.4. Псевдокод scan (in-place, inclusive)

Ниже представлен алгоритм для $n = 2^m$. Он работает на исходном массиве *на месте* (без дополнительной памяти) и состоит из двух проходов по уровням $d = 0, \dots, \log_2 n - 1$. На уровне d массив мысленно разбит на блоки длины 2^{d+1} ; «правым концом» блока, начинающегося в позиции k , считается элемент $x[k + 2^{d+1} - 1]$. Для фиксированного d все операции во внутреннем цикле по k независимы и выполняются *параллельно*.

Таким образом, задача сводится к вычислению **префиксных произведений** матриц $X_0, X_1, \dots, X_{n-1}$. Поскольку умножение матриц ассоциативно, это делается через **scan** за $O(\log n)$ времени в параллельной модели.

Замечание

В примере с матрицами при перемножении используются *две* операции над элементами:

$$(A \cdot B)_{ij} = \bigoplus_k (A_{ik} \odot B_{kj}).$$

Поэтому обобщение работает не для одной операции, а для **полукольца** $(S, \oplus, \odot)$, в котором:

- $\oplus$ ассоциативна (обычно коммутативна) и имеет нейтральный элемент 0;
- $\odot$ ассоциативна и имеет нейтральный элемент 1;
- $\odot$ дистрибутивна относительно $\oplus$:

$$a \odot (b \oplus c) = (a \odot b) \oplus (a \odot c), \quad (a \oplus b) \odot c = (a \odot c) \oplus (b \odot c);$$

- 0 поглощает умножение: $0 \odot a = a \odot 0 = 0$.

Примеры полуколец $(S, \oplus, \odot)$:

$$(\{0, 1\}, \vee, \wedge), \quad (\{0, 1\}, \oplus, \wedge), \quad (\mathbb{R} \cup \{+\infty\}, \min, +).$$

3.6. Сумматор Ладнера–Фишера

Определение

Сумматор — это вычислительная схема над булевыми операциями $(\vee, \wedge, \oplus, \neg, \dots)$, которая принимает на вход биты $x_0, \dots, x_{n-1}$ и $y_0, \dots, y_{n-1}$ и выдаёт биты $z_0, \dots, z_n$ такие, что $z = x + y$, где

$$x = \sum_{k=0}^{n-1} x_k 2^k, \quad y = \sum_{k=0}^{n-1} y_k 2^k, \quad z = \sum_{k=0}^n z_k 2^k.$$

Биты переноса и сведение к scan

Обозначим через c_k перенос в разряд k (по определению $c_0 = 0$). Тогда для $k = 0, \dots, n-1$ выполняется **формула переноса**:

$$c_{k+1} = (x_k \wedge y_k) \vee (x_k \wedge c_k) \vee (y_k \wedge c_k).$$

Введём обозначения $a_k := x_k \wedge y_k$ и $b_k := x_k \vee y_k$. Тогда формула переноса принимает компактный вид $c_{k+1} = a_k \vee (b_k \wedge c_k)$. Если все переносы $c_0, \dots, c_n$ известны, ответ восстанавливается по формулам

$$z_k = x_k \oplus y_k \oplus c_k \quad (k = 0, \dots, n-1), \quad z_n = c_n.$$

Ключевая идея: сведение к scan. Рекуррента $c_{k+1} = a_k \vee (b_k \wedge c_k)$ является аффинным преобразованием. Положим

$$C_k = (1, c_k), \quad X_k = \begin{pmatrix} 1 & a_k \\ 0 & b_k \end{pmatrix},$$

где операции понимаются над булевым полукольцом $(\{0, 1\}, \vee, \wedge)$. Тогда $C_{k+1} = C_k X_k$, и задача нахождения всех переносов сводится к вычислению префиксных произведений

$$C_k = C_0 X_0 X_1 \cdots X_{k-1},$$

что выполняется через **scan** за $O(\log n)$ времени в параллельной модели.

3.7. Модель PRAM

Определение

PRAM (Parallel Random Access Machine) — идеализированная модель параллельных вычислений:

- имеется общая глобальная память;
- число процессоров теоретически не ограничено; каждый процессор является RAM-машиной с собственным номером $p = 1, 2, \dots$;
- процессоры работают **синхронно**: за единицу времени каждый выполняет одну инструкцию.

Разрешение конфликтов доступа к памяти

Различают два режима для чтения и записи: *concurrent* (одновременный доступ нескольких процессоров к одной ячейке разрешён) и *exclusive* (одновременный доступ запрещён). Стандартные варианты модели:

CRCW PRAM, CREW PRAM, ERCW PRAM, EREW PRAM.

В модели **CRCW** при одновременной записи нескольких процессоров в одну ячейку необходимо задать правило разрешения конфликта (например, *arbitrary*: в ячейке остаётся одно из записанных значений).

Определение

NC — класс задач, которые решаются на PRAM за время $(\log n)^{O(1)}$ при использовании полиномиального числа процессоров $n^{O(1)}$, где n — размер входа.

3.8. Scan на p процессорах в модели PRAM

Пусть дан массив $A[0..n-1]$ и p процессоров. Разобьём массив на блоки длины $m = n/p$ (считаем $p \mid n$).

1. Каждый процессор i вычисляет **локальный scan** на своём блоке $A[im..(i+1)m-1]$ последовательно за время $O(m) = O(n/p)$. Обозначим сумму i -го блока через T_i .
2. Вычисляем **scan по массиву блок-сумм** $T_0, T_1, \dots, T_{p-1}$ за время $O(\log p)$. Пусть

$$P_i = \sum_{j=0}^{i-1} T_j$$

— сумма всех предшествующих блоков (смещение для блока i).

3. Каждый процессор прибавляет смещение P_i ко всем элементам своего блока за время $O(n/p)$.

Утверждение 3.2: Время работы scan на p процессорах

Суммарное время составляет

$$O\left(\frac{n}{p} + \log p\right).$$

В частности, при $p = \Theta\left(\frac{n}{\log n}\right)$ получаем время $O(\log n)$.

Пример : Сортировка подсчётом (counting sort) в PRAM

Дан массив $a[0..n-1]$, где $a[k] \in \{0, \dots, U\}$ при $U = O(1)$. Используем p процессоров с блоками длины $m = n/p$.

1. **Локальный подсчёт частот.** Процессор i строит таблицу частот $c_i[0..U]$:

$$c_i(x) = \#\{k \in [im, (i+1)m-1] \mid a[k] = x\}.$$

Время: $O(n/p)$.

2. **Сборка и scan.** Формируем массив длины $(U+1)p$:

$$C[xp+i] = c_i(x), \quad x = 0, \dots, U, \quad i = 0, \dots, p-1.$$

Вычисляем scan по C и получаем массив P .

3. **Расстановка элементов.** Процессор i знает, что элементы значения x из его блока нужно поместить в позиции

$$[P[xp+i] - c_i(x), P[xp+i]).$$

Время: $O(n/p)$.

Итоговое время. Локальный подсчёт — $O(n/p)$; scan по массиву длины $(U+1)p$ — $O(\log p)$, так как $U = O(1)$. Итого:

$$O\left(\frac{n}{p} + \log p\right).$$

4. Трудоёмкость локальных вычислений, коммуникации и синхронизации при вычислениях в модели барьерно-синхронного параллелизма (BSP). Префиксные суммы в BSP, оптимальность.

Определение : Машинное слово

Машинное слово (в моделях, подобных RAM) — это целочисленный тип данных, способный принимать все значения, необходимые для работы алгоритма. Машинное слово состоит из некоторого числа бит.

4.1. Модель блочно-синхронного параллелизма (*Bulk Synchronous Parallel, BSP*)

Модель **BSP** задаётся параметрами p , g и L , где:

- p — число компьютеров (процессоров);
- g — время передачи или получения одного машинного слова;
- L — время синхронизации.

Обычно предполагается, что $g \gg 1$, $L \gg 1$.

1. Модель состоит из p компьютеров (или процессоров), пронумерованных числами от 1 до p . Каждый из них представляет собой полноценную RAM-модель со своей локальной памятью.
2. Компьютеры объединены сетью, и каждый из них может посылать сообщения размера m машинных слов другому компьютеру за gm единиц времени.
3. После этапа локальных вычислений все компьютеры “синхронизируются”, ожидая завершения работы всех остальных компьютеров. На этом этапе доставляются все отправленные сообщения. Каждый компьютер, получающий m машинных слов, тратит $gm + L$ единиц времени: gm на получение сообщения и L на синхронизацию.

4.2. Супершаги

Алгоритм в модели BSP выполняется по *супершагам*. Каждый супершаг состоит из трёх последовательных этапов:

1. локальные вычисления на каждом компьютере;
2. обмен сообщениями между компьютерами;
3. синхронизация, на которой все компьютеры ждут друг друга.

Обозначим через $t_k^{(s)}$ время локальных вычислений, выполненных k -м компьютером на s -м супершаге. Через $c_k^{(s)}$ обозначим количество машинных слов, которые k -й компьютер отправил или получил на этом супершаге.

Тогда время выполнения s -го супершага определяется самым медленным вычислением, самой большой коммуникационной нагрузкой и временем синхронизации. Поэтому оно равно

$$\max_{k=1,\dots,p} t_k^{(s)} + \max_{k=1,\dots,p} (g c_k^{(s)}) + L.$$

4.3. Стоимость BSP-алгоритма

Пусть алгоритм работает $\hat{S}$ супершагов. Тогда его время работы в модели BSP удобно измерять тремя компонентами:

1. **вычисления**

$$\sum_{s=1}^{\hat{S}} \max_{k=1,\dots,p} t_k^{(s)};$$

2. коммуникация

$$\sum_{s=1}^{\hat{S}} \max_{k=1,\dots,p} c_k^{(s)};$$

3. синхронизация

$$\hat{S} L.$$

Во второй компоненте формально должен стоять множитель g , то есть $\sum_{s=1}^{\hat{S}} \max_k (g c_k^{(s)})$, однако в асимптотических оценках его часто опускают, поскольку g считается константой и поглощается O -большим.

Соглашение о вводе и выводе. Обычно предполагается, что входные данные не лежат целиком в памяти одного компьютера. Они считаются доступными через коммуникацию с некоторым внешним хранилищем: чтобы прочитать ввод, компьютеры получают сообщения от внешнего хранилища, а чтобы выдать результат, передают его сообщениями обратно во внешнее хранилище.

Пример : Вычисление префиксных сумм в модели BSP

Рассмотрим задачу вычисления префиксных сумм в модели BSP. Дана последовательность из n чисел $x_1, x_2, \dots, x_n$, где $n \gg p$. Требуется вычислить

$$y_1, y_2, \dots, y_n, \quad y_k = \sum_{i=1}^k x_i.$$

Для простоты будем считать, что p делит n . Тогда k -й компьютер обрабатывает блок $x_{(k-1)n/p+1}, \dots, x_{kn/p}$.

Первый супершаг. k -й компьютер считывает из внешнего хранилища свой блок длины n/p . Вычисления не производятся, коммуникации: $O\left(\frac{n}{p}\right)$.

Второй супершаг. k -й компьютер вычисляет сумму элементов своего блока и отправляет эту сумму компьютеру 1. Вычисления: $O\left(\frac{n}{p}\right)$, коммуникации: $O(p)$.

Третий супершаг. Компьютер 1 вычисляет префиксные суммы сумм блоков, после чего отправляет k -му компьютеру сумму всех предыдущих блоков, которую нужно прибавить к локальным префиксным суммам внутри k -го блока. Вычисления: $O(p)$, коммуникации: $O(p)$.

Четвёртый супершаг. Каждый компьютер вычисляет префиксные суммы внутри своего блока, прибавляет к ним полученную добавку и записывает результат в выходной массив. Вычисления: $O\left(\frac{n}{p}\right)$, коммуникации: $O\left(\frac{n}{p}\right)$.

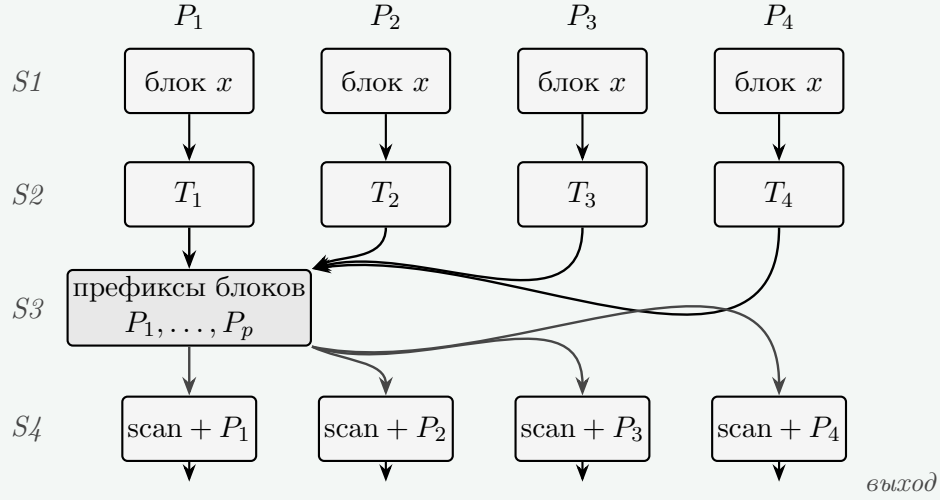


Рис. Поток данных в BSP-алгоритме префиксных сумм ($p = 4$). **S1**: каждый P_k читает свой блок длины n/p . **S2**: P_k считает сумму блока T_k и пересылает её на P_1 (сбор, *gather*). **S3**: P_1 последовательно считает префиксы блок-сумм P_k и рассылает P_k обратно (рассылка, *scatter*); именно здесь возникает узкое место $O(p)$. **S4**: каждый P_k делает локальный scan и прибавляет смещение P_k . Число супершагов константно ($\hat{S} = 4$).

Общее время работы. Суммируя по четырём супершкагам:

- вычисления: $O\left(p + \frac{n}{p}\right)$;
- коммуникации: $O\left(p + \frac{n}{p}\right)$;
- синхронизация: $O(1)$, так как число супершагов константно.

Если $p^2 \ll n$, то $p + \frac{n}{p} = O\left(\frac{n}{p}\right)$. Следовательно, и вычисления, и коммуникации имеют порядок $O\left(\frac{n}{p}\right)$.

Оптимальность. Алгоритм асимптотически оптимален: каждое входное машинное слово нужно хотя бы один раз прочитать, поэтому требуется не менее $\Omega\left(\frac{n}{p}\right)$ коммуникаций и не менее $\Omega\left(\frac{n}{p}\right)$ локальных вычислений.

5. Параллельное вычисление обобщённых линейных рекуррентных соотношений и параллельное сложение двоичных чисел при помощи задачи префиксного накопления.

5.1. Обобщение: операция scan

Определение

Операция **scan** (префиксное накопление) для ассоциативной бинарной операции $\odot$ на множестве S (то есть для **полугруппы** $(S, \odot)$) определяется как вычисление массива

$$y_i = x_0 \odot x_1 \odot \cdots \odot x_i, \quad i = 0, \dots, n-1.$$

Схема Ладнера–Фишера работает для любой ассоциативной операции. Примеры допустимых операций $\odot$:

$$(\mathbb{R}, +), \quad (\mathbb{R}, \min), \quad (\mathbb{R}, \max), \quad (\mathbb{R}^{k \times k}, \cdot) \text{ (умножение матриц).}$$

Утверждение 5.1: Оценки глубины и размера схемы Ладнера–Фишера

Обозначим через $T(n)$ глубину схемы (длину критического пути), а через $S(n)$ — её размер (число операций). Тогда при $n = 2^m$:

$$\begin{aligned} T(n) &= 2 + T\left(\frac{n}{2}\right), \quad T(1) = 0 \implies T(n) = 2 \log n, \\ S(n) &= (n-1) + S\left(\frac{n}{2}\right) \implies S(n) = 2n - \log n - 2. \end{aligned}$$

5.2. Сведение к scan: линейная рекуррента

Пример : Рекуррента вида $c_{k+1} = a_k + b_k c_k$

Даны числа $a_0, \dots, a_{n-1}$ и $b_0, \dots, b_{n-1}$. Требуется найти $c_0, \dots, c_n$ такие, что

$$c_0 = 0, \quad c_{k+1} = a_k + b_k c_k.$$

Определим строчный вектор $C_k = (1, c_k)$. Тогда

$$C_{k+1} = C_k X_k, \quad X_k = \begin{pmatrix} 1 & a_k \\ 0 & b_k \end{pmatrix}.$$

Отсюда

$$C_1 = C_0 X_0, \quad C_2 = C_0 X_0 X_1, \quad C_3 = C_0 X_0 X_1 X_2, \quad \dots$$

Таким образом, задача сводится к вычислению **префиксных произведений** матриц $X_0, X_1, \dots, X_{n-1}$. Поскольку умножение матриц ассоциативно, это делается через **scan** за $O(\log n)$ времени в параллельной модели.

Замечание

В примере с матрицами при перемножении используются две операции над элементами:

$$(A \cdot B)_{ij} = \bigoplus_k (A_{ik} \odot B_{kj}).$$

Поэтому обобщение работает не для одной операции, а для **полукольца** $(S, \oplus, \odot)$, в котором:

- $\oplus$ ассоциативна (обычно коммутативна) и имеет нейтральный элемент 0;
- $\odot$ ассоциативна и имеет нейтральный элемент 1;
- $\odot$ дистрибутивна относительно $\oplus$:

$$a \odot (b \oplus c) = (a \odot b) \oplus (a \odot c), \quad (a \oplus b) \odot c = (a \odot c) \oplus (b \odot c);$$

- 0 поглощает умножение: $0 \odot a = a \odot 0 = 0$.

Примеры полуколец $(S, \oplus, \odot)$:

$$(\{0, 1\}, \vee, \wedge), \quad (\{0, 1\}, \oplus, \wedge), \quad (\mathbb{R} \cup \{+\infty\}, \min, +).$$

Утверждение 5.2: Сложность вычисления

Задача вычисления $c_0, c_1, \dots, c_n$ из рекуррентности $c_{k+1} = a_k + b_k c_k$ сводится к **scan** над ассоциативным умножением 2×2 -матриц. Схема Ладнера–Фишера решает эту задачу с глубиной $O(\log n)$ и размером $O(n)$. То же верно для любой ассоциативной рекуррентности $C_{k+1} = C_k \cdot X_k$ над произвольным ассоциативным полукольцом $(S, \oplus, \odot)$.

5.3. Сумматор Ладнера–Фишера

Определение

Сумматор — это вычислительная схема над булевыми операциями $(\vee, \wedge, \oplus, \neg, \dots)$, которая принимает на вход биты $x_0, \dots, x_{n-1}$ и $y_0, \dots, y_{n-1}$ и выдаёт биты $z_0, \dots, z_n$ такие, что $z = x + y$, где

$$x = \sum_{k=0}^{n-1} x_k 2^k, \quad y = \sum_{k=0}^{n-1} y_k 2^k, \quad z = \sum_{k=0}^n z_k 2^k.$$

Биты переноса и сведение к scan

Обозначим через c_k перенос в разряд k (по определению $c_0 = 0$). Тогда для $k = 0, \dots, n-1$ выполняется **формула переноса**:

$$c_{k+1} = (x_k \wedge y_k) \vee (x_k \wedge c_k) \vee (y_k \wedge c_k).$$

Введём обозначения $a_k := x_k \wedge y_k$ и $b_k := x_k \vee y_k$. Тогда формула переноса принимает компактный вид $c_{k+1} = a_k \vee (b_k \wedge c_k)$. Если все переносы $c_0, \dots, c_n$ известны, ответ восстанавливается по формулам $z_k = x_k \oplus y_k \oplus c_k$ (для $k = 0, \dots, n-1$) и $z_n = c_n$.

Ключевая идея: сведение к scan. Рекуррента $c_{k+1} = a_k \vee (b_k \wedge c_k)$ является аффинным

преобразованием. Положим

$$C_k = (1, c_k), \quad X_k = \begin{pmatrix} 1 & a_k \\ 0 & b_k \end{pmatrix},$$

где операции понимаются над булевым полукольцом $(\{0, 1\}, \vee, \wedge)$. Тогда $C_{k+1} = C_k X_k$, и задача нахождения всех переносов сводится к вычислению префиксных произведений

$$C_k = C_0 X_0 X_1 \cdots X_{k-1},$$

что выполняется через **scan** за $O(\log n)$ времени в параллельной модели.

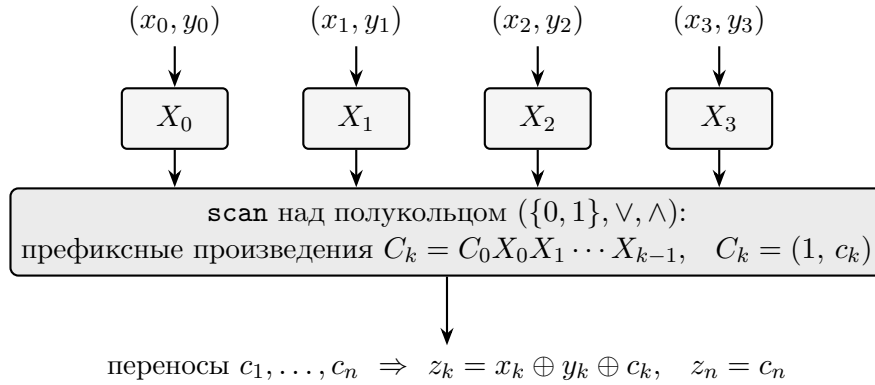


Рис. 13: Сведение сложения двоичных чисел к **scan**. Из пары битов (x_k, y_k) строится образующая $X_k = \begin{pmatrix} 1 & a_k \\ 0 & b_k \end{pmatrix}$, где $a_k = x_k \wedge y_k$ и $b_k = x_k \vee y_k$. Префиксные произведения этих матриц над булевым полукольцом дают все переносы c_k , после чего разряды суммы получаются как $z_k = x_k \oplus y_k \oplus c_k$. Префиксные произведения вычисляет схема Ладнера–Фишера за глубину $O(\log n)$.

Утверждение 5.3: Сложность сумматора Ладнера–Фишера

Задача нахождения всех битов переноса $c_1, \dots, c_n$ сводится к **scan** над булевым полукольцом $(\{0, 1\}, \vee, \wedge)$. Применяя схему Ладнера–Фишера, получаем сумматор с глубиной $O(\log n)$ и размером $O(n)$.

6. Алгоритмы во внешней памяти. Сортировка слиянием. Сортировка связного списка

6.1. Модель внешней памяти

Модель внешней памяти задаётся двумя параметрами: B и M . Обычно предполагается, что

$$B \ll M,$$

а часто дополнительно $B^2 \leq M$.

- Алгоритм располагает неограниченной внешней памятью. За единицу времени он может прочитать или записать один блок, состоящий из B подряд идущих машинных слов внешней памяти.
- Алгоритм обладает внутренней памятью объёмом M машинных слов, то есть M/B блоков.

- Все вычисления над данными, уже находящимися во внутренней памяти, считаются бесплатными. Входные данные изначально расположены во внешней памяти.

Временем работы алгоритма считается число операций ввода-вывода, то есть число прочитанных и записанных блоков.

Основные примеры.

- Сумма N чисел вычисляется за $O(N/B)$ операций ввода-вывода (обозначается $\text{scan}(N)$).
- Поиск и вставка в B -дереве выполняются за $O(\log_B N)$ операций.

6.2. Сортировка слиянием во внешней памяти

Алгоритм является (M/B) -путевой сортировкой слиянием.

1. Разбить массив длины N на $\Theta(M/B)$ подмассивов и рекурсивно отсортировать каждый из них.
2. Из каждого подмассива читать по одному блоку. Так как число подмассивов равно $\Theta(M/B)$, все блоки помещаются во внутреннюю память.
3. Выполнять слияние блоков во внутренней памяти. Когда блок какого-либо подмассива заканчивается, дочитывать следующий блок этого подмассива из внешней памяти за $O(1)$.
4. Результат слияния записывать во внешнюю память блоками.

Стоимость одного уровня слияния (шаги 2–4): $O(N/B)$ операций ввода-вывода.

Рекуррентная оценка:

$$T(N) = \Theta\left(\frac{M}{B}\right) T\left(\frac{N}{M/B}\right) + O\left(\frac{N}{B}\right).$$

Решение:

$$T(N) = O\left(\frac{N}{B} \log_{M/B} \frac{N}{B}\right) = O\left(\frac{N}{B} \log_{M/B} \frac{N}{M}\right).$$

Эту величину обозначают $\text{sort}(N)$.

6.3. Задача ранжирования связного списка (List Ranking)

Постановка. Дан массив $A[0..N-1]$, содержащий N узлов кругового однонаправленного связного списка в произвольном порядке. У каждого узла x есть поля $x.\text{next}$, $x.w$, $x.\text{id}$; изначально $A[i].\text{id} = i$.

Ниже $\text{pred}(x)$ обозначает предшественника узла x в текущем списке (это не поле структуры данных, а математическое обозначение для анализа).

Требуется вычислить для каждой вершины x префиксную сумму весов:

$$\text{pref}(x) = \sum_{y \text{ от головы до } x} y.w.$$

После работы алгоритма в поле $x.w$ записывается значение $\text{pref}(x)$.

Если изначально $w \equiv 1$, то $\text{pref}(x)$ есть номер x в списке, считая от головы. Отсортировав узлы по полученным значениям, получаем порядок обхода списка.

6.4. Идея рекурсивного сжатия

Пусть $S \subseteq A$ — множество удаляемых узлов такое, что:

$$\forall x \in S: \text{pred}(x) \notin S, \quad |S| = \Theta(N).$$

Тогда все узлы из S можно удалить независимо: для каждого $x \in S$ выполнить

$$\text{pred}(x).w += x.w, \quad \text{pred}(x).\text{next} = x.\text{next}.$$

После этого рекурсивно вычислить префиксные суммы на укороченном списке, затем восстановить удалённые узлы: для каждого $x \in S$

$$\text{pred}(x).\text{next} = x, \quad \text{pred}(x).w -= x.w, \quad x.w += \text{pred}(x).w.$$

6.5. Выбор множества S

Каждой вершине независимо присваивается случайный бит $x.r \in \{0, 1\}$ с вероятностью $1/2$ каждый; голове списка принудительно назначается $\text{head}.r = 0$.

Затем полагаем:

$$x \in S \iff x.r = 1 \text{ и } \text{pred}(x).r = 0.$$

Корректность. Если $x \in S$, то $\text{pred}(x).r = 0$, откуда $\text{pred}(x) \notin S$. Значит, никакие два соседних по списку узла не удаляются одновременно.

Ожидаемый размер S . Для каждой внутренней вершины x :

$$\Pr[x \in S] = \Pr[\text{pred}(x).r = 0 \wedge x.r = 1] = \frac{1}{4}.$$

Поэтому $\mathbb{E}[|S|] \approx N/4$, а после одного шага остаётся в среднем $\approx 3N/4$ вершин.

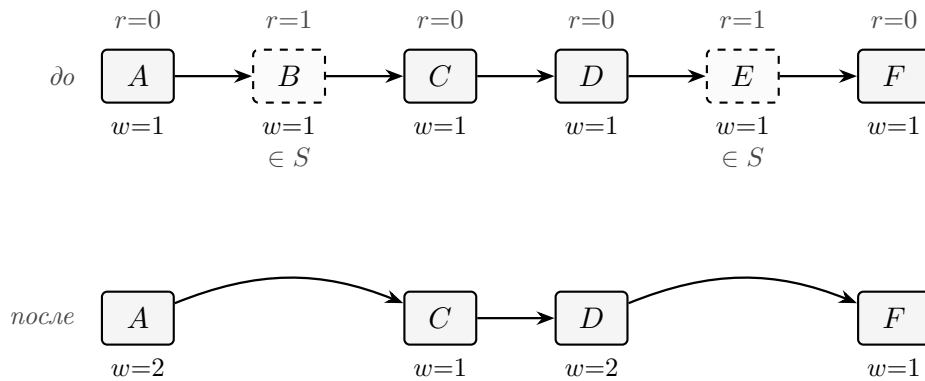


Рис. 14: Один шаг сжатия списка. Каждой вершине брошен случайный бит r ; в S попадают вершины шаблона 01 — узел с $r=1$, у которого предшественник имеет $r=0$ (здесь B и E). Никакие два таких узла не соседствуют, поэтому их удаляют независимо: вес удаляемого узла прибавляется к предшественнику ($A.w$ и $D.w$ становятся равными 2), а указатель предшественника перенаправляется на преемника. На укороченном списке ($\approx 3/4$ вершин в среднем) задача решается рекурсивно.

6.6. Концептуальный рекурсивный алгоритм

Algorithm 6 PREFIXSUMS-CONCEPTUAL(A)

Require: Текущий список A

Ensure: Для каждой вершины x значение $x.w$ равно $\text{pref}(x)$

```

1: if  $|A| \leq 1$  then
2:   return  $A$  ▷ список из одного узла уже отранжирован
3: Для каждой вершины  $x \in A$  независимо выбрать  $x.r \in \{0, 1\}$  равновероятно
4:  $S = \{x \in A \mid \text{pred}(x) \text{ существует, } \text{pred}(x).r = 0, x.r = 1\}$  ▷  $S$  — «правые концы»
   шаблонa 01; это независимое множество
5: for all  $x \in S$  do ▷ сжатие: удаляем узлы из  $S$  независимо друг от друга
6:    $\text{pred}(x).w += x.w$  ▷ вес  $x$  переходит к предшественнику
7:    $\text{pred}(x).\text{next} = x.\text{next}$  ▷ предшественник перепрыгивает  $x$ 
8: PREFIXSUMS-CONCEPTUAL( $A \setminus S$ ) ▷ та же задача на укороченном (в среднем  $\approx 3N/4$ )
   списке
9: for all  $x \in S$  do ▷ восстановление: возвращаем удалённые узлы и дописываем ответы
10:   $\text{pred}(x).\text{next} = x$  ▷ возвращаем  $x$  в список
11:   $\text{pred}(x).w -= x.w$  ▷ откат агрегации: теперь  $\text{pred}(x).w = \text{pref}(\text{pred}(x))$ 
12:   $x.w += \text{pred}(x).w$  ▷  $\text{pref}(x) = \text{pref}(\text{pred}(x)) + x.w$ 
13: return  $A$ 

```

6.7. Реализация во внешней памяти через сортировки

Главная цель — заменить случайные обращения по указателям на несколько сортировок и линейных проходов.

Ключевое наблюдение. Пусть $\tilde{A} = \text{sort}(A \text{ по } x.\text{next})$. Тогда при одновременном просмотре A (отсортированного по $x.\text{id}$) и $\tilde{A}$ можно сопоставить каждый узел $A[i]$ с его предшественником $\tilde{A}[i]$ (поскольку $\tilde{A}[i].\text{next} = A[i].\text{id}$). Шаблон 01 — это условие $\tilde{A}[i].r = 0$ и $A[i].r = 1$.

После удаления узла $A[i]$ его предшественник $\tilde{A}[i]$ начинает указывать туда же, куда раньше указывал $A[i]$. При повторной сортировке по **next** появляются пары с одинаковым **next**; живая вершина в паре определяется по биту r .

Algorithm 7 COMPRESS(A)

Require: Массив $A[0..N-1]$, отсортированный по id

Ensure: Сжатый массив $\hat{A}$ и массив удалённых вершин S

```
1: Для каждой  $x \in A$  независимо выбрать  $x.r \in \{0, 1\}$  равновероятно
2:  $\tilde{A} = \text{sort}(A \text{ по } x.\text{next})$   $\triangleright$  после сортировки  $\tilde{A}[i]$  — предшественник  $A[i]$ , т.к.
    $\tilde{A}[i].\text{next} = A[i].\text{id}$ 
3:  $S = []$ 
    $\triangleright$  ищем шаблон 01 и сразу переносим вес удаляемого узла в предшественника
4: for  $i = 0$  to  $N - 1$  do
5:   if  $\tilde{A}[i].r = 0$  and  $A[i].r = 1$  then  $\triangleright$  предшественник  $r=0$ , сам узел  $r=1 \Rightarrow$  удаляем
      $A[i]$ 
6:      $\tilde{A}[i].w += A[i].w$   $\triangleright$  вес удаляемого узла — предшественнику
7:      $\tilde{A}[i].\text{next} = A[i].\text{next}$   $\triangleright$  предшественник указывает на преемника удалённого
8:      $S.\text{PUSH\_BACK}(A[i])$   $\triangleright$  сохраняем удалённый узел для восстановления
9:    $\tilde{A} = \text{sort}(\tilde{A} \text{ по } x.\text{next})$   $\triangleright$  предшественник и (ещё присутствующий) удалённый узел
     теперь имеют общий  $\text{next} \Rightarrow$  встанут рядом
10:   $B = []$ 
11:   $i = 0$ 
12:  while  $i < N$  do  $\triangleright$  из каждой пары с общим  $\text{next}$  оставляем одну живую вершину
13:    if  $i + 1 < N$  and  $\tilde{A}[i].\text{next} = \tilde{A}[i + 1].\text{next}$  then
14:      if  $\tilde{A}[i].r = 1$  then  $\triangleright$  вершина с  $r=1$  — удалённая
15:         $\text{swap}(\tilde{A}[i], \tilde{A}[i + 1])$   $\triangleright$  ставим живую ( $r=0$ ) первой
16:         $B.\text{PUSH\_BACK}(\tilde{A}[i])$   $\triangleright$  в  $B$  кладём живую вершину
17:         $i += 2$ 
18:      else
19:         $B.\text{PUSH\_BACK}(\tilde{A}[i])$   $\triangleright$  одиночная вершина — просто переносим
20:         $i += 1$ 
    $\triangleright$  переопределяем  $\text{next}$ : старые  $\text{id} \mapsto$  новые (плотные) позиции в массиве
21:   $\tilde{B} = \text{sort}(B \text{ по } x.\text{next})$ 
22:   $i = 0; j = 0; k = 0$ 
23:  while  $j < N - |S|$  do
24:    while  $k < |S|$  and  $i = S[k].\text{id}$  do  $\triangleright$  пропускаем  $\text{id}$ , принадлежащие удалённым
25:       $i += 1; k += 1$ 
26:     $\tilde{B}[j].\text{next} = i$   $\triangleright$  цель указателя получает новый плотный индекс  $j$ 
27:     $i += 1; j += 1$ 
28:   $\hat{A} = \text{sort}(\tilde{B} \text{ по } x.\text{id})$   $\triangleright$  возвращаемся к порядку по  $\text{id}$ ; веса уже агрегированы
29: return  $(\hat{A}, S)$ 
```

Algorithm 8 RESTORE($\hat{A}, S, N$)

```
1:  $\tilde{S} = \text{sort}(S \text{ по } x.\text{next})$ 
2:  $\tilde{A} = \text{sort}(\hat{A} \text{ по } x.\text{next})$   $\triangleright i$  — по старым  $\text{id}$ ;  $k$  — по  $S$  в порядке  $\text{id}$ ;  $j$  — по живым в
   порядке  $\text{next}$ ;  $m$  — по  $S$  в порядке  $\text{next}$ 
3:  $i = 0$ ;  $j = 0$ ;  $k = 0$ ;  $m = 0$ 
4: while  $j < N - |S|$  do
5:   while  $k < |S|$  and  $i = S[k].\text{id}$  do
6:      $i += 1$ ;  $k += 1$   $\triangleright$  пропускаем старые  $\text{id}$  удалённых вершин
7:      $\tilde{A}[j].\text{next} = i$   $\triangleright$  восстанавливаем исходный  $\text{id}$  преемника
8:     if  $m < |S|$  and  $\tilde{A}[j].\text{next} = \tilde{S}[m].\text{next}$  then  $\triangleright$  между этим узлом и его
       преемником нужно вернуть удалённую вершину
9:        $\tilde{A}[j].\text{next} = \tilde{S}[m].\text{id}$   $\triangleright$  предшественник снова указывает на восстановленную
       вершину
10:       $\tilde{A}[j].w -= \tilde{S}[m].w$   $\triangleright$  теперь  $\tilde{A}[j].w = \text{pref}$  до предшественника
11:       $\tilde{S}[m].w += \tilde{A}[j].w$   $\triangleright$  теперь  $\tilde{S}[m].w = \text{pref}$  до восстановленной вершины
12:       $m += 1$ 
13:     $i += 1$ ;  $j += 1$ 
14:  $A = \text{sort}(x \in (\tilde{S} \cup \tilde{A}) \text{ по } \text{id})$   $\triangleright$  сливаем живые и восстановленные, сортируем по  $\text{id}$ 
15: return  $A$ 
```

Algorithm 9 EM_PREFIXSUMS(A)

Require: Массив A , отсортированный по id

Ensure: Для каждой вершины x значение $x.w$ равно $\text{pref}(x)$

```
1: if  $|A| \leq 1$  then
2:   return  $A$ 
3:  $(\hat{A}, S) = \text{COMPRESS}(A)$   $\triangleright$  удаляем независимое множество, агрегируя веса
4:  $\hat{A} = \text{EM\_PREFIXSUMS}(\hat{A})$   $\triangleright$  рекурсия на укороченном списке
5:  $A = \text{RESTORE}(\hat{A}, S, |A|)$   $\triangleright$  возвращаем удалённые вершины и дописываем им  $\text{pref}$ 
6: return  $A$ 
```

6.8. Оценка сложности

Один уровень рекурсии состоит из константного числа сортировок и линейных проходов по массивам, поэтому существует константа $c > 0$ такая, что работа одного уровня не превосходит $c \text{sort}(N)$.

Теорема 6.1

$$\mathbb{E}[T(N)] = O(\text{sort}(N)).$$

Доказательство. Докажем по индукции, что $\mathbb{E}[T(n)] \leq 4c \text{sort}(n)$.

Раскладывая по возможным значениям $|S|$ и применяя предположение индукции:

$$\mathbb{E}[T(n)] \leq \sum_{k=0}^{n-1} \Pr[|S| = k] \cdot 4c \text{sort}(n - k) + c \text{sort}(n).$$

Используем оценку $\text{sort}(n - k) \leq \frac{n-k}{B} \log_{M/B} \frac{n}{B}$:

$$\mathbb{E}[T(n)] \leq 4c \log_{M/B} \frac{n}{B} \sum_{k=0}^{n-1} \Pr[|S| = k] \frac{n-k}{B} + c \text{sort}(n) = \frac{4c \mathbb{E}[n - |S|]}{B} \log_{M/B} \frac{n}{B} + c \text{sort}(n).$$

Так как $\mathbb{E}[n - |S|] \leq \frac{3n}{4}$:

$$\mathbb{E}[T(n)] \leq 3c \text{sort}(n) + c \text{sort}(n) = 4c \text{sort}(n).$$

■

Таким образом,

$$\mathbb{E}[T(N)] = O(\text{sort}(N)) = O\left(\frac{N}{B} \log_{M/B} \frac{N}{B}\right).$$

7. Задача линейного программирования. Допустимые и оптимальные решения. Стандартная и каноническая формы. Существование оптимального решения (без доказательства). Лемма Фаркаша.

7.1. Постановка задачи

Определение : Задача линейного программирования (стандартная форма)

$$\max c^T x \quad \text{при} \quad Ax \leq b,$$

где $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, $c \in \mathbb{R}^n$. Неравенство $Ax \leq b$ понимается покомпонентно: $(Ax)_i \leq b_i$ для всех $i = 1, \dots, m$.

Определение : Каноническая форма

$$\min c^T x \quad \text{при} \quad Ax = b, \quad x \geq 0.$$

Обе формы эквивалентны; стандартная является удобной для теоретических рассуждений, каноническая — для алгоритмов.

7.2. Эквивалентные преобразования

- $\min c^T x \iff \max (-c)^T x$.
- $a_k^T x \geq b_k \iff -a_k^T x \leq -b_k$.
- $a_k^T x = b_k \iff \{a_k^T x \leq b_k, -a_k^T x \leq -b_k\}$.
- Свободная переменная x_j заменяется разностью $x_j = x_j^+ - x_j^-$, где $x_j^+, x_j^- \geq 0$.
- Неравенство $a_k^T x \leq b_k$ заменяется равенством $a_k^T x + s_k = b_k$, $s_k \geq 0$ (добавочная переменная; её смысл — остаток ресурса до границы ограничения).

Каноническая форма из стандартной. Дана задача $\max c^T x, Ax \leq b$.

1. Меняем знак: $\min (-c)^T x$.
2. Каждое неравенство $a_k^T x \leq b_k$ заменяем равенством $a_k^T x + s_k = b_k, s_k \geq 0$.
3. Каждую переменную x_j заменяем $x_j^+ - x_j^-, x_j^+, x_j^- \geq 0$.

Стандартная форма из канонической. Дана задача $\min c^T x, Ax = b, x \geq 0$. Каждое равенство $a_k^T x = b_k$ заменяем парой $a_k^T x \leq b_k$ и $-a_k^T x \leq -b_k$. Условие $x \geq 0$ записывается как $-x \leq 0$.

7.3. Допустимые и оптимальные решения

Определение : Множество допустимых решений

Множество

$$P = \{x \in \mathbb{R}^n : Ax \leq b\}$$

называется **множеством допустимых решений**. Оно является **выпуклым**:

$$\forall x, y \in P \forall \lambda \in (0, 1): \lambda x + (1 - \lambda)y \in P.$$

Это следует из того, что P есть пересечение полупространств $\{x : a_k^T x \leq b_k\}$, а каждое полупространство выпукло.

Определение : Оптимальное решение

Точка $x^* \in P$ называется **оптимальным решением**, если $x^* \in \arg \max_{x \in P} c^T x$.

Определение : Ограниченность

Задача называется **ограниченной**, если $P \neq \emptyset$ и

$$\exists M \in \mathbb{R} \forall x \in P: c^T x \leq M$$

(для задачи на максимум; для минимума аналогично $c^T x \geq M$).

7.4. Вершины

Определение : Вершина

Точка $z \in P$ называется **вершиной**, если её нельзя представить в виде нетривиальной выпуклой комбинации двух различных точек из P :

$$\nexists y, z' \in P, y \neq z', \nexists \lambda \in (0, 1): z = \lambda y + (1 - \lambda)z'.$$

Интуитивно, если задача ограничена, то оптимальное решение достигается в одной из вершин P .

Теорема 7.1

Если задача линейного программирования ограничена, то её оптимум достигается.

Количество вершин может быть экспоненциальным, поэтому простой перебор вершин неэффективен.

Пример

n -мерный куб $\{0 \leq x_i \leq 1 : i = 1, \dots, n\}$ задаётся $2n$ неравенствами и имеет 2^n вершин.

7.5. Лемма Фаркаша

Теорема 7.2: Лемма Фаркаша

Для любых $A \in \mathbb{R}^{m \times n}$, $c \in \mathbb{R}^n$ выполняется ровно одно из двух условий:

1. $\exists y \geq 0 : A^T y = c$;
2. $\exists z \in \mathbb{R}^n : Az \leq 0$ и $c^T z > 0$.

Геометрическая интерпретация

Обозначим строки матрицы A через $a_1^T, \dots, a_m^T$. Тогда $K = \{A^T y : y \geq 0\}$ — конус, натянутый на векторы $a_1, \dots, a_m$ (все их неотрицательные линейные комбинации). Условие (1) равносильно $c \in K$. Если $c \notin K$, то существует гиперплоскость $\{x : x^T z = 0\}$, отделяющая c от K : каждое $a_i^T z \leq 0$ (то есть $Az \leq 0$), а $c^T z > 0$.

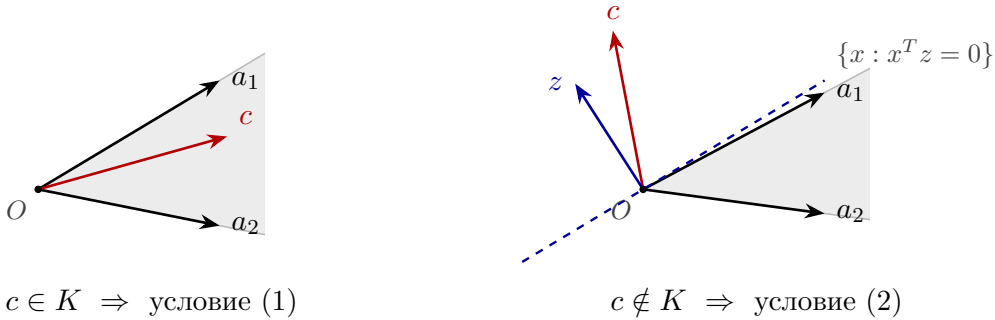


Рис. 15: Дихотомия леммы Фаркаша. Слева: c лежит в конусе $K = \{A^T y : y \geq 0\}$, то есть c — неотрицательная комбинация строк A (условие (1)). Справа: $c \notin K$; тогда конус и c разделяет гиперплоскость с нормалью z , для которой все $a_i^T z \leq 0$ (конус по одну сторону), а $c^T z > 0$ (условие (2)).

Доказательство

Условия (1) и (2) не могут выполняться одновременно. Если $A^T y = c$, $y \geq 0$ и $Az \leq 0$, то $c^T z = (A^T y)^T z = y^T (Az) \leq 0$, что противоречит $c^T z > 0$.

Если (1) не выполнено, то выполнено (2).

Рассмотрим конус $K = \{A^T y : y \geq 0\}$. Предположим, что $c \notin K$.

Выберем точку $\delta \in K$, ближайшую к c :

$$|\delta - c| = \inf_{u \in K} |u - c|.$$

Существование δ получается стандартным предельным переходом: берём $\delta_k \in K$ с $|c - \delta_k| \leq \inf |c - u| + 1/k$, выделяем сходящуюся подпоследовательность, используем замкнутость K .

Положим $z = c - \delta$. В двумерном подпространстве $\text{Lin}(c, \delta)$ из элементарной геометрии следует, что z ортогонален опорной прямой к конусу K в точке δ :

$$\delta^T z = 0, \quad c^T z > 0.$$

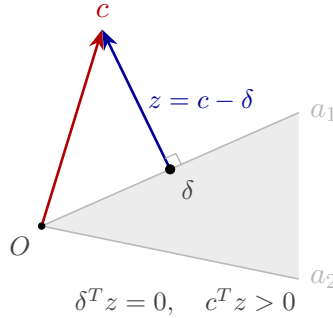


Рис. 16: Ближайшая к c точка конуса δ и вектор $z = c - \delta$. Поскольку δ — проекция c на K , вектор z ортогонален опорной прямой конуса в точке δ , откуда $\delta^T z = 0$ и $c^T z > 0$.

Докажем, что $Az \leq 0$, то есть $a_i^T z \leq 0$ для всех i .

Предположим противное: для некоторого k выполнено $a_k^T z > 0$. Так как $\delta \in K$ и $a_k \in K$, весь отрезок

$$[a_k, \delta] = \{\lambda a_k + (1 - \lambda)\delta : \lambda \in [0, 1]\}$$

лежит в K . Из условий $\delta^T z = 0$ и $a_k^T z > 0$ следует, что точки отрезка $[a_k, \delta]$ лежат по положительную сторону гиперплоскости $\{x : x^T z = 0\}$. Из геометрии в подпространстве $\text{Lin}(c, \delta, a_k)$ найдётся точка $\delta' \in [a_k, \delta]$ с $|c - \delta'| < |c - \delta|$, что противоречит выбору δ как ближайшей точки.

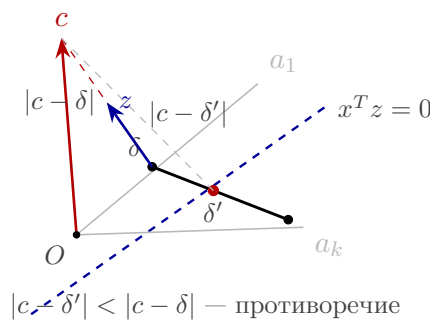


Рис. 17: Шаг от противного. Если бы какой-то a_k давал $a_k^T z > 0$, то отрезок $[a_k, \delta] \subset K$ заходил бы на положительную сторону гиперплоскости, и нашлась бы точка δ' конуса ближе к c , чем δ , — противоречие с минимальностью δ .

Следовательно, $a_i^T z \leq 0$ для всех i , то есть $Az \leq 0$.

Вместе с $c^T z > 0$ это и даёт условие (2). ■

8. Теорема о двойственности в линейном программировании (без доказательства леммы Фаркаша). Системы линейных неравенств. Полиномиальные алгоритмы решения лин. прог. Общая идея алгоритма внутренней точки. Преобразование задачи для нахождения какой-то внутренней точки.

8.1. Теорема о двойственности

Для $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, $c \in \mathbb{R}^n$ рассмотрим прямую и двойственную задачи:

Прямая задача ($\mathcal{P}$)	Двойственная задача ($\mathcal{D}$)
$\max c^T x$	$\min b^T y$
$Ax \leq b$	$A^T y = c$
	$y \geq 0$

Предложение 8.1: Слабая двойственность

Для любых допустимых x (задачи $\mathcal{P}$) и y (задачи $\mathcal{D}$) выполнено $c^T x \leq b^T y$.

Доказательство. Так как $A^T y = c$, имеем $c^T x = (A^T y)^T x = y^T (Ax)$. Далее, из $Ax \leq b$ и $y \geq 0$ следует $y^T (Ax) \leq y^T b = b^T y$. Следовательно, $c^T x \leq b^T y$. ■

Любой вектор y с условиями $A^T y = c$, $y \geq 0$ задаёт верхнюю оценку $b^T y$ на значение прямой задачи. Естественно искать наименьшую из таких оценок — это и приводит к задаче $\mathcal{D}$; симметрично, двойственная к $\mathcal{D}$ снова есть $\mathcal{P}$.

Предложение 8.2: $\mathcal{P}$ и $\mathcal{D}$ образуют двойственную пару

Задача $\mathcal{D}$ двойственна к $\mathcal{P}$, а задача $\mathcal{P}$ двойственна к $\mathcal{D}$.

Теорема 8.1: Сильная двойственность (теорема о двойственности)

Если одна из задач ($\mathcal{P}$) или ($\mathcal{D}$) имеет оптимальное решение, то и другая имеет оптимальное решение, причём

$$c^T x^* = b^T y^*,$$

где x^* и y^* — оптимальные решения прямой и двойственной задач.

Если целевая функция одной из задач не ограничена, то другая задача не имеет допустимых решений.

Доказательство. 1. Слабая двойственность уже доказана: $c^T x \leq b^T y$ для любых допустимых x, y . Отсюда, если обе задачи имеют допустимые решения, то обе ограничены, а значит, обе имеют оптимальные решения x^* и y^* .

2. Построение оптимального y^* по x^* .

Пусть x^* — оптимальное решение ($\mathcal{P}$). Перестановкой строк системы $Ax \leq b$ разобьём

ограничения на активные и неактивные:

$$A = \begin{pmatrix} A' \\ A'' \end{pmatrix}, \quad b = \begin{pmatrix} b' \\ b'' \end{pmatrix}, \quad A'x^* = b', \quad A''x^* < b''.$$

Покажем, что $\nexists z \in \mathbb{R}^n: A'z \leq 0$ и $c^T z > 0$. В самом деле, если бы такой z существовал, то для точки $x^{**} = x^* + \varepsilon z$ выполнялось бы $A'x^{**} = A'x^* + \varepsilon A'z \leq b'$ (так как $A'x^* = b'$ и $A'z \leq 0$), а из строгого неравенства $A''x^* < b''$ при достаточно малом $\varepsilon > 0$ следовало бы $A''x^{**} \leq b''$. Значит, $Ax^{**} \leq b$, то есть x^{**} допустима, но $c^T x^{**} = c^T x^* + \varepsilon c^T z > c^T x^*$ — противоречие с оптимальностью x^* .

Следовательно, для матрицы A' второе условие леммы Фаркаша не выполнено, поэтому выполнено первое:

$$\exists y' \geq 0: (A')^T y' = c.$$

Положим $y = \begin{pmatrix} y' \\ 0 \end{pmatrix} \in \mathbb{R}^m$. Тогда $y \geq 0$ и $A^T y = (A')^T y' + (A'')^T 0 = c$, то есть y — допустимое решение $(\mathcal{D})$.

Проверим значения целевых функций:

$$b^T y = (b')^T y' = (A'x^*)^T y' = (x^*)^T (A')^T y' = (x^*)^T c = c^T x^*.$$

По слабой двойственности $b^T y = c^T x^*$ — наибольшее достижимое значение, поэтому y оптимально в $(\mathcal{D})$, а x^* — в $(\mathcal{P})$.

3. Симметричность.

Покажем, что если оптимальное решение имеет задача $(\mathcal{D})$, то и $(\mathcal{P})$ тоже. Перепишем $(\mathcal{D})$ в виде $\max (-b^T y)$, а ограничения $A^T y = c, y \geq 0$ — системой неравенств:

$$\max (-b^T y) \quad \text{при} \quad \begin{cases} A^T y \leq c, \\ -A^T y \leq -c, \\ -y \leq 0. \end{cases}$$

Задача, двойственная к этой, в точности эквивалентна исходной задаче $(\mathcal{P})$: $\max c^T x, Ax \leq b$. Применяя к ней ту же конструкцию, по оптимальному решению $(\mathcal{D})$ строим допустимое (и потому оптимальное) решение $(\mathcal{P})$. ■

8.2. Системы линейных неравенств

Замечание : Сведение ЛП к задаче выполнимости

Пусть имеется алгоритм, за полиномиальное время решающий задачу выполнимости системы линейных неравенств (находит решение либо сообщает, что решений нет). Тогда нахождение оптимального решения задачи ЛП сводится к проверке выполнимости одной системы. Действительно, пары (x, y) являются оптимальными решениями $(\mathcal{P})$ и $(\mathcal{D})$ тогда и только тогда, когда выполнена система

$$Ax \leq b, \quad A^T y = c, \quad y \geq 0, \quad b^T y - c^T x \leq 0.$$

По слабой двойственности для любых допустимых x, y всегда $b^T y - c^T x \geq 0$, поэтому добавленное неравенство $b^T y - c^T x \leq 0$ эквивалентно равенству $b^T y = c^T x$, то есть условию оптимальности.

При желании равенство $A^T y = c$ заменяется парой неравенств $A^T y \leq c$ и $-A^T y \leq -c$.

Таким образом, задача ЛП полностью сводится к проверке выполнимости системы линейных неравенств, и наоборот.

8.3. Полиномиальные алгоритмы

Определение : Длина входа

Пусть все элементы A, b, c рациональны и кодируются не более чем w битами каждый. Тогда длиной входа называется

$$L = (mn + m + n)w = \Theta(mnw).$$

Определение : Полиномиальный алгоритм

Алгоритм решения задачи ЛП называется *полиномиальным*, если его время работы равно $O(L^{O(1)})$.

Симплекс-метод обходит вершины многогранника, двигаясь по рёбрам в направлении вектора c . Большинство его вариантов имеют экспоненциальную сложность в худшем случае, но хорошо работают на практике. Полиномиальные алгоритмы: метод эллипсоидов (Хачиян, 1979) и метод внутренней точки (Кармаркар, 1984).

8.4. Метод внутренней точки

Определение : Внутренняя точка

Точка x_0 называется *внутренней допустимой точкой* для системы $Ax \leq b$, если $Ax_0 < b$ покомпонентно.

Нахождение внутренней точки

Замечание : Как найти внутреннюю точку

Для поиска внутренней точки исходной системы $Ax \leq b$ рассмотрим вспомогательную задачу

$$\max t \quad \text{при} \quad Ax + t \mathbf{1} \leq b,$$

где $\mathbf{1} = (1, \dots, 1)^T \in \mathbb{R}^m$. Если её оптимальное значение $t^* > 0$, то соответствующая точка x^* удовлетворяет $Ax^* < b$, то есть является внутренней для исходной системы. У вспомогательной задачи всегда есть очевидная внутренняя точка, например

$$x = 0, \quad t = - \max_{1 \leq i \leq m} |b_i| - 1,$$

поскольку тогда $t < b_i$ для всех i , а значит $Ax + t \mathbf{1} = t \mathbf{1} < b$.

Барьерная задача и центральный путь

Пусть строки матрицы A имеют вид $a_1^T, \dots, a_m^T$, а $s_i(x) := b_i - a_i^T x$ обозначают зазоры до ограничений.

Определение : Логарифмический барьер

Для x во внутренности многогранника ($Ax < b$) положим

$$g(x) := \sum_{i=1}^m \ln s_i(x) = \sum_{i=1}^m \ln(b_i - a_i^T x).$$

Для параметра $w > 0$ рассмотрим **барьерную задачу**

$$\max (w c^T x + g(x)) \quad \text{при} \quad Ax \leq b.$$

Одно из её решений обозначим x_w^* .

Смысл логарифмических слагаемых: при приближении x к границе многогранника какой-нибудь зазор $s_i(x) \rightarrow 0^+$, поэтому $g(x) \rightarrow -\infty$. Барьер g «отталкивает» от границы, и максимизатор x_w^* остаётся строго внутри допустимой области. При этом слагаемое $w c^T x$ тянет точку в сторону роста целевой функции; баланс между ними и определяет положение x_w^* .

Предложение 8.3: Центральный путь

Семейство $\{x_w^*\}_{w>0}$ образует *центральный путь*. Поскольку

$$\arg \max_{Ax < b} (w c^T x + g(x)) = \arg \max_{Ax < b} (c^T x + \frac{1}{w} g(x)),$$

множитель при барьере есть $\frac{1}{w}$. При $w \rightarrow +\infty$ этот множитель убывает, влияние барьера ослабевает, и точки x_w^* приближаются к оптимальному решению исходной задачи; наоборот, при $w \rightarrow 0^+$ доминирует барьер, и x_w^* стремится к «аналитическому центру» многогранника.

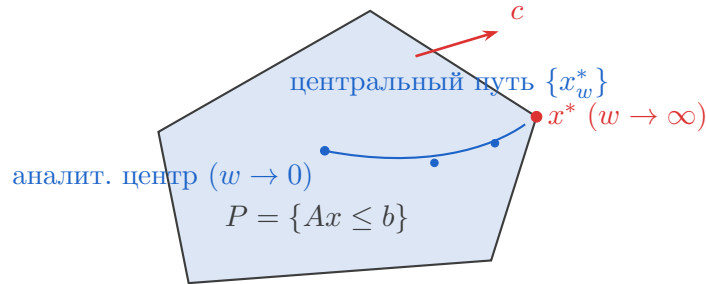


Рис. 18: Центральный путь $\{x_w^*\}$ внутри многогранника P . При малом w доминирует барьер и точка близка к аналитическому центру; при $w \rightarrow \infty$ вес барьера $\frac{1}{w}$ убывает, и центральные точки движутся вдоль пути к оптимальной вершине x^* . Метод следует именно вдоль этой кривой.

Идея алгоритма

Метод следует вдоль центрального пути. Начав с внутренней точки x_0 , он постепенно увеличивает параметр w (то есть уменьшает вес барьера $\frac{1}{w}$) и на каждом шаге сдвигает текущую точку ближе к соответствующей центральной точке x_w^* . Поскольку при $w \rightarrow +\infty$ центральные точки приближаются к оптимуму исходной задачи, итерации сходятся к оптимальному решению. При аккуратном выборе шага метод даёт полиномиальный алгоритм решения задачи ЛП.

9. Задача о максимальном потоке в сети. Формулировка в лин. прог. Дополняющие пути. Алгоритм Форда–Фалкерсона. Анализ трудоёмкости для целочисленной задачи о максимальном потоке. Теорема о максимальном потоке и минимальном разрезе

Определение : Транспортная сеть

Пусть дан ориентированный граф $G = (V, E, c)$, где функция

$$c: E \rightarrow \mathbb{R}_{\geq 0}$$

задаёт пропускные способности рёбер.

Граф G называется *транспортной сетью*, если в нём выделены две вершины:

$$s \in V \quad (\text{исток}), \quad t \in V \quad (\text{сток}),$$

причём

$$\deg^-(s) = 0, \quad \deg^+(t) = 0.$$

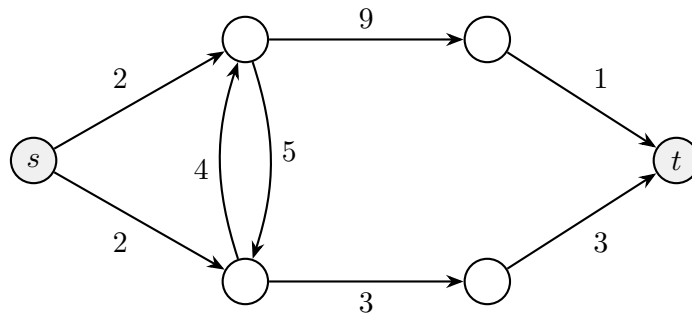


Рис. 19: Пример транспортной сети. Числа у рёбер — пропускные способности $c(u, v)$.

Определение : Поток

Потоком в транспортной сети называется функция $f: E \rightarrow \mathbb{R}_{\geq 0}$, такая что:

1) **Ограничение пропускной способности:**

$$0 \leq f(u, v) \leq c(u, v) \quad \forall (u, v) \in E.$$

2) **Закон сохранения потока:** для любой вершины $u \in V \setminus \{s, t\}$ выполнено

$$\sum_{(v,u) \in E} f(v, u) = \sum_{(u,w) \in E} f(u, w).$$

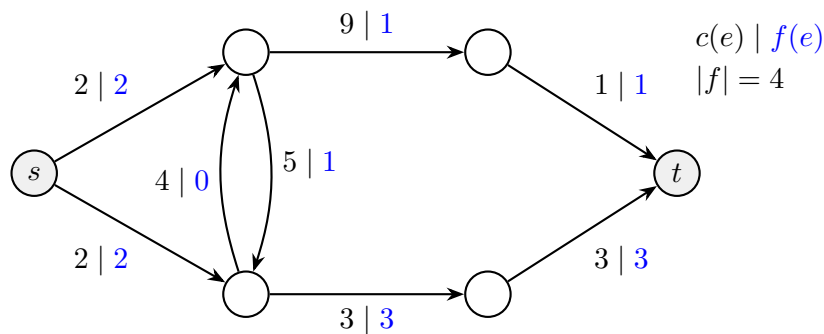


Рис. 20: Поток величины 4 в транспортной сети из рис. 19. Рядом с каждым ребром указана пара *пропускная способность / поток*.

Определение : Величина потока

Величиной потока f называется число

$$|f| = \sum_{(s,v) \in E} f(s,v).$$

Эквивалентно,

$$|f| = \sum_{(u,t) \in E} f(u,t).$$

Предложение 9.1

Для любого потока f выполнено

$$\sum_{(s,v) \in E} f(s,v) = \sum_{(u,t) \in E} f(u,t).$$

Доказательство. Просуммируем закон сохранения потока по всем вершинам из множества $V \setminus \{s, t\}$. Все слагаемые, соответствующие внутренним рёбрам, сократятся попарно, и останется только суммарный поток, выходящий из истока, и суммарный поток, входящий в сток. ■

9.1. Задача о максимальном потоке

Требуется найти поток максимальной величины.

Замечание

Многие комбинаторные задачи допускают сведение к задаче о максимальном потоке.

Замечание

Будем считать, что в графе нет кратных рёбер. Это не ограничивает общность: несколько параллельных рёбер можно заменить одним ребром с суммарной пропускной способностью.

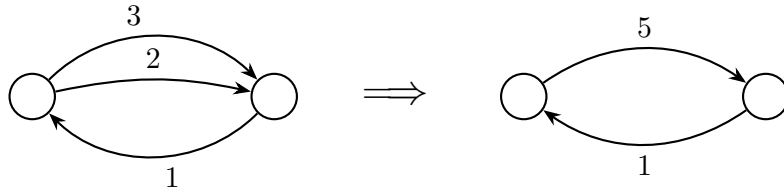


Рис. 21: Два параллельных ребра $u \rightarrow v$ с пропускными способностями 3 и 2 заменяются одним ребром с суммарной пропускной способностью 5. Ребро $v \rightarrow u$ остаётся без изменений.

Предложение 9.2

Задача о максимальном потоке допускает формулировку в виде задачи линейного программирования:

$$\max \sum_{(s,v) \in E} f(s,v)$$

при ограничениях

$$0 \leq f(u,v) \leq c(u,v) \quad \forall (u,v) \in E,$$

$$\sum_{(v,u) \in E} f(v,u) - \sum_{(u,w) \in E} f(u,w) = 0 \quad \forall u \in V \setminus \{s,t\}.$$

9.2. Дополняющие пути и остаточная сеть

Начинаем с нулевого потока $f \equiv 0$ и на каждом шаге пытаемся увеличить поток, пропуская дополнительную порцию из s в t по некоторому допустимому пути. Однако иногда для увеличения общего потока необходимо не только добавлять поток по прямым рёбрам, но и частично *отменять* ранее проведённый поток по некоторым рёбрам. Для формализации этого вводится остаточная сеть.

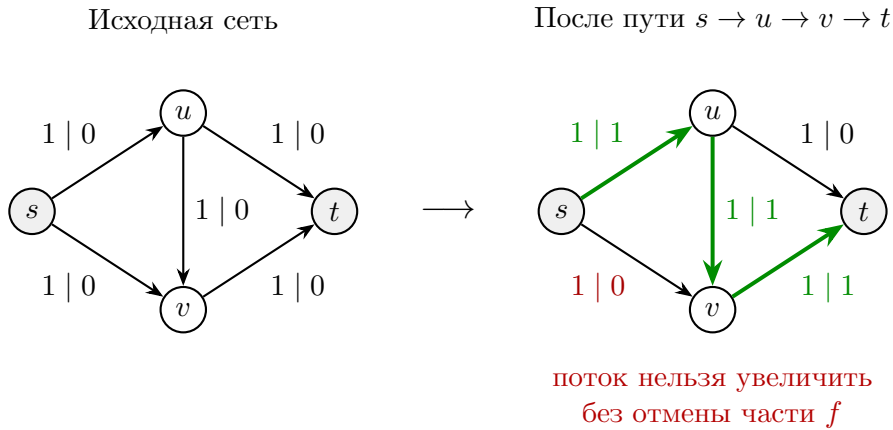


Рис. 22: Жадный выбор пути $s \rightarrow u \rightarrow v \rightarrow t$ насыщает ребро $v \rightarrow t$ и блокирует путь $s \rightarrow v \rightarrow t$. Для получения оптимального потока величины 2 необходимо «отменить» единицу потока через ребро $u \rightarrow v$ с помощью обратного ребра остаточной сети.

Определение

Пусть f — поток в сети $G = (V, E, c)$. *Остаточной сетью* называется ориентированный граф

$$G_f = (V, E_f, c_f),$$

который строится следующим образом. Для каждого ребра $(u, v) \in E$:

- 1) если $c(u, v) - f(u, v) > 0$, то в E_f добавляется *прямое* ребро (u, v) остаточной пропускной способности

$$c_f(u, v) = c(u, v) - f(u, v);$$

- 2) если $f(u, v) > 0$, то в E_f добавляется *обратное* ребро (v, u) остаточной пропускной способности

$$c_f(v, u) = f(u, v).$$

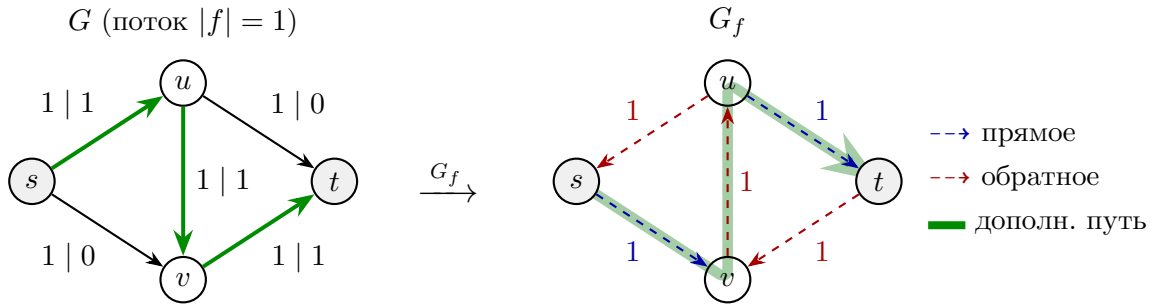


Рис. 23: Построение остаточной сети G_f . Синие пунктирные рёбра — прямые (можно добавить поток), красные пунктирные — обратные (можно отменить поток). Зелёным выделен дополняющий путь $s \rightarrow v \rightarrow u \rightarrow t$ в G_f , позволяющий увеличить поток до 2.

Замечание

В остаточной сети могут появляться кратные рёбра, если прямое и обратное направления возникают независимо. Это не влияет на корректность алгоритма.

Определение

Путь

$$P = (v_0 = s, v_1, \dots, v_k = t)$$

в графе G_f называется *дополняющим путём* для потока f .

Определение

Пусть $P = (v_0 = s, v_1, \dots, v_k = t)$ — дополняющий путь в G_f . Его *пропускной способностью* называется число

$$\delta(P) = \min_{0 \leq i < k} c_f(v_i, v_{i+1}).$$

Предложение 9.3

Пусть $P = (v_0 = s, v_1, \dots, v_k = t)$ — дополняющий путь в G_f и $\delta = \delta(P)$. Тогда можно построить новый поток f' величины

$$|f'| = |f| + \delta.$$

Доказательство. Для каждого ребра пути действуем следующим образом:

- 1) если ребро (v_i, v_{i+1}) является прямым ребром исходной сети, то полагаем

$$f'(v_i, v_{i+1}) = f(v_i, v_{i+1}) + \delta;$$

- 2) если ребро (v_i, v_{i+1}) является обратным, то есть в исходной сети имеется ребро (v_{i+1}, v_i) , то полагаем

$$f'(v_{i+1}, v_i) = f(v_{i+1}, v_i) - \delta.$$

На всех остальных рёбрах поток не меняется.

Поскольку $\delta \leq c_f(v_i, v_{i+1})$ для каждого ребра пути, ограничения пропускной способности сохраняются. Закон сохранения потока также сохраняется: в каждой промежуточной вершине изменение входящего потока компенсируется изменением исходящего потока на ту же величину δ .

Остаётся проверить, что величина потока увеличилась на δ . По определению $|f| = \sum_{(s,v) \in E} f(s, v)$. Первое ребро дополняющего пути, выходящее из s , является прямым ребром исходной сети, поэтому на этом ребре поток увеличивается на δ . Другие изменения, связанные с вершиной s , отсутствуют. Следовательно,

$$|f'| = \sum_{(s,v) \in E} f'(s, v) = \sum_{(s,v) \in E} f(s, v) + \delta = |f| + \delta.$$

■

9.3. Алгоритм Форда–Фалкерсона

Утверждение 9.1

Алгоритм Форда–Фалкерсона строит поток итеративно следующим образом:

- 1) положить $f \equiv 0$;
- 2) построить остаточную сеть G_f ;
- 3) найти в G_f путь P из s в t ;
- 4) если такого пути нет, остановиться: текущий поток f является максимальным;
- 5) если путь P найден, увеличить поток вдоль него на величину $\delta(P)$, получив новый поток f ;
- 6) перестроить остаточную сеть G_f для нового потока и перейти к шагу 3.

Замечание

На шаге поиска пути можно использовать как DFS, так и BFS. От выбора способа поиска существенно зависит итоговая оценка времени работы.

9.4. s - t -разрез

Определение

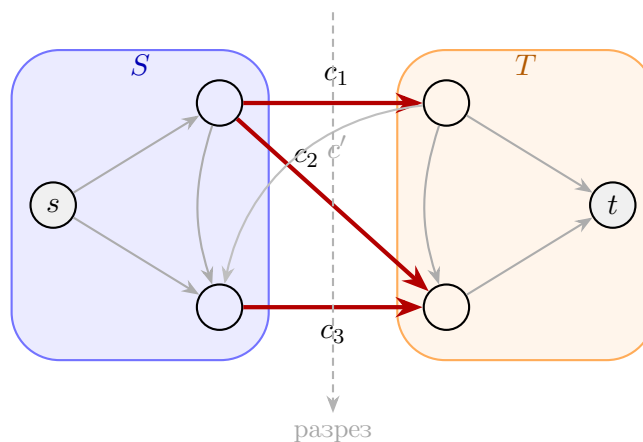
s - t -разрезом называется разбиение множества вершин $V = S \sqcup T$, $s \in S$, $t \in T$.

Определение

Величиной разреза (S, T) называется сумма пропускных способностей всех рёбер, идущих из S в T :

$$c(S, T) = \sum_{\substack{(u,v) \in E \\ u \in S, v \in T}} c(u, v).$$

Рёбра из T в S в эту сумму не входят.



$$c(S, T) = c_1 + c_2 + c_3$$

(ребро $c' : T \rightarrow S$ не учитывается)

Рис. 24: s - t -разрез (S, T) и его величина $c(S, T)$. Рёбра из S в T с пропускными способностями c_1, c_2, c_3 входят в сумму; ребро из T в S с пропускной способностью c' не учитывается.

Предложение 9.4

Для любого потока f и любого s - t -разреза (S, T) выполнено неравенство $|f| \leq c(S, T)$.

Доказательство. Рассмотрим сумму

$$\sum_{u \in S} \left(\sum_{(v,u) \in E} f(v, u) - \sum_{(u,w) \in E} f(u, w) \right).$$

Для всех вершин из $S \setminus \{s\}$ эта величина равна нулю по закону сохранения потока, а для вершины s она равна $-|f|$. После сокращения внутренних рёбер получаем

$$|f| = \sum_{\substack{(u,v) \in E \\ u \in S, v \in T}} f(u, v) - \sum_{\substack{(u,v) \in E \\ u \in T, v \in S}} f(u, v).$$

Следовательно,

$$|f| \leq \sum_{\substack{(u,v) \in E \\ u \in S, v \in T}} f(u,v) \leq \sum_{\substack{(u,v) \in E \\ u \in S, v \in T}} c(u,v) = c(S,T).$$

■

9.5. Теорема о максимальном потоке и минимальном разрезе

Теорема 9.1

Для потока f в транспортной сети эквивалентны следующие условия:

- 1) f является максимальным потоком;
- 2) в остаточной сети G_f нет дополняющего пути из s в t ;
- 3) существует s - t -разрез (S, T) такой, что

$$|f| = c(S, T).$$

Доказательство. Докажем циклически $1) \Rightarrow 2) \Rightarrow 3) \Rightarrow 1)$.

$1) \Rightarrow 2)$. Если бы в G_f существовал дополняющий путь, то по предыдущему предложению поток можно было бы увеличить. Это противоречит максимальной f .

$2) \Rightarrow 3)$. Пусть S — множество вершин, достижимых из s в остаточной сети G_f , а $T = V \setminus S$. Тогда $s \in S$, а $t \notin S$, поскольку по предположению пути из s в t в G_f нет.

Покажем, что $|f| = c(S, T)$. Из доказательства оценки для разреза:

$$|f| = \sum_{\substack{(u,v) \in E \\ u \in S, v \in T}} f(u,v) - \sum_{\substack{(u,v) \in E \\ u \in T, v \in S}} f(u,v).$$

Используем определение множества S .

Если $(u, v) \in E$, $u \in S$, $v \in T$: в G_f нет прямого ребра (u, v) , иначе v была бы достижима из s . Значит,

$$c(u, v) - f(u, v) = 0, \quad \text{то есть} \quad f(u, v) = c(u, v).$$

Если $(u, v) \in E$, $u \in T$, $v \in S$: в G_f нет обратного ребра (v, u) , иначе u была бы достижима из s . Значит, $f(u, v) = 0$.

Следовательно,

$$|f| = \sum_{\substack{(u,v) \in E \\ u \in S, v \in T}} c(u,v) = c(S, T).$$

$3) \Rightarrow 1)$. Для любого потока f' и любого разреза (S, T) верно $|f'| \leq c(S, T)$. Если для некоторого потока f выполнено $|f| = c(S, T)$, то никакой поток не может иметь величину больше $|f|$. Следовательно, f максимален. ■

Предложение 9.5

Максимальная величина потока равна минимальной величине s - t -разреза.

Доказательство. Теорема утверждает существование потока и разреза, для которых $|f| = c(S, T)$. С другой стороны, для любого потока и любого разреза выполнено $|f| \leq c(S, T)$. Отсюда следует утверждение. ■

9.6. Оценки времени работы

Предложение 9.6

Одна итерация алгоритма Форда–Фалкерсона работает за время $O(|E|)$, если поиск дополняющего пути выполняется обходом графа в глубину или в ширину.

Доказательство. Построение остаточной сети и поиск пути в ней занимают линейное по числу рёбер время. ■

Предложение 9.7

Если все пропускные способности целочисленны, то алгоритм Форда–Фалкерсона завершится не более чем за $|f_{\max}|$ итераций, где $f_{\max}$ — максимальный поток.

Доказательство. При целочисленных пропускных способностях каждая остаточная пропускная способность также целочисленна. Следовательно, на каждой итерации $\delta(P) \geq 1$, и величина потока увеличивается хотя бы на единицу. ■

Следствие

В случае целочисленных пропускных способностей время работы алгоритма Форда–Фалкерсона оценивается как $O(|E| \cdot |f_{\max}|)$. Найденный поток при этом целочисленен.

Замечание

Для произвольных вещественных пропускных способностей алгоритм Форда–Фалкерсона при неудачном выборе дополняющих путей может работать очень долго; в общем случае возможны патологические сценарии.

Утверждение 9.2

Если на каждом шаге искать кратчайший по числу рёбер дополняющий путь с помощью BFS, получается алгоритм Эдмондса–Карпа со временем работы $O(|V| \cdot |E|^2)$.

Замечание

Существуют и более быстрые алгоритмы: алгоритм Диница — $O(|V|^2|E|)$ и алгоритм Орлина (2013) — $O(|V||E|)$.

10. Приближённые алгоритмы. Задача о вершинном покрытии минимальной мощности, приближённый полиномиальный алгоритм для её решения. Задача о вершинном покрытии минимальной стоимости, её приближённое решение при помощи лин. прог.

Многие важные задачи оптимизации являются NP-полными: для них неизвестно полиномиальных алгоритмов, а нахождение точного ответа, по всей видимости, требует экспонен-

циального времени. На практике такие задачи обычно решаются либо эвристиками (без гарантий качества), либо *приближёнными алгоритмами* — полиномиальными методами, гарантирующими ответ, близкий к оптимальному.

Определение : Приближённый алгоритм (ρ -приближение)

Пусть дана задача оптимизации $f: \text{input} \rightarrow \mathbb{R}$, решаемая на максимум или минимум. Для $\rho \geq 1$ алгоритм g называется ρ -приближением, если для любого входа x :

$$g(x) \geq \frac{f^*(x)}{\rho} \quad (\text{для задачи максимизации}),$$

$$g(x) \leq f^*(x) \cdot \rho \quad (\text{для задачи минимизации}),$$

где $f^*(x)$ — оптимальное значение на входе x .

Замечание

Величина ρ называется *коэффициентом приближения* (approximation ratio). Чем меньше ρ , тем лучше алгоритм. Значение $\rho = 1$ соответствует точному решению.

10.1. Задача о вершинном покрытии

Определение : Вершинное покрытие

Дан граф $G = (V, E)$. *Вершинное покрытие* — это подмножество $C \subseteq V$ такое, что для каждого ребра $e \in E$ хотя бы один из его концов принадлежит C :

$$\forall \{u, v\} \in E \quad \exists w \in C: w \in \{u, v\}.$$

Задача: найти вершинное покрытие минимального размера $|C|$.

Задача о вершинном покрытии является NP-полной. Для неё неизвестно полиномиальных алгоритмов (в стандартной теории $P \neq NP$). Однако существует простой 2-приближённый алгоритм, основанный на жадном построении паросочетания.

Алгоритм жадного паросочетания (2-приближение)

1. Выбрать произвольное ребро $\{u, v\} \in E$.
2. Добавить обе вершины в покрытие: $C := C \cup \{u, v\}$.
3. Удалить из графа u, v и все инцидентные им рёбра: $G := G \setminus \{u, v\}$.
4. Если $E(G) \neq \emptyset$, перейти к шагу 1.

На выходе C является вершинным покрытием, поскольку каждое ребро было «обработано» на некотором шаге.

Утверждение 10.1: Алгоритм жадного паросочетания даёт 2-приближение

Пусть C^* — минимальное вершинное покрытие. Тогда $|C| \leq 2|C^*|$.

Доказательство. В ходе алгоритма строится набор рёбер M (паросочетание): на каждом

шаге выбирается ребро $\{u, v\}$, причём после удаления u и v ни одно из последующих рёбер не касается u или v . Таким образом, M — паросочетание размера $|C|/2$.

Так как C^* является вершинным покрытием, каждое ребро из M должно иметь хотя бы один конец в C^* . Поскольку рёбра паросочетания не делят вершин, различным рёбрам из M соответствуют различные вершины из C^* . Следовательно,

$$|C^*| \geq |M| = \frac{|C|}{2} \implies |C| \leq 2|C^*|.$$

■

10.2. Задача о взвешенном вершинном покрытии

Дан граф $G = (V, E)$ и функция весов $w: V \rightarrow \mathbb{R}_{>0}$. Требуется найти вершинное покрытие $C \subseteq V$ минимального суммарного веса:

$$\sum_{v \in C} w(v) \rightarrow \min.$$

Алгоритм жадного паросочетания в этом случае *не работает* с прежней гарантией, поскольку не учитывает веса вершин. Задача о взвешенном вершинном покрытии также NP-полная (эквивалентна по трудности невзвешенному случаю).

Сведение к целочисленному линейному программированию

Введём переменные $x_v \in \{0, 1\}$ для каждой вершины v : $x_v = 1$ означает, что вершина v включена в покрытие. Задача записывается как:

$$\begin{cases} \min_x \sum_{v \in V} w(v) \cdot x_v, \\ x_u + x_v \geq 1 \quad \forall \{u, v\} \in E, \\ 0 \leq x_v \leq 1 \quad \forall v \in V, \\ x_v \in \{0, 1\}. \end{cases}$$

2-приближённый алгоритм через LP-релаксацию

Снимем условие целочисленности и решим непрерывную LP-релаксацию (без ограничения $x_v \in \{0, 1\}$):

$$\begin{cases} \min \sum_{v \in V} w(v) \cdot x_v, \\ x_u + x_v \geq 1 \quad \forall \{u, v\} \in E, \\ 0 \leq x_v \leq 1 \quad \forall v \in V. \end{cases}$$

LP-релаксация решается за полиномиальное время, и мы находим оптимальное дробное решение $\{x_v^*\}$.

Построим целочисленное решение *округлением*:

$$C = \left\{ v \in V \mid x_v^* \geq \frac{1}{2} \right\}, \quad \bar{x}_v = \begin{cases} 1, & v \in C, \\ 0, & v \notin C. \end{cases}$$

Теорема 10.1: О 2-приближении через LP-релаксацию

Множество C , построенное описанным выше методом, является вершинным покрытием и 2-приближением задачи о взвешенном вершинном покрытии.

Доказательство. *Корректность покрытия.* Для каждого ребра $\{u, v\} \in E$ из ограничения LP: $x_u^* + x_v^* \geq 1$. Следовательно, хотя бы одно из двух неравенств $x_u^* \geq \frac{1}{2}$ или $x_v^* \geq \frac{1}{2}$ выполнено, то есть хотя бы одна из вершин попадает в C . Значит, C — вершинное покрытие.

Оценка качества. Так как $\bar{x}_v = 1$ только при $x_v^* \geq \frac{1}{2}$, имеем $\bar{x}_v \leq 2x_v^*$ для всех v . Поэтому:

$$\sum_{v \in V} w(v) \bar{x}_v \leq \sum_{v \in V} w(v) \cdot 2x_v^* = 2 \sum_{v \in V} w(v) x_v^* \leq 2 \sum_{v \in V} w(v) x_v^{**},$$

где $\{x_v^{**}\}$ — оптимальное целочисленное решение (оно допустимо для LP, поэтому $\sum w(v)x_v^* \leq \sum w(v)x_v^{**}$).

Следовательно, $\sum_{v \in C} w(v) \leq 2 \sum_{v \in V} w(v) x_v^{**}$. ■

Замечание

Эта задача является ещё и более оптимальной: известно, что $(2 - \varepsilon)$ -приближение для вершинного покрытия крайне сложно или, по-видимому, невозможно найти (при стандартных предположениях о сложности).

11. Метрическая задача коммивояжёра. Алгоритм двойного дерева и алгоритм Кристофидеса–Сердюкова для её приближённого решения

Определение : Приближённый алгоритм (ρ -приближение)

Пусть дана задача оптимизации $f: \text{input} \rightarrow \mathbb{R}$, решаемая на максимум или минимум. Для $\rho \geq 1$ алгоритм g называется ρ -приближением, если для любого входа x :

$$\begin{aligned} g(x) &\geq \frac{f^*(x)}{\rho} \quad (\text{для задачи максимизации}), \\ g(x) &\leq f^*(x) \cdot \rho \quad (\text{для задачи минимизации}), \end{aligned}$$

где $f^*(x)$ — оптимальное значение на входе x .

Замечание

Величина ρ называется *коэффициентом приближения* (approximation ratio). Чем меньше ρ , тем лучше алгоритм. Значение $\rho = 1$ соответствует точному решению.

11.1. Задача коммивояжёра (TSP)

Определение : Задача коммивояжёра (Travelling Salesman Problem)

Дан полный граф $K_n = G(V, E)$ с весами $c: E \rightarrow \mathbb{R}^+$ на рёбрах. Требуется найти гамильтонов цикл H (цикл, проходящий через каждую вершину ровно один раз) минимального веса:

$$\sum_{e \in E(H)} c(e) \rightarrow \min.$$

Задача о гамильтоновом цикле (даже без весов) является NP-полной, поэтому TSP — NP-полная задача. Более того, задача коммивояжёра в общем случае *не имеет* хорошей приближённой аппроксимации:

Предложение 11.1: Об иносупростируемости общего TSP

Для любого $\rho \geq 1$: если существует ρ -приближённый полиномиальный алгоритм для задачи коммивояжёра, то существует полиномиальный алгоритм для задачи о гамильтоновом цикле (и, следовательно, $P = NP$).

Доказательство. Пусть дан граф $\tilde{G}(V, \tilde{E})$, и мы хотим проверить, есть ли в нём гамильтонов цикл. Построим полный граф $K_{|V|}$ с весами:

$$c(u, v) = \begin{cases} 1, & \{u, v\} \in \tilde{E}, \\ M, & \{u, v\} \notin \tilde{E}, \end{cases}$$

где M — очень большое число (например, $M = \rho \cdot |V| + 1$). Запустим ρ -приближённый алгоритм для TSP на этом графе. Если в $\tilde{G}$ есть гамильтонов цикл, то оптимальное решение имеет вес $|V|$, и алгоритм вернёт решение веса $\leq \rho \cdot |V| < M$. Если гамильтонова цикла нет, любой цикл использует хотя бы одно «тяжёлое» ребро, и его вес $\geq M > \rho \cdot |V|$. Таким образом, по ответу алгоритма можно отличить два случая. ■

11.2. Метрическая задача коммивояжёра

Ситуация улучшается, если потребовать от весов выполнения *неравенства треугольника*:

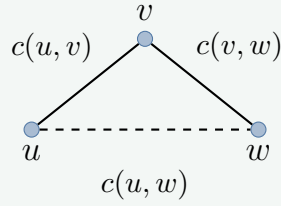
Определение : Метрическая задача коммивояжёра

TSP называется *метрическим*, если веса рёбер удовлетворяют неравенству треугольника:

$$\forall u, v, w \in V: \quad c(u, v) + c(v, w) \geq c(u, w).$$

Пример : Примеры метрических весов

1. $V \subseteq \mathbb{R}^2$, c — евклидово расстояние (Евклидова задача коммивояжёра).
2. $c: E \rightarrow \{1, 2\}$ — весовая функция, принимающая только значения 1 и 2. Это также удовлетворяет неравенству треугольника ($1 + 1 \geq 1$, $1 + 2 \geq 2$, $2 + 2 \geq 2$).



Утверждение 11.1: Обобщённое неравенство треугольника

В метрическом пространстве для любого пути $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k$:

$$\sum_{i=1}^{k-1} c(v_i, v_{i+1}) \geq c(v_1, v_k).$$

Доказательство. Индукция по числу промежуточных вершин. База $k = 2$: тривиально. Шаг: для пути $v_1, \dots, v_k$ по индукционному предположению $\sum_{i=1}^{k-2} c(v_i, v_{i+1}) \geq c(v_1, v_{k-1})$. Тогда

$$\sum_{i=1}^{k-1} c(v_i, v_{i+1}) \geq c(v_1, v_{k-1}) + c(v_{k-1}, v_k) \geq c(v_1, v_k),$$

где последнее неравенство — неравенство треугольника. ■

Алгоритм двойного остова (2-приближение для метрического TSP)

Алгоритм:

1. Найти минимальное остовное дерево T (алгоритмом Прима или Краскала).
2. Продублировать каждое ребро дерева T . Тогда все вершины получают чётную степень, и в полученном мультиграфе существует *Эйлеров цикл* $u_1 \rightarrow u_2 \rightarrow \dots \rightarrow u_k$.
3. Построить гамильтонов цикл $\tilde{H}$: обходим вершины Эйлерова цикла по порядку; если вершина u_i уже встречалась ранее, не добавляем её повторно, а переходим к u_{i+1} («срезаем» вершину, используя метрическое свойство).

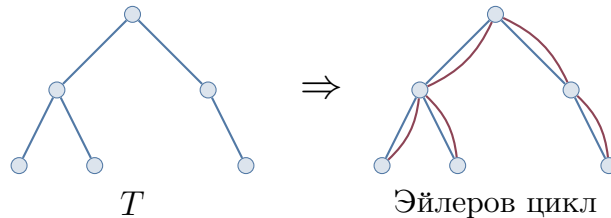


Рис. 25: Алгоритм двойного остова: исходное дерево T (тёмные рёбра) и его задвоенная копия (цветные рёбра). После удвоения все вершины имеют чётную степень, что гарантирует существование Эйлерова цикла.

Теорема 11.1: О 2-приближении алгоритма двойного остова

Пусть H^* — минимальный по весу гамильтонов цикл и T — минимальное остовное дерево. Тогда построенный алгоритмом гамильтонов цикл $\tilde{H}$ удовлетворяет:

$$c(\tilde{H}) \leq 2c(H^*).$$

Доказательство.

1. *Оценка $c(T) \leq c(H^*)$.* Для каждого ребра $e \in E(H^*)$ граф $H^* \setminus \{e\}$ является остовным деревом, поэтому $c(T) \leq c(H^* \setminus \{e\}) \leq c(H^*)$.
2. *Вес задвоенного дерева.* После дублирования всех рёбер T суммарный вес путей по Эйлерову циклу:

$$\sum_i c(u_i, u_{i+1}) = 2c(T).$$

3. *Срезание вершин не увеличивает вес.* При построении $\tilde{H}$, если вершина u_i уже встречалась, «перепрыгиваем» через неё: вместо рёбер (u_{i-1}, u_i) и (u_i, u_{i+1}) используем ребро (u_{i-1}, u_{i+1}) . По неравенству треугольника:

$$c(u_{i-1}, u_{i+1}) \leq c(u_{i-1}, u_i) + c(u_i, u_{i+1}).$$

Каждое такое срезание не увеличивает вес. Поэтому $c(\tilde{H}) \leq 2c(T) \leq 2c(H^*)$. ■

11.3. Алгоритм Кристофидеса–Сердюкова ($\frac{3}{2}$ -приближение)

Алгоритм Кристофидеса–Сердюкова улучшает коэффициент приближения с 2 до $\frac{3}{2}$, заменяя дублирование всех рёбер дерева минимальным паросочетанием на вершинах нечётной степени.

Алгоритм:

1. Найти минимальное остовное дерево T в G .
2. Выделить множество вершин нечётной степени в T :

$$V_0 = \{v \in V(T) \mid \deg_T(v) \equiv 1 \pmod{2}\}.$$

По лемме о рукопожатиях $|V_0|$ чётно.

3. Построить минимальное по весу совершенное паросочетание M на вершинах V_0 (алгоритм Эдмондса работает за полиномиальное время).
4. Добавить рёбра паросочетания M к дереву T ; получим мультиграф $T' = T \cup M$, в котором все вершины имеют чётную степень.
5. Найти Эйлеров цикл в T' : $u_1 \rightarrow u_2 \rightarrow \dots \rightarrow u_k \rightarrow u_1$.
6. Построить гамильтонов цикл $\tilde{H}$, срезая повторные вершины (как в алгоритме двойного остова).

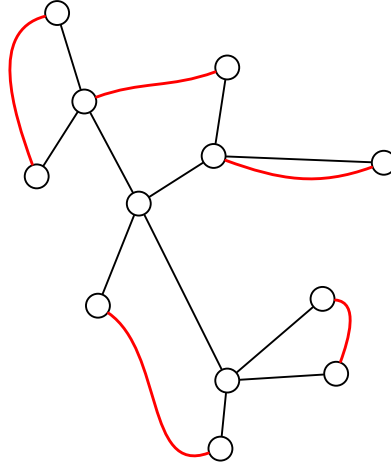


Рис. 26: Пример работы алгоритма Кристофидеса–Сердюкова: чёрные рёбра — минимальное остовное дерево T , красные рёбра — минимальное совершенное паросочетание M на вершинах нечётной степени V_0 . Мультиграф $T' = T \cup M$ имеет все вершины чётной степени.

Теорема 11.2: Алгоритм Кристофидеса–Сердюкова даёт $\frac{3}{2}$ -приближение

Для метрической задачи коммивояжёра алгоритм Кристофидеса–Сердюкова строит гамильтонов цикл $\tilde{H}$ такой, что:

$$c(\tilde{H}) \leq \frac{3}{2} c(H^*).$$

Доказательство. Вес $c(\tilde{H})$ не превышает суммарного веса Эйлера цикла в T' (срезание не увеличивает вес по неравенству треугольника):

$$c(\tilde{H}) \leq c(T) + c(M),$$

где $c(M) = \sum_{e \in M} c(e)$ — вес минимального паросочетания на V_0 .

Шаг 1: $c(T) \leq c(H^*)$. Граф $H^* \setminus \{e\}$ для любого $e \in E(H^*)$ является остовным деревом, поэтому $c(T) \leq c(H^*)$.

Шаг 2: $c(M) \leq \frac{c(H^*)}{2}$. Пронумеруем вершины $V_0 = \{v_1, v_2, \dots, v_{2r}\}$ в порядке их встречаемости при обходе оптимального цикла H^* . Рассмотрим два паросочетания на V_0 :

$$M_1 = \{v_1v_2, v_3v_4, \dots, v_{2r-1}v_{2r}\}, \quad M_2 = \{v_2v_3, v_4v_5, \dots, v_{2r}v_1\}.$$

По обобщённому неравенству треугольника (применённому к путям вдоль H^* между соседними вершинами V_0):

$$c(M_1) + c(M_2) \leq c(H^*).$$

Следовательно, $\min(c(M_1), c(M_2)) \leq \frac{c(H^*)}{2}$. Минимальное паросочетание не хуже наилучшего из M_1, M_2 :

$$c(M) \leq \min(c(M_1), c(M_2)) \leq \frac{c(H^*)}{2}.$$

Итог:

$$c(\tilde{H}) \leq c(T) + c(M) \leq c(H^*) + \frac{c(H^*)}{2} = \frac{3}{2} c(H^*).$$

■

Ключевую идею оценки минимального паросочетания на V_0 иллюстрирует следующая схема: цикл H^* разбивается на два паросочетания M_1 и M_2 , и их суммарный вес не превосходит $c(H^*)$.

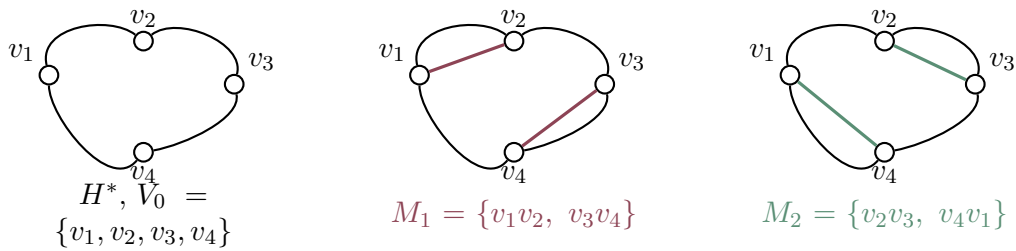


Рис. 27: Разбиение оптимального цикла H^* на два паросочетания M_1 и M_2 . Их суммарный вес $c(M_1) + c(M_2) \leq c(H^*)$, откуда минимальное паросочетание удовлетворяет $c(M) \leq \frac{c(H^*)}{2}$.

Замечание

Алгоритм Кристофидеса–Сердюкова долгое время оставался лучшим известным приближением для метрического TSP. Лишь в 2020–2023 годах были получены алгоритмы с коэффициентом $\frac{3}{2} - \varepsilon$ для частных случаев (например, для графового TSP).

12. Вероятностные алгоритмы. Задача проверки умножения матриц. Алгоритм Фрейвальдса. Задача проверки равенства многочленов. Лемма Шварца–Зиппеля

Вероятностный алгоритм — это алгоритм, использующий генератор случайных чисел random.

Определение : Функция random

Функция $\text{random}(i, j): \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$ возвращает равновероятно выбранное значение $x \in \{i, i+1, \dots, j\}$.

Замечание

На практике в языках программирования используют *псевдогенераторы*. В данном курсе будем считать, что генератор истинно случаен. Современные криптографические инструкции (аппаратные генераторы шума) приближаются к истинной случайности.

Пример : Поиск позиции нуля в массиве

Вход: массив $a[1, \dots, n]$, в котором не менее половины элементов равны 0. **Задача:** найти позицию 0.

```
do {
  p = random(1, n);
} while (a[p] != 0);
return p;
```

На каждой итерации вероятность успеха не менее $\frac{1}{2}$, поэтому ожидаемое число итераций не превышает 2.

Пусть $t(a)$ — время работы алгоритма на конкретном входе a . Величина $t(a)$ случайна (зависит от выборов random).

Определение : Ожидаемое время работы

Ожидаемое время работы алгоритма на входах длины n :

$$T(n) = \max_{a \in I_n} \mathbf{E}[t(a)],$$

где I_n — множество всех входов длины n .

Для алгоритма поиска нуля $T(n) \leq O(\sum_{z=1}^{\infty} z \cdot \frac{1}{2^z}) = O(1) \ll O(n)$.

Классы вероятностных алгоритмов

Определение : Алгоритм Лас-Вегас

Алгоритм Лас-Вегас — вероятностный алгоритм, который **всегда** возвращает корректный ответ, но время работы $t(n)$ является случайной величиной.

Определение : Алгоритм Монте-Карло

Алгоритм Монте-Карло — вероятностный алгоритм, который может вернуть **некорректное** решение, однако вероятность ошибки мала и поддаётся оценке.

Замечание

Алгоритмы Лас-Вегас встречаются чаще. Алгоритмы Монте-Карло важны, когда детерминированное решение слишком дорого.

12.1. Алгоритм Фрейвальдса: проверка произведения матриц

Пример : Проверка $AB = C$

Вход: $A, B, C \in \mathbb{R}^{n \times n}$. Верно ли, что $AB = C$?

Детерминированные алгоритмы работают за $O(n^\omega)$, $\omega > 2,37$.

Алгоритм Фрейвальдса (Монте-Карло, $O(n^2)$):

1. Выбрать $\delta \in \{0, 1\}^n$ равномерно и по координатам независимо.
2. Вычислить $AB\delta$ и $C\delta$.
3. Если $AB\delta = C\delta$, вернуть «да»; иначе — «нет».

Теорема 12.1: Оценка ошибки алгоритма Фрейвальдса

Если $AB = C$, алгоритм всегда отвечает верно. Если $AB \neq C$, то $\Pr(\text{ошибка}) \leq \frac{1}{2}$.

Доказательство. Случай $AB = C$ очевиден. Пусть $AB \neq C$. Пусть матрицы отличаются в строке k ; обозначим $x := (AB)[k, \cdot]$, $y := C[k, \cdot]$, тогда $x \neq y$. Ошибка — событие $\{x\delta = y\delta\}$, т.е. $(x - y)\delta = 0$.

Пусть ℓ — индекс, в котором $x_\ell \neq y_\ell$. Зафиксируем все δ_i при $i \neq \ell$. Условие $(x - y)\delta = 0$ записывается как

$$(x_\ell - y_\ell) \delta_\ell = -\sum_{i \neq \ell} (x_i - y_i) \delta_i.$$

1. Если правая часть равна 0: равенство выполняется только при $\delta_\ell = 0$, поэтому $\Pr(x\delta = y\delta \mid \delta_{-\ell}) = \frac{1}{2}$.
2. Если правая часть $\neq 0$: уравнение имеет не более одного решения $\delta_\ell \in \{0, 1\}$, поэтому $\Pr(x\delta = y\delta \mid \delta_{-\ell}) \leq \frac{1}{2}$.

В обоих случаях $\Pr(\text{ошибка}) \leq \frac{1}{2}$. ■

Уменьшение вероятности ошибки

Чтобы добиться $\Pr(\text{ошибка}) < \varepsilon$, достаточно запустить алгоритм k раз независимо, где $k \in \mathbb{N}$ удовлетворяет $\frac{1}{2^k} < \varepsilon$. Если хотя бы в одном запуске ответ «нет» — вернуть «нет»; если во всех «да» — вернуть «да». Тогда $\Pr(\text{ошибка}) \leq \frac{1}{2^k} < \varepsilon$.

Замечание

Исторически метод Монте-Карло применялся для вычисления интегралов и решения дифференциальных уравнений: случайные точки (p_x, p_y) накладываются равномерно, и доля точек с $p_y \leq f(p_x)$ приближает $\int f(x) dx$.

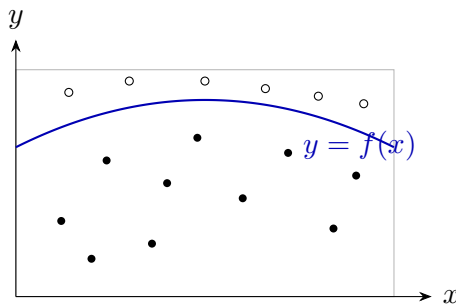


Рис. 28: Оценка $\int f(x) dx$ методом Монте-Карло: точки бросаются равномерно в прямоугольник, закрашенные попадают под кривую ($p_y \leq f(p_x)$). Доля закрашенных точек, умноженная на площадь прямоугольника, приближает площадь под графиком — то есть интеграл.

12.2. Проверка тождественного равенства многочленов

Определение : Задача PIT

Даны многочлены $p(x_1, \dots, x_n)$ и $q(x_1, \dots, x_n)$ над полем $\mathbb{F}$. Проверить: $p \equiv q$, то есть $\forall x_1, \dots, x_n: p = q$. Эквивалентно: $f \equiv 0$, где $f := p - q$.

Если f задан вычислительной схемой размера $O(k)$, то $\deg f \leq 2^k$. При $|\mathbb{F}| = +\infty$ детерминированный алгоритм мог бы раскрыть f в сумму мономов $f(x_1, \dots, x_n) = \sum_i a_i x_1^{d_1} \cdots x_n^{d_n}$

и проверить все $a_i = 0$, однако это экспоненциально от размера входа; в детерминированном случае никто лучше не может. Для Монте-Карло алгоритма: выбрать случайное $a \in \mathbb{F}^n$ и проверить $f(a) \neq 0$.

Определение : Степень многочлена от нескольких переменных

$$\deg f = \max \left\{ \sum_j d_j \mid x_1^{d_1} \cdots x_n^{d_n} \text{ — моном } f \right\}.$$

12.3. Лемма Шварца–Зиппеля

Лемма 12.1: Шварца–Зиппеля

Пусть $f(x_1, \dots, x_n) \not\equiv 0$ над полем $\mathbb{F}$, $S \subseteq \mathbb{F}$, $0 < |S| < +\infty$. Если $x_1, \dots, x_n$ выбраны равномерно и независимо из S , то

$$\Pr_{x_1, \dots, x_n \in S} (f(x_1, \dots, x_n) = 0) \leq \frac{\deg f}{|S|}.$$

Доказательство. Индукция по n .

База $n = 1$. Многочлен $f(x_1)$ имеет не более $\deg f$ корней, поэтому $\Pr_{x_1 \in S} (f(x_1) = 0) \leq \frac{\deg f}{|S|}$.

Шаг $n > 1$. Запишем

$$f(x_1, \dots, x_n) = \sum_{k=0}^d p_k(x_1, \dots, x_{n-1}) x_n^k, \quad d = \deg f.$$

Так как $f \not\equiv 0$, существует наибольшее k' такое, что $p_{k'} \not\equiv 0$. По индукционному предположению, применённому к многочлену $p_{k'}$, имеем

$$\Pr(p_{k'}(x_1, \dots, x_{n-1}) \neq 0) \geq 1 - \frac{\deg p_{k'}}{|S|}.$$

Зафиксируем $x_1, \dots, x_{n-1}$ так, что $p_{k'}(x_1, \dots, x_{n-1}) \neq 0$. Тогда $g(x_n) := f(x_1, \dots, x_{n-1}, x_n) = \sum_{k=0}^d p_k(x_1, \dots, x_{n-1}) x_n^k$ является ненулевым многочленом от x_n степени k' . Кроме того, $k' + \deg p_{k'} \leq d$, поэтому $k' \leq d - \deg p_{k'}$. По базе индукции

$$\Pr(g(x_n) = 0) \leq \frac{k'}{|S|} \leq \frac{d - \deg p_{k'}}{|S|},$$

то есть $\Pr(g(x_n) \neq 0) \geq 1 - \frac{d - \deg p_{k'}}{|S|}$. Следовательно,

$$\begin{aligned} \Pr(f(x_1, \dots, x_n) \neq 0) &\geq \Pr(p_{k'}(x_1, \dots, x_{n-1}) \neq 0) \Pr(g(x_n) \neq 0 \mid p_{k'}(x_1, \dots, x_{n-1}) \neq 0) \\ &\geq \left(1 - \frac{\deg p_{k'}}{|S|}\right) \left(1 - \frac{d - \deg p_{k'}}{|S|}\right) \\ &= 1 - \frac{d}{|S|} + \frac{\deg p_{k'}(d - \deg p_{k'})}{|S|^2} \geq 1 - \frac{d}{|S|}. \end{aligned}$$

Значит, $\Pr(f(x_1, \dots, x_n) = 0) \leq \frac{d}{|S|}$. ■

Замечание

С помощью леммы Шварца–Зиппеля задача РИТ решается Монте-Карло алгоритмом: выбрать точку из S^n и проверить $f = 0$. Удобно считать поле бесконечным — тогда вероятность ошибки стремится к нулю с ростом $|S|$.

13. Хеш-таблицы, ключи. Амортизированное и ожидаемое время работы. Время работы хеш-таблицы с открытой адресацией при идеализированном случайном равновероятном хешировании

Определение : Словарь (хеш-таблица)

Словарь — структура данных для хранения пар «ключ–значение» ($x \in U, v$), поддерживающая операции:

- $\text{insert}(x, v)$ — вставить значение v по ключу x ;
- $\text{delete}(x)$ — удалить запись с ключом x ;
- $v = \text{query}(x)$ — найти значение по ключу x (или null, если x нет).

Здесь U — *универсум* всех возможных ключей (большое множество, например $\mathbb{Z}$).

Замечание

Реализация на основе АВЛ-дерева поддерживает все операции за $O(\log n)$, где n — текущее количество элементов. Хеш-таблица реализует их в среднем за $O(1)$.

Хеширование произвольных ключей

Если U — не целые числа, используется один из двух подходов.

1. **Уникальные объекты** ($x = y \Leftrightarrow$ это один и тот же объект в памяти): каждому объекту сопоставляется его адрес в памяти $\in \mathbb{N}$ (в C++ это буквально указатель).
2. **Неуникальные объекты** (строки, кортежи, векторы): сопоставляем им *хеш* (например, хеш Карпа–Рабина). Коллизия возникает с малой вероятностью.

13.1. Хеш-таблица с открытой адресацией

Определение

Хранится таблица $T[0..m-1]$, где $n < m$, n — текущее число ключей. *Хеш-функция* $h: U \rightarrow \{0, \dots, m-1\}$ определяет, в какую ячейку помещается ключ $x \in U$: он помещается в $T[h(x)]$.

Определение

Коллизия — ситуация, когда $h(x) = h(y)$ при $x \neq y$.

Утверждение 13.1

Если $|U| > m$, то коллизии неизбежны при любой хеш-функции h .

Разрешение коллизий линейным зондированием

При *линейном зондировании* ячейки таблицы просматриваются последовательно начиная с $h(x)$, по кругу. Используются два специальных значения: $\emptyset$ (пустая ячейка) и $\dagger$ (tombstone — удалённая ячейка).

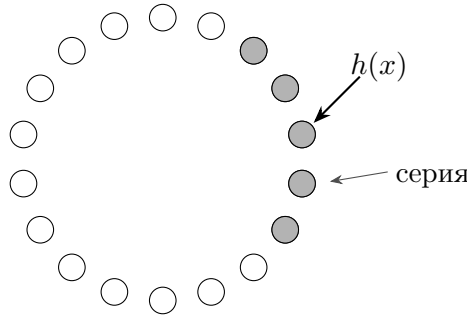


Рис. 29: Хеш-таблица $T[0..m-1]$ как кольцевой буфер. Серые ячейки образуют *серия* — максимальную цепочку подряд занятых ячеек. Стрелка отмечает позицию $h(x)$, с которой начинается зондирование; время операции пропорционально длине хвоста серии от $h(x)$ до её конца.

Вставка

```

1: function INSERT( $x, v$ )
2:    $t = h(x); \quad t_{\dagger} = -1$ 
3:   while  $T[t] \neq \emptyset$  and ( $T[t] = \dagger$  or  $T[t].key \neq x$ ) do
4:     if  $T[t] = \dagger$  and  $t_{\dagger} = -1$  then
5:        $t_{\dagger} = t$ 
6:      $t = (t + 1) \bmod m$ 
7:   if  $T[t].key \neq x$  then
8:     if  $t_{\dagger} \neq -1$  then
9:        $t = t_{\dagger}$ 
10:     $T[t].key = x; \quad n = n + 1$ 
11:     $T[t].value = v$ 
12:    if  $2n > m$  then
13:      Создать новую пустую таблицу  $T'$  размера  $2m$ 
14:      Перехешировать все элементы из  $T$  в  $T'$  с учётом нового размера
15:       $T = T'; \quad m = 2m$ 
16: end function

```

Удаление

```

1: function DELETE( $x$ )
2:    $t = h(x)$ 
3:   while  $T[t] \neq \emptyset$  and  $T[t].key \neq x$  do
4:      $t = (t + 1) \bmod m$ 
5:   if  $T[t] \neq \emptyset$  then
6:      $T[t] = \dagger$ 

```

▷ n не меняется

13.2. Амортизированное и ожидаемое время работы

Определение : Ожидаемое время работы

Пусть $t(a)$ — (случайное) время работы на входе a . Ожидаемое время работы алгоритма на входах длины n :

$$T(n) = \max_{a \in I_n} \mathbf{E}[t(a)],$$

где I_n — множество всех входов длины n .

Определение : Амортизированное время работы

Говорят, что *амортизированное время работы* операций структуры данных равно $O(f(n))$, если любая последовательность из n операций, применённых на изначально пустой структуре данных, выполняется за время $O(n f(n))$.

Утверждение 13.2

Ожидаемое амортизированное время работы операций в хеш-таблице с открытой адресацией равно $O(1)$.

Доказательство (набросок). Из-за линейности математического ожидания достаточно показать, что суммарное ожидаемое количество шагов при выполнении n операций равно $O(n)$. Перестройка таблицы (при $2n > m$) происходит лишь после того, как число элементов удваивается; амортизированная стоимость перестройки не превышает $O(1)$ на операцию вставки (по аналогии с динамическим массивом). Остаётся оценить ожидаемое число шагов одной операции без перестройки — этим занимается следующая теорема. ■

13.3. Идеализированное предположение и анализ

Определение : Идеализированное предположение

Хеш-функция h генерируется случайно и равновероятно из множества *всех* функций $U \rightarrow \{0, \dots, m-1\}$, т.е. для каждого $x \in U$ значение $h(x)$ выбирается независимо и равновероятно из $\{0, \dots, m-1\}$.

Теорема 13.1

При идеализированном предположении ожидаемое время работы каждой операции равно $O(1)$ (если не происходит перестройка таблицы).

Доказательство. Будем считать, что все ключи различны и $m = 2n$. Время работы операции с ключом x пропорционально длине «хвоста» серии, начиная с позиции $t = h(x)$. Обозначим:

- *серия* — максимальная цепочка подряд занятых ячеек $t', t' + 1, \dots, t' + k - 1$, где ячейки $t' - 1$ и $t' + k$ пусты (нумерация по модулю m);

- q_k — вероятность того, что в таблице существует серия длины k , начинающаяся в фиксированной позиции.

Ожидаемое время. Пусть p_ℓ — вероятность встретить в позиции $t = h(x)$ «хвост» длины ℓ некоторой серии длины $k \geq \ell$. Если серия имеет длину $k \geq \ell$, то для хвоста длины ℓ в позиции t она обязана начинаться в позиции $t - (k - \ell)$, поэтому

$$p_\ell = \sum_{k=\ell}^n q_k.$$

Тогда ожидаемое время равно $O(\sum_{\ell=0}^n p_\ell \cdot \ell)$. Преобразуем:

$$\sum_{\ell=0}^n p_\ell \cdot \ell = \sum_{\ell=0}^n \sum_{k=\ell}^n q_k \cdot \ell = \sum_{k=0}^n q_k \cdot (0 + 1 + \dots + k) = \sum_{k=0}^n q_k \cdot \Theta(k^2).$$

Оценка q_k . Если в таблице есть серия в позициях $t', t' + 1, \dots, t' + k - 1$, то ровно k ключей отображены в эти k ячеек, а остальные $n - k$ — вне них. При $m = 2n$:

$$q_k \leq \binom{n}{k} \left(\frac{k}{2n}\right)^k \left(1 - \frac{k}{2n}\right)^{n-k}.$$

Оценим каждый множитель сверху, используя неравенства

$$\binom{n}{k} \leq \frac{n^n}{k^k (n - k)^{n-k}}, \quad 1 + x \leq e^x.$$

Первое неравенство следует из бинома Ньютона: $n^n = (k + (n - k))^n = \sum_{i=0}^n \binom{n}{i} k^i (n - k)^{n-i}$. Все слагаемые этой суммы неотрицательны, поэтому, оставив лишь слагаемое с $i = k$, получаем $n^n \geq \binom{n}{k} k^k (n - k)^{n-k}$. Тогда

$$\begin{aligned} q_k &\leq \frac{n^n}{k^k (n - k)^{n-k}} \cdot \frac{k^k}{(2n)^k} \cdot \left(\frac{2n - k}{2n}\right)^{n-k} = \frac{1}{2^k} \cdot \left(\frac{2n - k}{2(n - k)}\right)^{n-k} \\ &= \frac{1}{2^k} \cdot \left(1 + \frac{k}{2(n - k)}\right)^{n-k} \leq \frac{1}{2^k} \cdot e^{k(n-k)/[2(n-k)]} = \frac{1}{2^k} \cdot e^{k/2} = \left(\frac{\sqrt{e}}{2}\right)^k. \end{aligned}$$

Поскольку $\frac{\sqrt{e}}{2} < 1$, получаем $q_k \leq e^{-\Omega(k)}$. Следовательно,

$$\sum_{k=0}^n q_k \cdot \Theta(k^2) \leq \Theta\left(\sum_{k=0}^{\infty} k^2 \cdot \left(\frac{\sqrt{e}}{2}\right)^k\right) = O(1).$$

Таким образом, ожидаемое время операции равно $O(1)$. ■

14. Универсальные семейства хеш-функций. 2-универсально семейства multiply-shift. Хеш-таблицы с разрешением коллизий цепочками

Определение : Универсальное и c -универсальное семейство хеш-функций

Семейство хеш-функций $\mathcal{H}$ над универсумом U , где каждая $h \in \mathcal{H}$ имеет вид $h: U \rightarrow \{0, \dots, m-1\}$, называется *универсальным*, если

$$\forall x, y \in U, x \neq y: \quad \Pr_{h \in \mathcal{H}}(h(x) = h(y)) \leq \frac{1}{m},$$

где вероятность берётся по случайному равновероятному выбору $h \in \mathcal{H}$.

Семейство $\mathcal{H}$ называется *c -универсальным* (для $c \geq 1$), если

$$\forall x, y \in U, x \neq y: \quad \Pr_{h \in \mathcal{H}}(h(x) = h(y)) \leq \frac{c}{m}.$$

Замечание

Чтобы выбрать h случайно и равновероятно из множества *всех* функций вида $U \rightarrow \{0, \dots, m-1\}$, потребовалось бы заполнить таблицу размера $|U|$ — это нереалистично. Идея состоит в том, чтобы выбирать h случайно и равновероятно из небольшого семейства $\mathcal{H} \subseteq \{0, \dots, m-1\}^U$ с малой вероятностью коллизий.

14.1. Семейство multiply-shift

Пусть ключи — w -битовые целые числа: $U = \{0, 1, \dots, 2^w - 1\}$, $m = 2^t$.

Определение : Семейство multiply-shift

$$\mathcal{H}_{\text{ms}} = \{h_a : a \in U, a \text{ нечётно}\}, \quad h_a(x) = (a \cdot x \bmod 2^w) \gg (w - t).$$

Чтобы выбрать $h_a \in \mathcal{H}_{\text{ms}}$ случайно и равновероятно:

1. положить $a = \text{random}(0, \dots, 2^w - 1)$;
2. положить $a := a \mid 1$ (побитовое ИЛИ с 1 делает a нечётным).

Замечание

Другое распространённое семейство — *multiply-add* (линейное хеширование по простому модулю): для произвольного $U = \{0, \dots, u-1\}$ фиксируем простое $p \geq u$ и полагаем

$$h_{a,b}(x) = ((ax + b) \bmod p) \bmod m, \quad \mathcal{H}_{\text{ma}} = \{h_{a,b} : a, b \in \{0, \dots, p-1\}, a \neq 0\}.$$

Теорема 14.1: 2-универсальность семейства multiply-shift

Семейство $\mathcal{H}_{\text{ms}} = \{h_a\}$ является 2-универсальным.

Доказательство. Пусть $x, y \in U = \{0, 1, \dots, 2^w - 1\}$, $x < y$, и $m = 2^t$. По определению h_a ,

$$h_a(x) = h_a(y) \iff ax \bmod 2^w \text{ и } ay \bmod 2^w \text{ имеют одинаковые старшие } t \text{ бит.}$$

Равенство старших t бит чисел $ax \bmod 2^w$ и $ay \bmod 2^w$ означает, что старшие t бит числа

$a(y - x) \bmod 2^w$ равны либо $\underbrace{00 \dots 0}_t$, либо $\underbrace{11 \dots 1}_t$ (второй случай возникает из-за возможного переноса при вычитании).

Запишем $y - x = z \cdot 2^b$, где z нечётно и $0 \leq b \leq w - 1$. Рассмотрим два случая.

Случай 1°: $b > w - t$. Так как a и z нечётны, число az нечётно, и

$$a(y - x) \bmod 2^w = (az \bmod 2^{w-b}) \cdot 2^b$$

является нечётным кратным 2^b . Поэтому в двоичной записи $a(y - x) \bmod 2^w$ бит номер b равен 1, а биты номеров $0, \dots, b - 1$ равны 0. Поскольку $b > w - t$, бит b принадлежит старшим t битам (номера $w - t, \dots, w - 1$), и среди тех же старших t бит найдётся бит номера $b - 1 \geq w - t$, равный 0. Значит, старшие t бит не могут быть равны ни $00 \dots 0$ (есть единица), ни $11 \dots 1$ (есть ноль) при любом нечётном a , то есть $\Pr_a(h_a(x) = h_a(y)) = 0$.

Случай 2°: $b \leq w - t$. Запишем нечётное a в виде $a = 2a_1 + 1$, где a_1 — случайное равновероятное $(w - 1)$ -битовое число (это эквивалентно случайному равновероятному выбору нечётного $a \in \{0, \dots, 2^w - 1\}$). Тогда

$$a(y - x) = (2a_1 + 1)z \cdot 2^b = (z + 2a_1z) \cdot 2^b.$$

Отображение $a_1 \mapsto a_1z \bmod 2^{w-1}$ является биекцией на множестве $(w - 1)$ -битовых чисел: если $a_1 < a_2$ — два различных $(w - 1)$ -битовых числа, то $a_2 - a_1 \not\equiv 0 \pmod{2^{w-1}}$, а так как z нечётно (и значит обратимо по модулю 2^{w-1}), то $z(a_2 - a_1) \not\equiv 0 \pmod{2^{w-1}}$, то есть $a_1z \not\equiv a_2z \pmod{2^{w-1}}$.

Следовательно, при случайном равновероятном выборе a_1 величина $a_1z \bmod 2^{w-1}$ также случайна и равновероятна на множестве $(w - 1)$ -битовых чисел, а значит младшие $(w - 1)$ бит числа $a(y - x) \bmod 2^w$ распределены случайно и равновероятно. Поэтому старшие t бит числа $a(y - x) \bmod 2^w$ равны $\underbrace{00 \dots 0}_t$ с вероятностью не более $\frac{1}{2^t}$ и равны $\underbrace{11 \dots 1}_t$ с вероятностью не более $\frac{1}{2^t}$. Суммируя,

$$\Pr_a(h_a(x) = h_a(y)) \leq \frac{2}{2^t} = \frac{2}{m}.$$

В обоих случаях $\Pr_a(h_a(x) = h_a(y)) \leq \frac{2}{m}$, то есть семейство multiply-shift является 2-универсальным. ■

14.2. Хеш-таблицы с разрешением коллизий цепочками

Определение

Пусть задана таблица $T[0, \dots, m - 1]$ и хеш-функция $h: U \rightarrow \{0, \dots, m - 1\}$. В хеш-таблице с разрешением коллизий цепочками каждая ячейка $T[k]$ хранит связный список всех ключей x , для которых $h(x) = k$. Операции insert, delete, query с ключом x обращаются к ячейке $T[h(x)]$ и просматривают (или обновляют) её список.

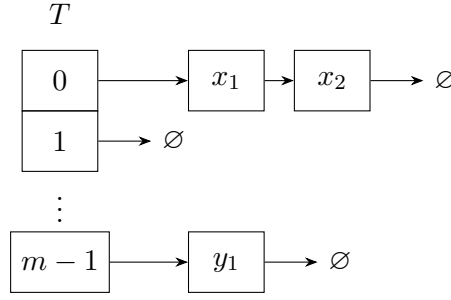


Рис. 30: Хеш-таблица с цепочками: ячейка $T[k]$ хранит связный список всех ключей x с $h(x) = k$. Время операции с ключом x определяется длиной списка в ячейке $T[h(x)]$.

Утверждение 14.1: Перевыделение таблицы

Зафиксируем константу $\alpha > 1$ (например, $\alpha = 2$). Если после очередной вставки число ключей n в таблице размера m превышает αm , выделяется новая таблица размера $m' = \beta m$ при некотором $\beta > 1$ (например, $\beta = 2$); все ключи переносятся в новую таблицу, после чего старая таблица удаляется. Эта стратегия гарантирует, что в любой момент $n \leq \alpha m$.

Теорема 14.2

Пусть h выбирается случайно и равновероятно из c -универсального семейства $\mathcal{H}$, где $c = O(1)$, а размер таблицы поддерживается стратегией перевыделения (так что $n \leq \alpha m$). Тогда ожидаемое амортизированное время каждой из операций равно $O(1)$.

Доказательство. Часть 1. Ожидаемое время операции без учёта перестройки. Достаточно показать, что для произвольного $x \in U$ ожидаемая длина списка $T[h(x)]$ равна $O(1)$.

Пусть $y_1, \dots, y_n$ — ключи, хранящиеся в таблице. Для каждого i положим

$$I_{y_i} = \begin{cases} 0, & h(y_i) \neq h(x), \\ 1, & h(y_i) = h(x). \end{cases}$$

Тогда длина списка $T[h(x)]$ равна $\sum_{i=1}^n I_{y_i}$, и по линейности математического ожидания и c -универсальности:

$$\mathbf{E} \left[\sum_{i=1}^n I_{y_i} \right] = \sum_{i=1}^n \Pr_{h \in \mathcal{H}} (h(x) = h(y_i)) \leq n \cdot \frac{c}{m} \leq (\alpha m) \cdot \frac{c}{m} = \alpha c = O(1).$$

Часть 2. Амортизированная стоимость перестройки. Перестройка происходит, когда $n > \alpha m$: создаётся таблица размера βm ($\beta > 1$ фиксировано), все $n \leq \alpha m + 1$ ключей перехэшируются. Стоимость одной перестройки — $O(m)$.

Покажем, что между двумя последовательными перестройками выполняется не менее $\Omega(m)$ операций вставки. Пусть сразу после перестройки к таблице размера βm в ней хранится n_0 ключей. Так как перестройка была инициирована условием $n > \alpha m$, до неё размер был m , значит $n_0 \leq \alpha m + 1$. Следующая перестройка произойдёт лишь когда число ключей превысит $\alpha \cdot \beta m$, то есть потребуются не менее

$$\alpha \beta m - n_0 \geq \alpha \beta m - \alpha m - 1 = \alpha(\beta - 1)m - 1$$

операций вставки. Стоимость перестройки $O(\beta m)$, распределяя её по этим вставкам:

$$\frac{O(\beta m)}{\alpha(\beta - 1)m} = O\left(\frac{\beta}{\alpha(\beta - 1)}\right) = O(1),$$

поскольку α и β — фиксированные константы, $\beta > 1$. ■

15. Совершенное хеширование. Двухуровневое совершенное хеширование

Задача. Дано фиксированное конечное множество $S \subseteq U$, $|S| = n$. Требуется построить структуру данных, обеспечивающую поиск любого элемента $x \in U$ за время $O(1)$ в худшем случае при использовании $O(n)$ памяти. Структура статическая: операции вставки и удаления не поддерживаются.

15.1. Однородное совершенное хеширование

Лемма 15.1

Пусть $\mathcal{H}$ — универсальное семейство хеш-функций $U \rightarrow \{0, \dots, m - 1\}$ и $|S| = n$, $m = n^2$. Тогда при случайном равновероятном выборе $h \in \mathcal{H}$

$$\Pr(\exists x \neq y \in S : h(x) = h(y)) \leq \frac{1}{2}.$$

Доказательство. Всего неупорядоченных пар $\{x, y\} \subseteq S$, $x \neq y$, ровно $\binom{n}{2}$, и по определению универсального семейства $\Pr(h(x) = h(y)) \leq \frac{1}{m} = \frac{1}{n^2}$. По неравенству объединения

$$\Pr(\exists x \neq y \in S : h(x) = h(y)) \leq \binom{n}{2} \cdot \frac{1}{m} = \frac{n(n-1)}{2} \cdot \frac{1}{n^2} \leq \frac{1}{2}.$$

■

Замечание

Это означает, что при многократном независимом выборе $h \in \mathcal{H}$ за ожидаемое число попыток ≤ 2 найдётся функция h , не имеющая коллизий на S . Однако размер таблицы $m = n^2$ может быть неприемлемо большим.

15.2. Двухуровневая схема

Для достижения линейного объёма памяти используется двухуровневая конструкция.

Первый уровень. Выбираем $h: U \rightarrow \{0, \dots, n - 1\}$ случайно и равновероятно из универсального семейства (то есть на первом уровне $m = n$). Для каждого $k = 0, \dots, n - 1$ полагаем

$$S_k = \{x \in S : h(x) = k\}, \quad n_k = |S_k|, \quad \sum_{k=0}^{n-1} n_k = n.$$

Второй уровень. В ячейке $T[k]$ первого уровня храним хеш-таблицу второго уровня размера n_k^2 с хеш-функцией h_k , выбранной случайно и равновероятно из универсального

семейства. По предыдущей лемме при каждой повторной попытке выбора h_k с вероятностью $\geq \frac{1}{2}$ в таблице k -го уровня не будет коллизий.

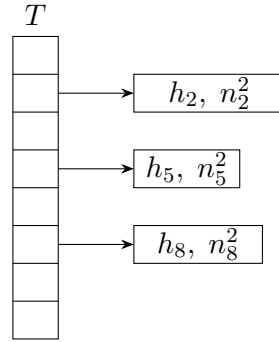


Рис. 31: Двухуровневая схема. Занятая ячейка $T[k]$ первого уровня (размер n) ссылается на таблицу второго уровня размера n_k^2 с собственной хеш-функцией h_k , не имеющей коллизий на S_k . Суммарный размер таблиц второго уровня составляет $O(n)$, поэтому вся структура занимает $O(n)$ памяти.

Поиск. По запросу x : вычислить $k = h(x)$, затем найти x в таблице S_k по адресу $h_k(x)$ — два вычисления хеш-функции, $O(1)$.

Предложение 15.1

В описанной схеме

$$\mathbf{E} \left[\sum_{k=0}^{n-1} n_k^2 \right] \leq 2n.$$

Доказательство. Из тождества $n_k^2 = n_k + 2 \binom{n_k}{2}$ и равенства $\sum_{k=0}^{n-1} n_k = n$ получаем $\sum_{k=0}^{n-1} n_k^2 = n + 2 \sum_{k=0}^{n-1} \binom{n_k}{2}$. Величина $\sum_k \binom{n_k}{2}$ равна числу неупорядоченных пар $\{x, y\} \subseteq S$, $x \neq y$, с $h(x) = h(y)$ (числу коллизий первого уровня). По линейности математического ожидания и свойству универсального семейства ($m = n$):

$$\mathbf{E} \left[\sum_{k=0}^{n-1} \binom{n_k}{2} \right] = \binom{n}{2} \cdot \Pr(h(x) = h(y)) \leq \binom{n}{2} \cdot \frac{1}{n} = \frac{n-1}{2}.$$

Следовательно, $\mathbf{E} \left[\sum_{k=0}^{n-1} n_k^2 \right] \leq n + 2 \cdot \frac{n-1}{2} = 2n - 1 \leq 2n$. ■

Следствие

По неравенству Маркова, применённому к неотрицательной величине $\sum_k n_k^2$, имеем $\Pr(\sum_k n_k^2 \geq 2 \mathbf{E}[\sum_k n_k^2]) \leq \frac{1}{2}$. Так как $\mathbf{E}[\sum_k n_k^2] \leq 2n$, при случайном выборе h с вероятностью $\geq \frac{1}{2}$ суммарный размер таблиц второго уровня не превышает $4n = O(n)$. Если это не выполнено, выбираем новую функцию h (в среднем $O(1)$ попыток).

Следствие

Существует хеш-таблица для фиксированного множества S , $|S| = n$, занимающая $O(n)$ памяти и обеспечивающая доступ к любому элементу за $O(1)$ в худшем случае.
Построение:

1. Выбирать $h \in \mathcal{H}$ (в $U \rightarrow \{0, \dots, n-1\}$) случайно и равновероятно до тех пор, пока $\sum_k n_k^2 \leq 4n$ (ожидаемое число попыток ≤ 2).
2. Для каждого k выделить таблицу второго уровня размера n_k^2 и выбирать $h_k \in \mathcal{H}$ (в $U \rightarrow \{0, \dots, n_k-1\}$) случайно и равновероятно до тех пор, пока коллизий в S_k нет (ожидаемое число попыток ≤ 2).
3. Поиск x : вычислить $k = h(x)$, затем проверить ячейку $h_k(x)$ таблицы S_k — одно обращение к памяти, $O(1)$.

16. Задача проверки простоты числа. Тест Ферма. Числа Кармайкла. Алгоритм Миллера–Рабина

Пусть число n задано своей двоичной записью, занимающей $\log_2 n$ бит (например, 4000 бит). Наивный алгоритм проверки простоты — перебор делителей от 2 до $\sqrt{n}$ — работает за $O(\sqrt{n})$ арифметических операций. Поскольку $\sqrt{n} = 2^{(\log_2 n)/2}$, для 4000-битного числа это порядка 2^{2000} операций — на практике неосуществимо. Цель: алгоритм со сложностью $O((\log n)^{O(1)})$, то есть полиномиальной от длины входа. Ключевая идея — малая теорема Ферма.

16.1. Тест Ферма

Теорема 16.1: Малая теорема Ферма

Для любого простого числа p и любого $a \geq 1$ выполнено $a^p \equiv a \pmod{p}$. В частности, если $\gcd(a, p) = 1$, то $a^{p-1} \equiv 1 \pmod{p}$.

Если $a^{n-1} \equiv 1 \pmod{n}$, то $a^{n-1} = 1 + kn$, откуда $\gcd(a, n) = 1$, то есть $a \in \mathbb{Z}_n^*$.

Определение

Множество $\mathbb{Z}_n^* = \{a \in \{1, \dots, n-1\} : \gcd(a, n) = 1\}$ образует мультипликативную группу по умножению по модулю n .

Определение

$F = \{a \in \mathbb{Z}_n^* : a^{n-1} \equiv 1 \pmod{n}\}$.

Утверждение 16.1

$F \leq \mathbb{Z}_n^*$, то есть F — подгруппа.

Доказательство. Для любых $a, b \in F$:

$$\begin{aligned}(ab)^{n-1} &= a^{n-1} \cdot b^{n-1} \equiv 1 \cdot 1 = 1 \pmod{n}, \\ (a^{-1})^{n-1} &= (a^{n-1})^{-1} \equiv 1^{-1} = 1 \pmod{n}.\end{aligned}$$

Единица $1 \in F$. Следовательно, F — подгруппа. ■

По теореме Лагранжа $|F|$ делит $|\mathbb{Z}_n^*|$.

Алгоритм (тест Ферма)

```

 $a \leftarrow \text{random}(2, n - 2)$ 
if  $a^{n-1} \not\equiv 1 \pmod{n}$  then
    вернуть «составное»
else
    вернуть «возможно простое»

```

Следствие

Если $\exists b \in \mathbb{Z}_n^* \setminus F$ (то есть $b^{n-1} \not\equiv 1 \pmod{n}$), то $|F| \leq \frac{|\mathbb{Z}_n^*|}{2}$. Тогда тест Ферма ошибается с вероятностью $\leq \frac{1}{2}$ на одной итерации и $\leq \frac{1}{2^k}$ при k запусках.

16.2. Числа Кармайкла

Определение

Составное n называется *числом Кармайкла*, если $F = \mathbb{Z}_n^*$, то есть $a^{n-1} \equiv 1 \pmod{n}$ для всех a с $\gcd(a, n) = 1$.

Замечание

На числах Кармайкла тест Ферма всегда ошибается (выдаёт «возможно простое»), независимо от числа итераций. Наименьшее число Кармайкла: $561 = 3 \cdot 11 \cdot 17$. Таких чисел бесконечно много.

Тест Миллера–Рабина устраняет этот недостаток.

16.3. Тест Миллера–Рабина

Пусть n нечётное, $n \geq 3$. Запишем $n - 1 = d \cdot 2^t$, где d нечётное, $t \geq 1$. Тогда для любого a :

$$\begin{aligned}
 a^{n-1} \equiv 1 \pmod{n} &\iff a^{d \cdot 2^t} - 1 \equiv 0 \pmod{n} \\
 &\iff (a^{d \cdot 2^{t-1}} + 1)(a^{d \cdot 2^{t-1}} - 1) \equiv 0 \pmod{n} \\
 &\iff (a^{d \cdot 2^{t-1}} + 1)(a^{d \cdot 2^{t-2}} + 1) \cdots (a^d + 1)(a^d - 1) \equiv 0 \pmod{n}.
 \end{aligned}$$

Утверждение 16.2

Если n простое и $\gcd(a, n) = 1$, то выполняется одно из условий:

$$a^d \equiv 1, \quad a^d \equiv -1, \quad a^{d \cdot 2} \equiv -1, \quad \dots, \quad a^{d \cdot 2^{t-1}} \equiv -1 \pmod{n}.$$

Доказательство. Так как $\mathbb{Z}/n\mathbb{Z}$ — поле, n делит ровно один из сомножителей разложения $a^{d \cdot 2^t} - 1 = (a^d - 1)(a^d + 1)(a^{d \cdot 2} + 1) \cdots (a^{d \cdot 2^{t-1}} + 1)$. ■

Алгоритм (тест Миллера–Рабина, k итераций)

```

for  $i = 1, \dots, k$  do
     $a \leftarrow \text{random}(2, n - 2)$ 

```


if $a^d \not\equiv 1 \pmod{n}$ и $a^d \not\equiv -1 \pmod{n}$ и $a^{d \cdot 2} \not\equiv -1 \pmod{n}$ и ... и $a^{d \cdot 2^{t-1}} \not\equiv -1 \pmod{n}$ then
 вернуть «составное»
 вернуть «возможно простое»

16.4. Анализ вероятности ошибки

Теорема 16.2

Для любого нечётного составного $n \geq 3$ вероятность того, что случайный $a \in \{2, \dots, n-2\}$ не будет обнаружен тестом Миллера–Рабина как свидетель составности n , не превышает $\frac{1}{2}$. При k независимых итерациях вероятность ошибки не превышает $\frac{1}{2^k}$.

Доказательство. Случай 1: n не является числом Кармайкла. Тогда $\exists b \in \mathbb{Z}_n^* \setminus F$, и по следствию выше $|F| \leq |\mathbb{Z}_n^*|/2$. Поскольку тест Миллера–Рабина строже теста Ферма (все a , проходящие тест МР, обязательно лежат в F), вероятность ошибки на одной итерации $\leq \frac{1}{2}$.

Случай 2: n — число Кармайкла ($F = \mathbb{Z}_n^*$).

Шаг 1. Выбор j и c . Выберем наибольшее $j \in \{0, 1, \dots, t-1\}$, для которого $\exists c \in \mathbb{Z}_n^*$ такое, что $c^{d \cdot 2^j} \equiv -1 \pmod{n}$. Такие j и c существуют: при $j = 0$, $c = -1$ имеем $(-1)^d \equiv -1 \pmod{n}$ (так как d нечётное).

Шаг 2. Определение K . Положим

$$K = \{a \in \mathbb{Z}_n^* : a^{d \cdot 2^j} \equiv \pm 1 \pmod{n}\}.$$

Если a проходит тест ($a^d \equiv 1$ или $a^{d \cdot 2^i} \equiv -1$ для некоторого i), то $a \in K$:

- $i < j$: $a^{d \cdot 2^j} = (a^{d \cdot 2^i})^{2^{j-i}} \equiv (-1)^{2^{j-i}} = 1$ (показатель 2^{j-i} чётный). ✓
- $i = j$: $a^{d \cdot 2^j} \equiv -1$. ✓
- $i > j$: невозможно по максимальной j .
- $a^d \equiv 1$: $a^{d \cdot 2^j} \equiv 1$. ✓

Если a проходит тест, то $a^{n-1} \equiv 1 \pmod{n}$, откуда $\gcd(a, n) = 1$ и $a \in \mathbb{Z}_n^*$.

Шаг 3. $K \leq \mathbb{Z}_n^$.* Для $a, b \in K$:

$$\begin{aligned} (ab)^{d \cdot 2^j} &= a^{d \cdot 2^j} \cdot b^{d \cdot 2^j} \equiv (\pm 1)(\pm 1) \equiv \pm 1 \pmod{n}, \\ (a^{-1})^{d \cdot 2^j} &= (a^{d \cdot 2^j})^{-1} \equiv (\pm 1)^{-1} \equiv \pm 1 \pmod{n}. \end{aligned}$$

Следовательно, K — подгруппа $\mathbb{Z}_n^*$.

Шаг 4. Нахождение $y \in \mathbb{Z}_n^ \setminus K$.* Известно (без доказательства), что число Кармайкла n не является степенью простого числа p^ℓ . Следовательно, $n = n_1 \cdot n_2$, где $\gcd(n_1, n_2) = 1$ и $n_1, n_2 > 1$. По Китайской теореме об остатках найдём $y \in \mathbb{Z}_n$ такое, что

$$y \equiv c \pmod{n_1}, \quad y \equiv 1 \pmod{n_2}.$$

Из $c^{d \cdot 2^j} \equiv -1 \pmod{n}$ следует $c^{d \cdot 2^j} \equiv -1 \pmod{n_1}$. Тогда:

1. $y^{d \cdot 2^j} \equiv c^{d \cdot 2^j} \equiv -1 \pmod{n_1}$, откуда $y^{d \cdot 2^j} \not\equiv 1 \pmod{n}$ (иначе $1 \equiv -1 \pmod{n_1}$, т. е. $n_1 \mid 2$; но n_1 нечётное — противоречие).
2. $y^{d \cdot 2^j} \equiv 1 \pmod{n_2}$, откуда $y^{d \cdot 2^j} \not\equiv -1 \pmod{n}$ (иначе $-1 \equiv 1 \pmod{n_2}$, т. е. $n_2 \mid 2$ — противоречие).

Следовательно, $y \notin K$.

Шаг 5. $y \in \mathbb{Z}_n^*$.

- $\gcd(c, n_1) = 1$ (так как $c \in \mathbb{Z}_n^*$) и $y \equiv c \pmod{n_1}$ влекут $\gcd(y, n_1) = 1$.
- $y \equiv 1 \pmod{n_2}$ влечёт $\gcd(y, n_2) = 1$.

Итого $\gcd(y, n) = 1$, то есть $y \in \mathbb{Z}_n^*$.

Вывод. Так как $y \in \mathbb{Z}_n^* \setminus K$, подгруппа K строгая. По теореме Лагранжа $|K| \leq \frac{|\mathbb{Z}_n^*|}{2}$, откуда вероятность ошибки теста на одной итерации $\leq \frac{1}{2}$, а при k независимых итерациях $\leq \frac{1}{2^k}$. ■

Замечание

Время работы: одно умножение N -разрядных чисел по модулю n стоит $O(N(\log N)^{O(1)})$. На одну итерацию нужно $O(N)$ таких умножений, итого $O(N^2(\log N)^{O(1)})$. Существует также детерминированный тест Агравала–Каяля–Саксены (2004), работающий за полиномиальное время порядка $O(N^6)$.

17. Онлайн-алгоритмы. Задача кэширования. Помечающий алгоритм

Определение

Онлайн-алгоритм — алгоритм, получающий входные данные по одному элементу и обязанный реагировать на каждый элемент (принимать необратимое решение) до получения следующего, не зная будущих элементов. Для сравнения, *офлайн-алгоритм* знает всю последовательность заранее.

17.1. Задача кэширования

Определение

Кэш состоит из k ячеек («страниц»). Пользователь последовательно запрашивает страницы $S_1, S_2, \dots \in \{1, \dots, N\}$, где $N > k$. Если запрошенная страница S_i находится в кэше — это *попадание* (hit, стоимость 0). Иначе — *промах* (miss, стоимость 1): нужно загрузить S_i в кэш, при необходимости вытеснив одну из k находящихся там страниц. Цель онлайн-алгоритма — минимизировать число промахов.

Определение

Оптимальный алгоритм OPT — алгоритм, решающий ту же задачу, но знающий всю последовательность запросов заранее и совершающий минимально возможное число промахов.

Определение

Онлайн-алгоритм называется *c-конкурентным* (*c-competitive*), если на любой последовательности запросов число его промахов не превышает $c \cdot \text{OPT}$ (плюс, возможно, аддитивная константа, зависящая от начального содержимого кэша), где OPT — число промахов оптимального алгоритма на той же последовательности.

Определение

Алгоритм LRU (Least Recently Used): при промахе вытесняет из кэша страницу, к которой дольше всего не было обращений.

Теорема 17.1

Алгоритм LRU является k -конкурентным. Константу k улучшить нельзя: существует последовательность, на которой LRU совершает в k раз больше промахов, чем OPT.

Доказательство нижней оценки. Рассмотрим кэш размера k и циклическую последовательность запросов к $k + 1$ различным страницам:

$$1, 2, \dots, k, k+1, 1, 2, \dots, k, k+1, \dots$$

После первых k запросов (заполнения кэша) LRU промахивается на каждом следующем шаге: запрашиваемая страница всегда оказывается той, что была вытеснена как наименее недавно использованная на предыдущем шаге. Алгоритм OPT, зная будущее, на той же последовательности совершает лишь один промах за каждые k запросов (вытесняя страницу, которая понадобится позже остальных). Отношение числа промахов равно k , то есть LRU не может быть c -конкурентным ни для какого $c < k$. ■

17.2. Помечающий алгоритм

Определение

Алгоритм MARKING поддерживает для каждой страницы в кэше метку 0 или 1. При запросе страницы s выполняется следующая процедура.

```
procedure MARK( $s$ )
  if  $s$  находится в кэше then
    пометить страницу  $s$  единицей и завершить обработку запроса
  else
    if все страницы в кэше помечены единицей then
      сбросить метки всех страниц в кэше в ноль           ▷ начало нового периода
      выбрать случайную страницу, помеченную нулём, и вытеснить её из кэша
      поместить  $s$  в кэш и пометить единицей
    end if
  end if
end procedure
```

Теорема 17.2

Пусть $H_k = \sum_{i=1}^k \frac{1}{i} \approx \ln k$. Для любой последовательности запросов $S_1, S_2, \dots$ математическое ожидание числа вытеснений алгоритма MARKING не более чем в $2H_k$ раз превосходит число вытеснений алгоритма OPT на той же последовательности:

$$\mathbf{E}[\#\text{вытеснений}_{\text{MARKING}}] \leq 2H_k \cdot \text{OPT}.$$

Анализ по периодам

Определение

Разобьём последовательность запросов $S_1, S_2, \dots$ слева направо на *периоды*: p -й период — это максимальный отрезок запросов $S_i, S_{i+1}, \dots, S_{i+\ell}$ такой, что

$$|\{S_i, \dots, S_{i+\ell}\}| = k, \quad |\{S_i, \dots, S_{i+\ell+1}\}| = k + 1,$$

то есть очередной запрос $S_{i+\ell+1}$ добавляет $(k + 1)$ -ю различную страницу.

Пример

Для $k = 2$ последовательность

$$a, b, c, c, b, e, e, s, f, f$$

разбивается на периоды $a, b \mid c, c, b \mid e, e, s \mid f, f$.

Определение

Пусть A_p — множество страниц, запрашиваемых в p -м периоде, $|A_p| = k$. Положим

$$m_p = |A_p \setminus A_{p-1}|$$

— число «новых» страниц p -го периода (не встречавшихся в $(p - 1)$ -м). Так как $|A_p| = |A_{p-1}| = k$, также $|A_{p-1} \setminus A_p| = m_p$ и $|A_p \cap A_{p-1}| = k - m_p$.

Замечание

В конце каждого периода (и только в эти моменты) все страницы кэша помечены единицей, после чего происходит сброс меток в ноль. Поэтому в начале p -го периода кэш содержит в точности k страниц множества A_{p-1} — и только их.

Нижняя оценка для OPT

Лемма 17.1

$$\text{OPT} \geq \frac{1}{2} \sum_p m_p.$$

Доказательство. Объединение $A_{p-1} \cup A_p$ содержит $|A_{p-1} \cup A_p| = |A_{p-1}| + |A_p \setminus A_{p-1}| =$

$k + m_p$ различных страниц. Поскольку кэш вмещает только k страниц, на объединённом отрезке запросов $(p - 1)$ -го и p -го периодов любой алгоритм, в частности ОРТ, совершает не менее m_p промахов.

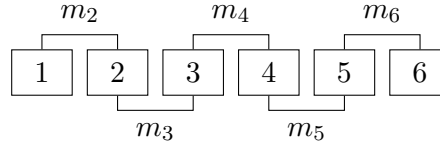


Рис. 32: Периоды (пронумерованы $1, \dots, 6$) и оценки промахов ОРТ. Каждая пара соседних периодов $(p - 1, p)$ вместе затрагивает $k + m_p$ различных страниц, поэтому даёт ОРТ не менее m_p промахов. Каждый период входит в две пары, $(p - 1, p)$ и $(p, p + 1)$, так что суммирование учитывает каждый промах ОРТ не более двух раз.

Каждый период p участвует в двух таких парах — $(p - 1, p)$ и $(p, p + 1)$, поэтому суммирование оценок по всем p учитывает каждый промах ОРТ не более двух раз, откуда $\sum_p m_p \leq 2 \text{ОРТ}$. ■

Верхняя оценка для MARKING

Лемма 17.2

Рассмотрим p -й период в худшем для MARKING порядке: первые m_p запросов — к страницам множества $A_p \setminus A_{p-1}$ («новые»), а оставшиеся $k - m_p$ запросов — к страницам $S^{(1)}, \dots, S^{(k-m_p)}$, образующим множество $A_p \cap A_{p-1}$ («старые»), в произвольном порядке. Тогда для каждого $j = 0, \dots, k - m_p - 1$ вероятность промаха на запросе $S^{(j+1)}$ не превышает $\frac{m_p}{k - j}$.

Доказательство. По предыдущему замечанию в начале p -го периода кэш состоит из всех k страниц множества A_{p-1} , и все они не помечены. Каждый из первых m_p запросов — промах: алгоритм вытесняет случайную непомеченную страницу. Так как ни одна из старых страниц ещё не помечена, эти m_p вытеснений выбирают (без повторений) равномерно случайное m_p -подмножество $E \subseteq A_{p-1}$.

Зафиксируем j и предположим, что $S^{(1)}, \dots, S^{(j)} \notin E$ (то есть к моменту своих запросов они не были вытеснены). По симметрии множество E равномерно распределено среди оставшихся $k - j$ страниц A_{p-1} , поэтому

$$\Pr(S^{(j+1)} \in E \mid S^{(1)}, \dots, S^{(j)} \notin E) = \frac{m_p}{k - j}.$$

Если $S^{(j+1)} \in E$, то страница уже вытеснена из кэша к моменту своего запроса, и этот запрос — промах. Следовательно, вероятность промаха на $S^{(j+1)}$ не превышает $\frac{m_p}{k - j}$. ■

Лемма 17.3

$\mathbb{E}[\text{\#промахов в } p\text{-м периоде}] \leq m_p H_k.$

Доказательство. В худшем порядке из предыдущей леммы первые m_p запросов периода — гарантированные промахи. Для остальных запросов положим

$$I_j = \begin{cases} 1, & (j+1)\text{-й «старый» запрос — промах,} \\ 0, & \text{иначе,} \end{cases} \quad j = 0, \dots, k - m_p - 1.$$

Тогда число промахов в p -м периоде равно $m_p + \sum_{j=0}^{k-m_p-1} I_j$, и по предыдущей лемме

$$\mathbf{E} \left[m_p + \sum_{j=0}^{k-m_p-1} I_j \right] \leq m_p + \sum_{j=0}^{k-m_p-1} \frac{m_p}{k-j} = m_p + m_p \sum_{h=m_p+1}^k \frac{1}{h} = m_p \left(1 + \sum_{h=m_p+1}^k \frac{1}{h} \right).$$

Так как $H_{m_p} = \sum_{i=1}^{m_p} \frac{1}{i} \geq 1$ при $m_p \geq 1$, выполняется $\sum_{h=m_p+1}^k \frac{1}{h} = H_k - H_{m_p} \leq H_k - 1$, откуда

$$\mathbf{E}[\text{\#промахов в } p\text{-м периоде}] \leq m_p(1 + (H_k - 1)) = m_p H_k.$$

При $m_p = 0$ оценка тривиальна. ■

Завершение доказательства

Доказательство теоремы. По линейности математического ожидания и предыдущей лемме,

$$\mathbf{E}[\text{\#вытеснений}_{\text{MARKING}}] = \sum_p \mathbf{E}[\text{\#промахов в } p\text{-м периоде}] \leq \sum_p m_p H_k = H_k \sum_p m_p.$$

По лемме о нижней оценке $\sum_p m_p \leq 2 \text{OPT}$, поэтому

$$\mathbf{E}[\text{\#вытеснений}_{\text{MARKING}}] \leq H_k \cdot 2 \text{OPT} = 2H_k \cdot \text{OPT}. \quad \blacksquare$$

Замечание

Известно также, что любой вероятностный алгоритм кэширования не может быть лучше H_k -конкурентного относительно OPT; таким образом, оценка $2H_k$ для MARKING оптимальна с точностью до множителя 2.

18. Задача нахождения минимального остовного дерева. Алгоритмы Ярника–Прима–Дейкстры и Краскала. Системы непересекающихся множеств (Union-Find)

Определение

Пусть $G = (V, E, c)$ — связный неориентированный граф с весовой функцией $c: E \rightarrow \mathbb{R}$. *Остовом* графа G называется подграф $T \subseteq G$, являющийся деревом (связным подграфом без циклов) и содержащий все вершины множества V . Остов T называется *минимальным* (МОД), если его суммарный вес $c(T) = \sum_{e \in E(T)} c(e)$ минимален среди всех остовов графа G .

Замечание

Размер входа равен $O(|V| + |E|)$. Граф удобно хранить списками смежности: для каждой вершины $v \in V$ хранится список пар $(u, c(v, u))$ для всех рёбер $vu \in E$ (структура вида «вектор векторов пар»). Это представление особенно удобно для алгоритмов прим-дейкстровского типа, где на каждом шаге требуется перебрать все рёбра, инцидентные текущей вершине.

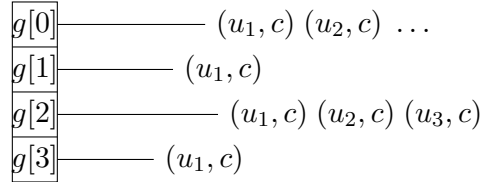


Рис. 33: Хранение графа списками смежности: для каждой вершины v хранится список пар $(u, c(v, u))$ по всем инцидентным рёбрам vu . Доступ ко всем рёбрам, инцидентным вершине, занимает время, пропорциональное её степени.

18.1. Лемма о разрезе

Определение

Разрезом графа $G = (V, E)$ называется пара непустых множеств $(W, V \setminus W)$, $W \neq \emptyset$, $V \setminus W \neq \emptyset$. Ребро $uv \in E$ называется *ребром разреза*, если один из его концов лежит в W , а другой — в $V \setminus W$.

Лемма 18.1: о минимальном ребре разреза

Пусть $(W, V \setminus W)$ — разрез графа G и

$$vw = \arg \min_{\substack{v \in V \setminus W, w \in W \\ vw \in E}} c(v, w)$$

— ребро разреза минимального веса. Тогда существует минимальный остов T графа G , для которого $vw \in E(T)$.

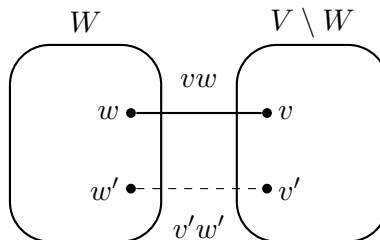


Рис. 34: Лемма о разрезе. vw — ребро разреза минимального веса (сплошное). При добавлении vw к остову T' возникает цикл, в котором найдётся ещё одно ребро разреза $v'w'$ (пунктирное). Замена $v'w'$ на vw не увеличивает вес остова и даёт МОД, содержащий vw .

Доказательство.

1. Пусть T' — произвольный минимальный остов графа G .

2. Если $vw \in E(T')$, то утверждение верно при $T = T'$.
3. Пусть $vw \notin E(T')$. Так как T' — дерево, граф $T' + vw$ содержит ровно один цикл, проходящий через ребро vw .
4. Этот цикл переходит из $V \setminus W$ в W , поэтому помимо vw в нём найдётся ещё одно ребро разреза $v'w'$ с $v' \in V \setminus W$, $w' \in W$.
5. Положим $T = T' + vw - v'w'$. Любой путь в T' , проходящий через ребро $v'w'$, можно заменить участком найденного цикла, содержащим vw , поэтому T остаётся связным. Кроме того, $|E(T)| = |E(T')| = |V| - 1$, значит T — дерево, то есть остов.
6. По выбору vw имеем $c(v, w) \leq c(v', w')$, следовательно $c(T) = c(T') + c(v, w) - c(v', w') \leq c(T')$.
7. Поскольку T' минимален, $c(T) \geq c(T')$; вместе с предыдущим неравенством $c(T) = c(T')$. Значит T — также минимальный остов, причём $vw \in E(T)$.

■

18.2. Алгоритм Прима (Ярника–Прима–Дейкстры)

Алгоритм Прима строит МОД, постепенно расширяя поддерево T : на каждом шаге к T присоединяется ребро минимального веса, соединяющее уже построенное дерево с оставшимися вершинами.

for $v \in V$ **do**

$d_v \leftarrow \infty$, $\pi_v \leftarrow \text{null}$

$d_{v_0} \leftarrow 0$

▷ v_0 — произвольная стартовая вершина

for $k = 1, \dots, |V| - 1$ **do**

$v \leftarrow \arg \min_{u \in V \setminus V(T)} d_u$

$T \leftarrow T \cup \{(v, \pi_v)\}$

for $vw \in E$ **do**

if $d_w > c(v, w)$ **then**

$d_w \leftarrow c(v, w)$

$\pi_w \leftarrow v$

Теорема 18.1: Корректность алгоритма Прима

На каждом шаге алгоритма ребро (v, π_v) , добавляемое в T , принадлежит некоторому минимальному остову графа G , содержащему все рёбра, уже добавленные в T ранее.

Доказательство. Индукция по числу добавленных рёбер.

База. В начале $T = \emptyset$ — подмножество любого минимального остова.

Шаг. Пусть T' — минимальный остов, содержащий все рёбра, добавленные в T к текущему моменту, и пусть $W = V(T)$ — множество вершин построенного поддерева. Так как d_u для $u \in V \setminus V(T)$ равно минимальному весу ребра, соединяющего u с T , выбор $v = \arg \min_{u \in V \setminus V(T)} d_u$ означает, что (v, π_v) — ребро минимального веса среди всех рёбер разреза $(W, V \setminus W)$.

1. Если $(v, \pi_v) \in E(T')$, утверждение для шага доказано.
2. Если $(v, \pi_v) \notin E(T')$, то, поскольку T' связен, в T' найдётся некоторое ребро $(x, x') \in$

$E(T')$ разреза $(W, V \setminus W)$. По лемме о разрезе дерево $T'' = T' - (x, x') + (v, \pi_v)$ также является минимальным остовом и содержит все рёбра T .

По индукции после $|V| - 1$ шагов получаем минимальный остов. ■

Замечание

Если $\arg \min$ вычисляется наивно (перебором), время работы составляет $O(|V|^2)$ — это оправдано для плотных (в частности, полных) графов. С очередью с приоритетами время работы составляет $O(|E| \log |V|)$.

18.3. Алгоритм Краскала

Алгоритм Краскала строит МОД, рассматривая рёбра в порядке возрастания веса и поддерживая лес деревьев: ребро добавляется, если оно соединяет вершины из разных деревьев.

Отсортировать рёбра так, что $c(e_1) \leq c(e_2) \leq \dots \leq c(e_{|E|})$

Инициализировать лес: каждая вершина — отдельное дерево

for $k = 1, \dots, |E|$ **do**

if ребро e_k соединяет вершины из разных деревьев леса T_1 и T_2 **then**

 добавить e_k в остов

 заменить T_1, T_2 на новое дерево $T_1 \cup T_2 \cup \{e_k\}$

Замечание : Корректность

Корректность доказывается аналогично корректности алгоритма Прима с использованием леммы о разрезе: если $e_k = (v, w)$ соединяет вершины v и w из разных деревьев T_1 и T_2 текущего леса, то e_k — ребро минимального веса среди рёбер разреза $(W, V \setminus W)$, где W — множество вершин дерева T_1 (все рёбра меньшего веса либо уже добавлены, либо соединяют вершины внутри одного дерева и потому не пересекают этот разрез).

18.4. Система непересекающихся множеств (Union-Find)

Для эффективной проверки «принадлежат ли v и w разным деревьям леса» в алгоритме Краскала используется структура данных СНМ (Union-Find).

Определение

Система непересекающихся множеств (СНМ) — структура данных, поддерживающая разбиение множества элементов $\{x_1, \dots, x_n\}$ на непересекающиеся подмножества с двумя операциями:

1. $\text{union}(x, x')$ — объединяет множества, содержащие x и x' , в одно новое множество, удаляя старые;
2. $\text{find}(x)$ — находит представителя множества, содержащего x .

Утверждение 18.1

Элементы x и y лежат в одном множестве $\iff \text{find}(x) = \text{find}(y)$.

Замечание

Каждый элемент x хранит указатель $x.par$ на родителя. Если $x.par = x$, то x — представитель своего множества. Таким образом множества представляются деревьями (лесом), а $\text{find}(x)$ — это переход по указателям par до корня.

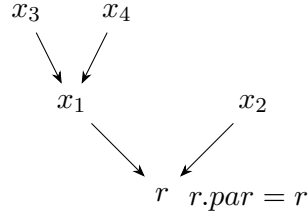


Рис. 35: Множество как дерево: указатели $x.par$ ведут к корню r — представителю множества ($r.par = r$). Операция $\text{find}(x)$ поднимается по указателям от x до корня.

Эвристика 1 (объединение по высоте)

Каждый элемент x хранит поле $x.h$ — высоту дерева с корнем x .

```

function UNION( $x, y$ )
   $x \leftarrow \text{find}(x)$ 
   $y \leftarrow \text{find}(y)$ 
  if  $x = y$  then return
  if  $x.h < y.h$  then
     $x.par \leftarrow y$ 
  else
     $y.par \leftarrow x$ 
    if  $x.h = y.h$  then
       $x.h \leftarrow x.h + 1$ 
end function
  
```

Эвристика 2 (сжатие путей)

```

function FIND( $x$ )
  if  $x.par \neq x$  then
     $x.par \leftarrow \text{find}(x.par)$ 
  return  $x.par$ 
end function
  
```

Лемма 18.2

При использовании эвристики 1 для любого элемента x поддерево с корнем x содержит не менее $2^{x.h}$ элементов.

Доказательство. Индукция по $h = x.h$.

База ($h = 0$): поддерево с корнем x состоит из одного элемента, $2^0 = 1$ — верно.

Переход: значение $x.h$ увеличивается с $h - 1$ до h только в операции union , когда дерево с корнем x высоты $h - 1$ объединяется с деревом высоты $h - 1$, корень которого становится сыном x . По предположению индукции каждое из этих двух поддеревьев содержит не менее 2^{h-1} элементов, поэтому поддерево с корнем x содержит не менее $2^{h-1} + 2^{h-1} = 2^h$ элементов. ■

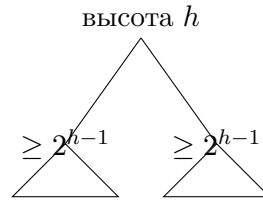


Рис. 36: Дерево высоты h образуется объединением двух деревьев высоты $h - 1$; по индукции каждое содержит $\geq 2^{h-1}$ элементов, а вместе — $\geq 2^h$. Отсюда высота ограничена логарифмом числа элементов.

Теорема 18.2

При использовании эвристики 1 высота любого дерева в СНМ не превышает $\log_2 n$, где n — общее число элементов. Следовательно, операции `find` и `union` выполняются за время $O(\log n)$.

Доказательство. Высота дерева с корнем x не превышает $x.h$. По предыдущей лемме поддерево с корнем x содержит не менее $2^{x.h}$ элементов, а так как всего элементов не более n , имеем $2^{x.h} \leq n$, откуда $x.h \leq \log_2 n$. Значит, высота любого дерева не превышает $\log_2 n$, и обход от произвольного элемента до корня в операциях `find` и `union` занимает $O(\log n)$ шагов. ■

18.5. Оценка сложности алгоритма Краскала

Замечание

Сортировка рёбер занимает время $O(|E| \log |E|)$, каждая операция с СНМ — $O(\log |V|)$, итого $O(|E| \log |E|)$. Так как граф простой, $|E| \leq |V|^2$, а значит $\log |E| = O(\log |V|)$. Поэтому итоговое время работы алгоритма Краскала

$$O(|E| \log |E|) = O(|E| \log |V|).$$

Замечание

С обеими эвристиками (объединение по высоте и сжатие путей) последовательность операций СНМ выполняется за $O(|E| \cdot \alpha(|V|))$, где α — обратная функция Аккермана (растёт крайне медленно). Однако сортировка рёбер всё равно остаётся узким местом, поэтому общая оценка алгоритма Краскала не улучшается.

19. Задача булевой выполнимости (SAT). Задачи k SAT и MAX- k SAT. Вероятностный и детерминированный приближённые алгоритмы для MAX-3SAT

19.1. Определения

Определение

Литерал — переменная x_i или её отрицание $\bar{x}_i$. *Клауза* — дизъюнкция литералов. *КНФ* — конъюнкция клауз.

SAT: Дана КНФ с n переменными и m клаузами. Существует ли набор значений переменных, при котором она истинна? (*NP-полная задача*)

k SAT: SAT для КНФ, где каждая клауза содержит ровно k литералов.

MAX- k SAT: Дана k -КНФ. Найти набор переменных, максимизирующий число истинных клауз. (*NP-полная при $k \geq 2$*)

Определение : Задача MAX-3SAT

Дана булева формула в 3-КНФ:

$$\varphi = C_1 \wedge C_2 \wedge \dots \wedge C_m, \quad C_j = (l_{j,1} \vee l_{j,2} \vee l_{j,3}),$$

где каждый литерал $l_{j,i}$ равен x_s или $\bar{x}_s$ для некоторого s .

Требуется найти набор $x_1, \dots, x_n \in \{0, 1\}$, максимизирующий число выполненных клауз.

19.2. Вероятностный алгоритм

Алгоритм. Каждую переменную x_i выбрать независимо и равновероятно из $\{0, 1\}$.

Лемма 19.1

Пусть X — число выполненных клауз при независимом равновероятном выборе переменных. Тогда:

1. $\mathbb{E}[X] = \frac{7}{8} m$;
2. $\mathbb{P}\left(X \geq \frac{7}{8} m\right) \geq \frac{1}{8m}$.

Доказательство. Часть 1. Клауза C_j ложна тогда и только тогда, когда все три её литерала ложны, что происходит с вероятностью $\frac{1}{8}$. Значит, $\mathbb{P}(C_j \text{ выполнена}) = \frac{7}{8}$. По линейности ожидания: $\mathbb{E}[X] = m \cdot \frac{7}{8}$.

Часть 2. Пусть $p_s = \mathbb{P}(X = s)$ и $p = \mathbb{P}(X \geq \lceil \frac{7m}{8} \rceil) = \sum_{s \geq \lceil 7m/8 \rceil} p_s$. Из $\mathbb{E}[X] = \frac{7m}{8}$:

$$\frac{7m}{8} = \sum_{s=0}^m s p_s = \underbrace{\sum_{s < \lceil 7m/8 \rceil} s p_s}_{\leq (\lceil 7m/8 \rceil - 1)(1-p)} + \underbrace{\sum_{s \geq \lceil 7m/8 \rceil} s p_s}_{\leq m p} \leq (\lceil \frac{7m}{8} \rceil - 1) + m p.$$

Проверим, что $\lceil \frac{7m}{8} \rceil - 1 \leq \frac{7m}{8} - \frac{1}{8}$: пишем $7m = 8q + r$, $r \in \{0, \dots, 7\}$. При $r = 0$: $q - 1 = \frac{7m}{8} - 1 \leq \frac{7m}{8} - \frac{1}{8}$. При $r \geq 1$: $q = \frac{7m-r}{8} \leq \frac{7m}{8} - \frac{1}{8}$.

Подставляя: $\frac{7m}{8} \leq \frac{7m}{8} - \frac{1}{8} + mp$, откуда $p \geq \frac{1}{8m}$. ■

Следствие

Для любой 3-КНФ существует набор переменных, выполняющий не менее $\frac{7}{8}m$ клауз.

Вероятностный $\frac{8}{7}$ -приближённый алгоритм

Генерировать случайные наборы до тех пор, пока число выполненных клауз $\geq \frac{7}{8}m$; вернуть такой набор.

Теорема 19.1

Ожидаемое число итераций описанного алгоритма не превышает $8m$.

Доказательство. Каждая итерация успешна с вероятностью $p \geq \frac{1}{8m}$. Число итераций до первого успеха — геометрическое: $\mathbb{E}[\text{итераций}] = \frac{1}{p} \leq 8m$. ■

19.3. Детерминированный алгоритм (дерандомизация методом условных ожиданий)

Весовая функция

Присваиваем переменным значения по одной: $x_1, x_2, \dots, x_n$. После k -го шага частичное присваивание $x_1, \dots, x_k$ уже зафиксировано.

Определение

Для каждой ещё не выполненной клаузы C при частичном присваивании $x_1, \dots, x_k$ её вес равен

$$w_C = \frac{1}{2^j},$$

где j — число переменных клаузы C , которым ещё не присвоено значение:

$$j = [i_1 > k] + [i_2 > k] + [i_3 > k].$$

Здесь $[\cdot]$ — скобки Айверсона. Суммарный вес:

$$W^{(k)} = \sum_{\text{не выполнена } C} w_C.$$

Замечание

w_C равен вероятности того, что C окажется невыполненной при случайном равновероятном завершении присваивания. Поэтому

$$W^{(k)} = \mathbb{E}[\text{число невыполненных клауз} \mid x_1, \dots, x_k].$$

При $k = 0$: $j = 3$ для каждой клаузы, $w_C = \frac{1}{8}$, и $W^{(0)} = \frac{m}{8}$.

Жадный алгоритм

На шаге $k + 1$ присвоить x_{k+1} то значение из $\{0, 1\}$, при котором $W^{(k+1)}$ минимально.

Утверждение 19.1

При жадном выборе $W^{(k+1)} \leq W^{(k)}$.

Доказательство. Пусть W_0 и W_1 — суммарные веса невыполненных клауз при $x_{k+1} = 0$ и $x_{k+1} = 1$ соответственно. Покажем, что $W_0 + W_1 = 2W^{(k)}$, откуда $\min(W_0, W_1) \leq W^{(k)}$.

Рассмотрим вклад каждой невыполненной клаузы C (с текущим весом w_C) в $W_0 + W_1$:

1. C не содержит x_{k+1} : вес w_C не меняется ни при 0, ни при 1. Вклад: $2w_C$.
2. C содержит позитивный литерал x_{k+1} : при $x_{k+1} = 1$ клауза выполняется (вклад в W_1 равен 0); при $x_{k+1} = 0$ число свободных переменных уменьшается на 1, вес удваивается до $2w_C$. Вклад в $W_0 + W_1$: $2w_C + 0 = 2w_C$.
3. C содержит отрицательный литерал $\bar{x}_{k+1}$: симметрично. При $x_{k+1} = 0$ клауза выполняется (вклад в W_0 равен 0); при $x_{k+1} = 1$ вес удваивается. Вклад в $W_0 + W_1$: $0 + 2w_C = 2w_C$.

Во всех случаях вклад равен $2w_C$, значит $W_0 + W_1 = 2W^{(k)}$, и $\min(W_0, W_1) \leq W^{(k)}$. ■

Корректность

Теорема 19.2

Жадный алгоритм дерандомизации за n шагов находит набор значений, при котором выполнено не менее $\frac{7}{8}m$ клауз.

Доказательство. По предыдущему утверждению $W^{(k+1)} \leq W^{(k)}$ на каждом шаге, поэтому

$$W^{(n)} \leq W^{(0)} = \frac{m}{8}.$$

По завершении ($k = n$) все переменные присвоены: $j = 0$ для каждой клаузы, поэтому $w_C = 1$ для невыполненной и $w_C = 0$ для выполненной. Следовательно,

$$\text{число невыполненных клауз} = W^{(n)} \leq \frac{m}{8},$$

откуда число выполненных $\geq m - \frac{m}{8} = \frac{7m}{8}$. ■

Замечание

Время работы: на каждом из n шагов перебирается два значения и вычисляется W за $O(m)$. Итого $O(nm)$ — полиномиально.

Замечание

Хостад (2001) доказал, что $(\frac{8}{7} - \varepsilon)$ -приближённый алгоритм для MAX-3SAT, работающий за полиномиальное время, влечёт $P = NP$.